PROJECT REPORT · SMART INDIA HACKATHON

SETU

Crowdsourced Road Defect Intelligence for India

Sensor-Enabled Tracking of Urban-road-damage

Millions of phones already ride on Indian roads inside delivery and ride-hailing driver apps. SETU turns them into a free, always-on road inspection network and gives municipal engineers a live map of confirmed potholes ranked by severity.

9,438	+53%	~1.2 cr	~Rs 3
pothole deaths 2020-2024	rise in pothole crashes in 5 years	gig workers = our sensor fleet	our cost to confirm one pothole

THE COST-GATED ESCALATION LADDER

- **LAYER 1** Phone sensors detect a bump · cost ~0 · noisy
- **LAYER 2** Server clusters many vehicles · cost ~0 · kills false alarms
- **LAYER 3** Video + YOLO confirms · costly · used at ~0.1% of places

Problem · research · datasets · architecture · tech stack · roadmap · feasibility · business plan · risk · sources

Generated from PROJECT.md · 13 August 2026

25 sections + appendix

SETU PROJECT REPORT

Contents & reading guide

How to read this document

Every major section carries its own accent colour. That colour is used for the section banner, its table headers, the left edge of its diagrams, the page header rule and the thumb tab printed on the outer edge of each page — so you can find a section by its colour alone when the report is printed.

MARK	MEANING	MARK	MEANING
OK	A fix, a verified claim, or a recommended choice	NO	A problem, a rejected option, or a wrong assumption
!	Caution — a trap, cost or risk to watch	!!	Critical warning
MUST	Compulsory technology (project fails without it)	REC	Strongly recommended
OPT	Optional / nice-to-have	v2	Deferred to a future version
#1	Highest-priority dataset or resource	ADOPT	Something we take directly from prior work

NOTE

Diagrams and code are shown in a monospaced block tinted with the section colour. Long diagrams continue across a page break and keep their alignment. Numbers, quotes and sources are reproduced from the source report without change.

Section colour map

SECTION 1 OF 25

1 EXECUTIVE SUMMARY

What is this in 1 page

One sentence: We turn the millions of phones already riding on Indian roads — inside Swiggy, Zomato, Ola, Uber and Rapido driver apps — into a free, always-on road inspection network, and we give municipal engineers a live map of confirmed potholes ranked by severity.

The three-layer trick that makes it work:

```
LAYER 1 — CHEAP & EVERYWHERE (Sensors)
   Phone accelerometer + gyroscope detect a "bump event".
   Cost: ~0 rupees. Runs on every phone. But it is NOISY.
   Output: "Something abnormal happened at 26.8467 N, 80.9462 E"
                    |
                    v
LAYER 2 — SMART FILTER (Server clustering)
   1 report = noise. 200 reports from 200 different vehicles
   in the same 5-metre spot = almost certainly a real defect.
   Cost: ~0 rupees. Kills 95%+ of false alarms.
   Output: "High-confidence candidate at averaged location"
                    |
                    v
LAYER 3 — EXPENSIVE & CERTAIN (Vision, used rarely)
   ONLY at confirmed candidates do we ask a few phones to record
   a 5-8 second clip. A YOLO model looks at it and says
   "yes, that is a pothole, 40cm wide, severity HIGH".
   Cost: high (data + compute) — which is why we only do it
   at ~0.1% of locations.
   Output: "CONFIRMED pothole, verified visually"
```

Nobody in the published literature does all three layers together with a **cost-gated escalation**. That is our core novelty. Everyone else either does sensors-only (cheap but noisy, lots of false positives) or camera-only (accurate but eats battery and mobile data all day).

What exists on demo day:

- Android app that records sensor data, runs a small on-device model, and pushes positives.
- FastAPI backend + PostgreSQL/PostGIS that clusters reports and issues commands.
- Live React + deck.gl + Google Maps dashboard with municipal admin login.
- YOLO model that confirms potholes from uploaded video frames.
- A "false-positive blacklist" that learns from mistakes (traffic jams, speed bumps, phone drops).

SECTION 2 OF 25

2 THE PROBLEM — WITH REAL NUMBERS

Why this matters

Use these exact numbers with judges. They are from Government of India and World Bank sources.

2.1 Deaths and crashes from potholes (MoRTH data, tabled in Lok Sabha)

Year	Pothole-caused accidents	Deaths
2020	3,713	1,555
2021	—	1,481
2022	—	1,856
2023	—	2,161
2024	5,432	2,385
Total 2020–24	—	**9,438**

- Deaths up **53%** in five years. Accidents up **53%** (3,713 → 5,432).
- Injuries in the same period: **over 19,000**; 4,643 in 2024 alone.
- That is roughly **6+ deaths every single day** from a hole in the road.
- **Uttar Pradesh alone: 5,127 deaths** — more than half the national total.
- Peak year on record was **2017 with 3,597 deaths**.

2.2 The data-quality scandal hiding inside those numbers

More than half a dozen states and UTs — including **Andhra Pradesh, Bihar, Goa and Chandigarh** — reported **ZERO** pothole crashes, injuries or deaths.

This is not because their roads are perfect. It is because a policeman filling an FIR writes "over-speeding" or "driver error", not "pothole". **The real number is certainly much higher than 9,438.**

> ***This is our single strongest pitch line:*** *"India does not have a pothole data problem, it has a pothole DATA problem. Nobody actually knows where the potholes are. We are building the measuring instrument."*

2.3 Money

Metric	Value	Source
Road crashes cost to Indian economy	**3–5% of GDP per year**	World Bank
Total road deaths per year (all causes)	~150,000 killed, ~450,000 injured	World Bank / MoRTH
India's share of global crash deaths	~11% (with ~1% of world's vehicles)	World Bank
If India halves road deaths 2014–2038	**+14% GDP per capita**	World Bank
BMC cost to fill ONE pothole (RTI)	**₹17,693**	RTI reply, 2019
Bengaluru: Shivajinagar ward	₹60,344 per pothole (232 potholes, ₹1.4 cr)	Bangalore Mirror
Bengaluru: CV Raman Nagar ward	₹20,028 per pothole (699 potholes, ₹1.4 cr)	Bangalore Mirror
Mumbai pothole repair tenders	₹203 cr (2023) → ₹156 cr (2024) → ₹90 cr (2025)	BMC / TOI
Nagpur (NMC)	₹10 crore over 3 years for 19,142 potholes	TOI
Indore (IMC)	₹50 crore/year pothole budget (~₹14 lakh/day)	TOI
Pune road-repair vehicles	₹1.10 crore/year *each* to run	Pune Times Mirror

Read the Bengaluru row again. Two wards, same ₹1.4 crore. One fixed 232 potholes, the other fixed 699. That is a **3x difference in cost per pothole in the same city**. There is no independent measurement, so there is no accountability.

> ***Second strongest pitch line:*** *"We are not selling a pothole detector. We are selling an audit trail. Our timestamped before/after data makes contractor payments verifiable for the first time."*

2.4 Why current methods fail

Method used today	Why it fails
Manual visual inspection by junior engineer	One person, one vehicle, subjective. A city has 1,000+ km of road. Takes weeks.
Citizen complaint apps (Mhari Sadak, Rajmargyatra, MoRTH grievance portal)	Reactive. Depends on an angry citizen bothering to file. No coverage guarantee. Duplicate reports. Easy to game.
Network Survey Vehicles (NSV) — NHAI deployed these across 23 states for ~20,933 km	Excellent accuracy, but a specialised vehicle costs crores and can only cover a road **once or twice a year**. Useless for a pothole that appears in July and kills someone in August.
NHAI's planned AI dashcam programme (40,000 km, 30+ defect types)	Great direction — but it is **highways only**. City roads, where most two-wheeler deaths happen, are still uncovered.

The gap we fill: high *frequency* (daily), high *coverage* (every road a delivery rider uses), at near-zero marginal cost. We are not competing with NSVs on accuracy — we are competing on **refresh rate**.

SECTION 3 OF 25

3 MOTIVATION BEHIND THIS IDEA

Why we picked this

3.1 The honest origin story (tell this to judges — it works better than jargon)

> *"Every one of us has been on a two-wheeler that hit a pothole. You feel it in your spine. Then you look around and realise: a thousand other people hit that same hole today, and every single one of them had a phone in their pocket that felt it too — and threw that information away. The measurement already exists. Nobody is collecting it."*

3.2 Five reasons this specific idea is strong

1. The sensor is already paid for. Every smartphone has a 3-axis accelerometer and gyroscope. We do not need to manufacture, deploy, or maintain any hardware. Compare this to IoT-sensor-on-lamppost projects (thousands of devices, SIM cards, batteries, theft, maintenance).

2. The data collector is already driving. This is the real insight. A gig worker does not need to be *motivated* to collect data — they are already covering 100+ km/day on exactly the roads that matter, on a two-wheeler (the vehicle most sensitive to potholes and most likely to be in a fatal pothole crash). Zero incentive cost.

India's gig workforce (our sensor fleet):

Metric	Number	Source
Gig workers in India, FY25	**~1.2 crore (12 million)**, up 55% from 77 lakh in FY21	Economic Survey
Swiggy delivery partners	~690,000 (up 32% YoY)	ET
Zomato monthly active delivery partners	~532,000	Company disclosure
Food delivery sector workers, FY24	1.37 million	Prosus report
NITI Aayog projection for 2029–30	**2.35 crore gig workers**	NITI Aayog

Even at **0.1% adoption**, 12,000 phones × 60 km/day = **720,000 km of road scanned per day.** No government survey fleet on Earth can match that.

3. Crowd redundancy beats sensor precision. This is the mathematical heart of the project. A ₹8,000 phone's accelerometer is a bad instrument. But **1,000 bad instruments measuring the same thing beat one good instrument**, because random errors cancel out and only the real signal survives. We are trading hardware quality for sample size — and sample size is free.

4. SDK distribution is the only realistic path to scale. A standalone "Pothole App" is dead on arrival. No citizen will keep it installed. But if the detection logic ships as a **200 KB SDK inside apps drivers already keep open all day**, we get national coverage without asking a single person to download anything. This is why we frame the hackathon build as a *demo shell around an SDK*, not as a consumer app.

5. It fits India's existing policy push. Smart Cities Mission, NITI Aayog's frontier-tech push, iRASTE in Nagpur, NHAI's NSV and AI-dashcam programmes — the government is *already buying* road-condition AI. We are not creating a market; we are entering one with a 100x cheaper cost structure.

3.3 Why we personally care (say this out loud)

Every team member on this project rides a two-wheeler. This is not an abstract civic-tech exercise. The problem statement is our commute.

SECTION 4 OF 25

4 WHAT WE ARE ACTUALLY BUILDING

Hackathon demo vs real product

Be completely clear about this internally, and be *honest but confident* about it with judges.

4.1 Hackathon deliverable (the demo)

A **standalone Android app** that acts as a shop-window for the technology. It is not the product. It is proof the product works.

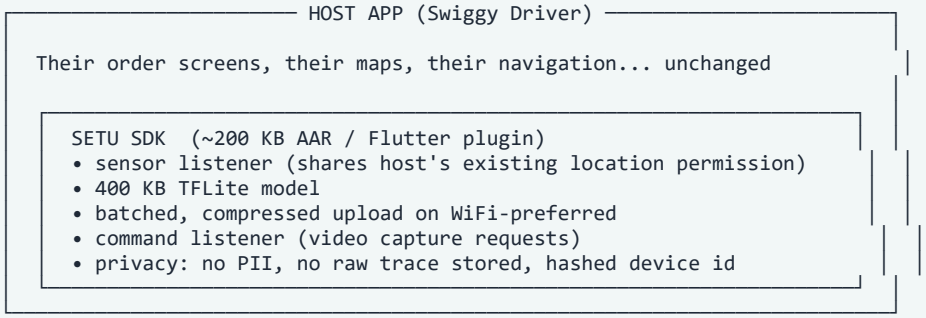
Only feature in normal (idle) mode:

1. Read accelerometer + gyroscope at ~100 Hz in a foreground service.
2. Slide a 2-second window over the stream.
3. Run a small on-device TFLite model on each window.
4. If model says "pothole-like" with confidence above threshold → send `{lat, lon, timestamp, confidence, speed, device_class, session_id, counter=1}` to the server.
5. Do nothing else. No camera. No upload. Battery cost near zero.

Everything else the app does is triggered by a server command. The app polls / holds a socket and waits. This is a deliberate design choice: it keeps the app dumb, keeps the intelligence in the cloud where we can update it without an app release, and keeps battery/data use minimal.

4.2 The real product (what we tell judges)

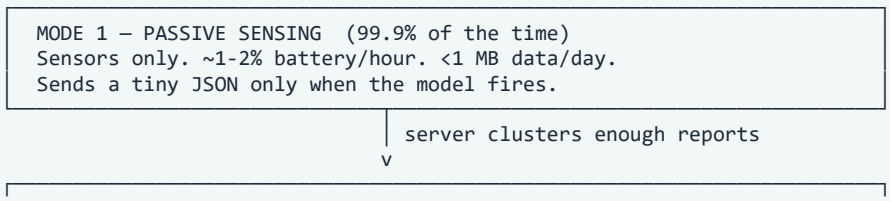
An **SDK / embeddable module** — think of it as "Google Analytics, but for road quality" — that a company like Swiggy or Ola drops into their existing driver app in an afternoon.



Why an SDK and not an app — the four-line answer for judges:

1. Distribution: 690,000 Swiggy riders instantly, vs 0 downloads for a new app.
2. Permissions: the host app already has background location. We inherit it, we don't beg for it.
3. Retention: nobody uninstalls our SDK, because they can't see it.
4. Business: we sell to ~10 platform companies, not to 100 million citizens. Vastly cheaper go-to-market.

4.3 The three operating modes of the app



```
MODE 2 — HUMAN CONFIRMATION  (rare)
Rider stops / is within geofence → notification:
"Is there a pothole near [landmark]?  [Yes] [No] [Not sure]"
One tap. Adds a human vote to the cluster.
! NEVER shown while the vehicle is moving. Safety-critical rule.
                              |
                              | candidate needs visual proof
                              v
MODE 3 — VISUAL CAPTURE  (very rare, ~0.1% of locations)
Geofence trigger 60-80 m BEFORE the target location.
Record 5-8 s @ 30 fps, 720p, rear camera, no audio.
Upload to pre-signed S3 URL when on WiFi or good 4G.
DELETE local file immediately on HTTP 200. Verified by checksum.
```

The geofence maths for Mode 3 — get this right, it is the detail judges will probe.

A rider at 40 km/h covers **11.1 m per second**. To have the target defect appear in the *middle* of a 6-second clip, recording must start ~33 m before it. But camera cold-start takes 0.5–1.5 s, so add 17 m. Total trigger distance **≈ 50 m**, and we widen to 60–80 m for safety.

Speed	m/s	Distance covered in 6 s clip	Trigger distance (target at mid-clip)
20 km/h	5.6	33 m	~30 m
40 km/h	11.1	67 m	~50 m
60 km/h	16.7	100 m	~70 m

→ So the trigger radius must be **speed-adaptive**, computed as `distance = speed × (clip_length/2 + camera_warmup)`. A fixed 5–10 m radius (our original idea) is **far too small** — at 40 km/h the rider crosses a 10 m geofence in under one second, which is not enough time to even open the camera. **This was a real bug in our initial plan; it is now fixed.**

SECTION 5 OF 25

5 RESEARCH PAPER DEEP-DIVE

What science already says

We read the literature so we don't repeat mistakes people already documented. Below is every useful paper, what it proved, and what we take from it. Full links in **Section 25**.

5.1 Sensor-based detection (the accelerometer track)

#	Paper	Method	Reported result	What WE learn
P1	**Sattar, Li & Chapman (2018)** — *Road Surface Monitoring Using Smartphone Sensors: A Review*, Sensors 18:3845	Survey of the whole field	—	The canonical review. Confirms the three families: threshold-based, ML-based, deep-learning-based. Start here.
P2	**Jan et al. (2023)** — *Crowdsensing for Road Pavement Condition Monitoring: Trends, Limitations, Opportunities*, IEEE Access 11:133143	Survey	—	Names the exact open problems: vehicle heterogeneity, phone placement, GPS error, no standard dataset, no incentive model. **Our project is basically an answer sheet to this paper.**
P3	**Khandakar et al. (2025)** — *Harnessing Smartphone Sensors for Enhanced Road Safety*, Scientific Data 12:418	Released a 10-sensor dataset from Rajshahi, Bangladesh	Bump vs pothole z-axis difference statistically significant: **T = −3.3488, p = 0.0008**. PCA components separate the two classes (Mann-Whitney U + KS tests significant)	**Our #1 dataset.** Proves potholes and bumps *are* separable from IMU alone. Sampling rate **89.82 Hz average (range 60–99 Hz)** — sets our target rate. Also gives us the crucial engineering detail: potholes show **higher mean, variance, range and std-dev** than bumps on the z-axis.
P4	**MDPI Sensors (2020) 20(19):5564** — *An Automated Machine-Learning Approach for Road Pothole Detection Using Smartphone Sensor Data*	AutoML over many classifiers	**Random Forest best: precision 88.5%, recall 75%.** Time-domain + frequency-domain features beat everything else	Two big lessons: (a) start with Random Forest, not deep learning; (b) **recall is the weak point (75%)** — sensors miss 1 in 4 potholes. Crowd redundancy is how we fix that: if one rider misses it, the next 50 won't all miss it.
P5	**Springer / J. Inst. Eng. India (2025)** — *Smartphone-Sensor Based Dynamic Time Warping Framework for Enhanced Pothole Detection*	DTW template matching, tested in 3 Indian cities	Post-validation accuracy: **Delhi 98.04%, Srinagar 97.02%, Rajasthan 91.02%**; precision 84.05%, recall 85.71%	The most India-relevant sensor paper. Note the **7-point accuracy drop between Delhi and Rajasthan** — this is *domain shift* from road type and vehicle mix. **Our model must be evaluated per-city, never on a single blended test set.**
P6	**Efficient pothole detection using smartphone sensors** (ResearchGate/conf.)	Neural network on accel+gyro	**94.78% classification accuracy**	Confirms accel+gyro fusion > accel alone. Gyroscope catches the *roll* asymmetry when only one wheel drops in.
P7	**ASCE (2019)** — *Smartphone-Based Pothole Detection Utilizing Artificial Neural Networks*	ANN on 4 engineered metrics	**~90% detection accuracy**	You don't need a huge model. Four good features get you to 90%. Feature engineering beats architecture hunting.
P8	**Springer Multimedia Tools (2023)** — *Road pothole detection from smartphone sensor data using improved LSTM*	LSTM sequence model	Improved over baselines	Sequence models help because a pothole is a *shape in time* (drop-then-rebound), not a single spike. Our CNN-over-window approach captures the same idea more cheaply.
P9	**Seraj et al. — RoADS** (Springer, 2016)	Wavelet decomposition + SVM	Real-time anomaly detection	Wavelets are excellent for transient shock events. Good fallback feature set if raw-window CNN underperforms.

#	Paper	Method	Reported result	What WE learn
P10	**Celaya-Padilla et al. (2018)** — *Speed Bump Detection Using Accelerometric Features: A Genetic Algorithm Approach*, Sensors 18(2):443	GA-selected features + logistic model	Speed bumps specifically detectable	**The single most important paper for our false-positive problem.** Speed bumps are our worst enemy and they are detectable as their own class. So: make speed bump a **positive label**, not "noise".
P11	**CEUR-WS Vol-2227 (KDD 2018 workshop)**	Analysis of FP sources	States plainly: road infrastructure has speed bumps and other structures producing vibrations that are **not defects**, and this "produces a large number of false positives, which degrades the accuracy of the pothole detection"	Confirms our biggest risk in a peer-reviewed sentence. Quote this to judges.
P12	**Fox et al. (2015)** — *Crowdsourcing undersampled vehicular sensor data for pothole detection*, IEEE SECON	3 crowdsourcing methods; trained on **CarSim simulated data** because real data was scarce	Handles vehicle differences, async sensors, GPS error, noise	Validates our whole thesis: **crowdsourcing works specifically because individual samples are undersampled and noisy.** Also gives us a trick — use simulation to bootstrap training data.
P13	**Fox et al. (follow-up)** — multi-lane pothole localisation using road inclination and bank angle	Adds road geometry	Better multi-lane discrimination	**Lane-level detail we had not considered.** A pothole in lane 1 is not the same defect as one in lane 3. Advanced feature for v2.
P14	**MDPI Sensors (2020) 20(2):409** — *Accuracy Enhancement of Anomaly Localization with Participatory Sensing Vehicles*	Statistical model to correct localisation	Found **error spread can exceed 32 m and mean localisation error can exceed 27 m at highway speeds** — "such large errors can make the application impractical for widespread use"	**!! The most important number in this entire report.** Our original plan assumed 5 m clustering radius. Published measurement says the error alone can be 27–32 m. **Our design must change** (see Section 8).
P15	**Martinelli et al. (2022)** — *Road Surface Anomaly Assessment Using Low-Cost Accelerometers: A ML Approach*, Sensors 22:3788	ML on cheap accelerometers	Practical accuracy	Confirms cheap hardware is sufficient if ML compensates.
P16	**Nature Sci. Rep. (2024) s41598-024-61757-1** — *Evaluation of data representation techniques for vibration based road surface condition classification*	Compares representations across **4 classes: normal, pothole, bad road surface, speedbump**	Representation choice matters more than classifier	**Adopt this exact 4-class scheme (we extend to 6).** Also: "bad road surface" as a separate class is smart — continuous roughness ≠ discrete pothole.
P17	**MDPI Applied Sciences (2024) 14(21):10027** — *Accelerometer-Based Pavement Classification Using Neural Networks*	NN vs multinomial logistic regression	NN **100% accuracy**, logistic 97.14%	**! Treat 100% as a red flag, not a win.** 100% on a small single-vehicle dataset = leakage or overfitting. This is a warning about how *we* must split our data (by route and by device, never randomly).
P18	**González et al.** — Chihuahua, Mexico dataset (12 cars and trucks)	Comparative benchmark	—	**The dataset is no longer publicly available.** Cited by Khandakar et al. as a field-wide problem. Lesson: publish our dataset properly and permanently — it is a differentiator *and* a contribution.
P19	**Carlos et al. — Pothole Lab**	Open-access web platform to synthesise virtual roads with configurable anomalies	Enables reproducible evaluation	Free synthetic data generator for augmentation and for testing our clustering logic without driving.
P20	**Nature Sci. Rep. (2026) s41598-025-34396-3** — road roughness via smartphone + SVM	4 vehicles, 50-m segments	Discriminates roughness/quality levels **80–100%** of the time; "even low-frequency smartphone IMU signals can provide useful roughness screening"	Two gifts: (a) **50-m segment aggregation** is a proven unit of analysis — better than point-clustering; (b) low sampling rates still work, so we can drop to 50 Hz on weak phones to save battery.

#	Paper	Method	Reported result	What WE learn
P21	**Taylor & Francis** — *Influence of surface distresses on smartphone-based pavement roughness evaluation*	Smartphone IRI vs Roughometer	Correlation **r = 0.862**; including distresses **increased average IRI by 61.8%**	Lets us output a *standards-compatible* IRI-like number, which is what engineers actually procure against.
P22	**MDPI Sensors (2018) 18(3):914** — bicycle-mounted smartphone IRI mapping	GPS + accel on a bicycle	Strong positive correlation with professional instruments; also recognised potholes/humps	Two-wheeler-mounted sensing is scientifically validated. Directly supports our delivery-rider model.
P23	**arXiv 2508.16626 — PoDAS**	Low-cost wireless sensor network, city-deployable, multiple implementation models	End-to-end system proposal	A competing architecture (dedicated sensors). Our answer: they need hardware per vehicle; we need zero.
P24	**arXiv 2606.03427** — *Multi-Modal Assessment of Road Roughness Using Smartphone Applications, Acceleration, and Passenger Ratings*	IRI vs Present Serviceability Rating vs accel	Significant **inverse** IRI↔PSR relation; **positive** IRI↔vertical acceleration relation	Scientific licence to blend machine measurement with human ratings — exactly what our "Yes/No" prompt does.
P25	**Ferreira et al. (2017)**, PLoS One 12:e0174959 — driver behaviour profiling with smartphone sensors	Multi-sensor + ML	Behaviours classifiable	Adjacent revenue stream: the same data stream scores driving behaviour (insurance/fleet product).

5.2 Vision-based detection (the camera track)

#	Paper / Resource	Key numbers	What WE learn
V1	**Arya, Maeda, Ghosh, Toshniwal, Sekimoto (2022)** — *RDD2022: A multi-national image dataset for automatic Road Damage Detection*, arXiv 2209.08538 (IIT Roorkee + University of Tokyo)	**47,420 images, 6 countries (Japan, India, Czech Republic, Norway, USA, China), 55,000+ annotated damage instances, 4 classes: D00 longitudinal crack, D10 transverse crack, D20 alligator crack, D40 pothole**	**BEST Our primary image dataset.** It has an India split, it is multi-country (so we can prove generalisation), and it is the CRDDC2022 challenge benchmark, so published scores are directly comparable to ours.
V2	**Road Damages Detection and Classification with YOLOv7** (CRDDC2022 entry)	**F1 81.7%** on US Google Street View data, **74.1%** on all test images. Used coordinate attention, label smoothing, ensembling	Sets our realistic target. Anyone claiming 99% on road damage is measuring wrong. Also: **Google Street View is a legitimate free source of labelled-able road imagery.**
V3	**Springer (2024)** — *Road damage detection and classification using deep neural networks*	**65.7% mAP on RDD2022**, beating Faster R-CNN and SSD	Confirms YOLO-family > two-stage detectors for this task.
V4	**YOLOv8-PD**, Nature Sci. Rep. (2024) s41598-024-62933-z	**2.3 M params, 6.1 GFLOPs** (74.1% / 74.3% of baseline), mAP **+1.4 pp**	Proof you can shrink and *gain* accuracy. Relevant if we ever move vision on-device.
V5	**YOLO-ROC**, arXiv 2507.23225	**mAP50 67.6%** on RDD2022_China_Drone (+2.11% over YOLOv8n); **D40 pothole (small target) mAP +16.8%**; final model **2.0 MB**	**!!** Note *why* they needed a special model: **potholes are the hardest, smallest class.** Their +16.8% on D40 specifically tells us small-object detection is our main vision challenge. A 2 MB model is a huge finding for on-device v2.
V6	**YOLO-RD**, MDPI Sensors (2025) 25(5):1442	Japan RDD2022 detection accuracy **25.75%**, **+4.93% on small objects** vs YOLOv8	The low absolute number is a reality check: on hard splits, road damage detection is genuinely hard. Do not promise judges 95%.
V7	**TD-RD**, arXiv 2501.14302	**7,088 high-res top-down images, 12,882 instances**, classes: cracks, potholes, patches	A *top-down* view dataset. Useful for augmentation diversity, and it includes "patches" — repaired areas — which we need to detect repairs for our audit-trail feature.
V8	**RAD — Road Anomaly Detection dataset (Bengaluru)**, Springer (2024)	Indian road damages + **pothole depth estimation** via smartphone camera	An **Indian-specific** dataset with *depth*. Depth → severity → repair priority. Directly powers our severity score.
V9	**MIIA Pothole Image Dataset** (Machine Intelligence Institute of Africa, 2019)	Classification challenge dataset	Extra negatives from a developing-country road context, similar to India.
V10	**Chitholian pothole dataset** (via Roboflow)	**665 annotated potholes**, re-split 70/20/10	Small but clean; good for a quick baseline in week 1.
V11	**Roboflow Universe public pothole dataset**	~1,200–4,000 images depending on version, YOLO format ready	Fastest possible start. One-line download.

#	Paper / Resource	Key numbers	What WE learn
V12	**RF-DETR + ByteTrack pothole pipeline** (Roboflow, 2026)	Detector + tracker gives **persistent IDs, unique pothole counting, severity flags**	**ADOPT Directly adopt this.** In a 6-second 180-frame clip, the *same* pothole appears in ~40 frames. Without tracking we would count it 40 times. ByteTrack fixes this. **We had this bug in our original plan.**
V13	**MIT Carbin app** (MIT Concrete Sustainability Hub)	**250,000+ miles collected since 2019**; delivers agency-grade data at lower cost, in real time	Existence proof from MIT that crowdsourced road data is credible to agencies. Great slide for judges.

5.3 What is deployed in India right now (competitive landscape)

Player	What it does	Our position vs them
NHAI Network Survey Vehicles — deployed across 23 states, ~20,933 km of highways, collecting surface cracking, potholes, patches	Gold-standard accuracy, laser + camera rigs	We do not compete on accuracy. We compete on **frequency** (daily vs annual) and on **city roads** (they do highways).
NHAI AI-dashcam programme — planned across ~40,000 km, AI detecting **30+ defect types** (potholes, cracks, rutting, faded lane markings, damaged crash barriers, non-functional streetlights)	Very close to our vision layer	Validates our approach at national policy level. But again: **highways only, dedicated cameras.** Also gives us a 30-defect taxonomy to grow into.
RoadBounce (Pune, founded 2016)	Phone-based vibration roughness measurement, IRI, suspension testing. Independently tested for accuracy across vehicles/phones/speeds/tyre pressures	Closest commercial competitor. They sell **surveys** (someone must drive with the app on). We sell **passive continuous data** with no dedicated driver.
RoadMetrics (Bengaluru)	AI road-defect detection; **50,000+ km mapped**; adopted by **Chennai** and other city governments; featured by NITI Aayog	Proves municipalities will pay. Also proves procurement is possible for a startup.
iRASTE, Nagpur (INAI/IIIT-H + NMC + Intel)	AI road safety, ADAS on municipal vehicles, black-spot identification, targeting **~50% reduction in crashes**, collision avoidance claimed to cut accidents/near-misses **up to 60%**	Complementary, not competing. They fix *behaviour* and *black spots*; we fix *surface*. Potential partner.
Citizen complaint apps — Mhari Sadak (Haryana), Rajmargyatra (NHAI), MoRTH grievance portal, NHAI helpline	Manual citizen reporting; Haryana even ordered **monthly pothole review by Deputy Commissioners**	These are our *distribution partners*, not competitors. Government already wants this data; they just have a terrible collection mechanism.
IJRASET (2026) — Smart Pothole Detection & Mapping for **NMC** (Nagpur Municipal Corporation): YOLOv8 + GIS dashboard	Academic pilot with a real municipality	Nearly identical to our web portal. Confirms the deliverable format municipalities expect: **map + prioritised list**.

SECTION 6 OF 25

6 GAPS IN EXISTING RESEARCH + OUR UNIQUE EDGE

Where we win

Reading 38+ papers, the same holes appear again and again. Each gap is an opportunity.

GAP 1 — Everybody detects, nobody *confirms*

What's missing: Almost every paper ends at "our classifier achieved X% accuracy on our test set." Not one builds a **multi-stage confirmation pipeline** where a cheap sensor triggers an expensive verifier only when statistically justified.

Our edge — the Cost-Gated Escalation Ladder:

```
Stage 0  Raw IMU window                    cost: ~0        1,000,000 windows/day
Stage 1  On-device model fires              cost: ~0            5,000 events/day
Stage 2  Spatio-temporal cluster forms      cost: ~0              200 candidates/day
Stage 3  Human Yes/No votes collected       cost: 1 tap            50 promoted/day
Stage 4  Video requested + vision confirms  cost: HIGH             20 CONFIRMED/day
```

Precision rises at every stage while cost per confirmed defect *falls*, because we never waste bandwidth on garbage. **This ladder is the intellectual contribution of our project.** Write it on the slide.

GAP 2 — Vehicle & phone heterogeneity is acknowledged, never solved

What's missing: Papers use one car and one phone (Khandakar: Poco X2, one 1995 Toyota. MDPI 14(21):10027: one vehicle). Then they report 100% accuracy and wonder why it fails in the field. The DTW paper's Delhi 98% → Rajasthan 91% drop is exactly this problem leaking through.

Our edge — Per-Device Self-Calibration:

- First 10 minutes of driving = calibration mode. Measure the device's own noise floor on *smooth* road (low variance stretches).
- Normalise every future window by that baseline: `z_normalised = (z - μ_device) / σ_device`.
- Also learn a **vehicle class** from the vibration signature (2-wheeler vs car vs auto have very different natural frequencies) and feed it as a model input.
- Result: a scooter with a hard suspension and a sedan with soft suspension both produce comparable, comparable-scale signals.
- **This is a genuinely publishable contribution.** Nobody in our reading list normalises per-device *and* per-vehicle-class in a deployed crowdsourced setting.

GAP 3 — GPS error is measured and then ignored

What's missing: MDPI Sensors 20(2):409 measured **27–32 m localisation error** and concluded it may make such applications "impractical". Almost every other paper then proceeds to plot single points on a map as if GPS were perfect.

Our edge — three fixes stacked:

1. **Snap to road, not to point.** Use OSRM/OSM map-matching to project every report onto a road segment. This throws away perpendicular error for free.
2. **Aggregate on 20-m road segments** (inspired by the 50-m segments in Sci. Rep. s41598-025-34396-3), not on raw lat/lon points. A segment is robust to noise; a point is not.
3. **Weighted centroid with inverse-variance weighting.** Reports arrive with a GPS `accuracy` field (metres). Weight each report by `1/accuracy²`. A 4 m-accuracy fix counts 100× more than a 40 m fix.

GAP 4 — Negative classes are treated as "noise" instead of as classes

What's missing: CEUR-WS Vol-2227 names speed bumps as a major FP source. Celaya-Padilla proved bumps are separately detectable. Yet most pipelines just threshold and hope.

Our edge — explicit 6-class taxonomy: `0 smooth_road | 1 rough_road | 2 pothole | 3 speed_bump | 4 rumble_strip/joint | 5 non_road_event (phone drop, hard brake, door slam, pocket movement)`

Class 5 is the one nobody publishes and it will be **the biggest source of garbage in a real deployment**. A phone falling off a bike mount produces a bigger spike than any pothole on Earth. We must train on it deliberately.

GAP 5 — Nobody models the *negative* — "there is definitely no pothole here"

What's missing: Every system is a pothole *detector*. None is a road *certifier*. But a municipality needs both: "these 40 spots are broken" AND "these 200 km are verified fine."

Our edge — the Negative Evidence Table. Every silent pass over a segment is *also* data. If 4,000 vehicles crossed segment S in 7 days and 3 fired, that segment is **certified good** with high confidence. This gives us:

- A road **health score** per segment, not just a defect list.
- **Repair verification** (audit trail): defect confirmed on 1 Aug, contractor paid 15 Aug, but reports keep firing on 20 Aug → **the repair failed or never happened.** This single feature is worth more to a municipality than the detection itself, and it is our commercial moat.

GAP 6 — The false-positive *blacklist* idea appears nowhere

What's missing: No paper we found implements negative-feedback geofencing — using a confirmed mistake to suppress an entire area.

Our edge — Adaptive Suppression Zones. Vision AI reviews a clip and sees a traffic jam, a signal, a toll plaza, a construction barricade, or a railway crossing. We then create a suppression polygon of 50–100 m around it with a **7-day TTL**, and down-weight (never fully drop) new reports there. Suppression is **stored with a reason code and an expiry**, so it is auditable and self-healing.

Refinement we should make explicit: use **decayed down-weighting (×0.2), not a hard block**, and never suppress reports that carry a *human YES vote*. Otherwise one bad clip could permanently hide a real pothole 60 m from a traffic light — a genuinely dangerous failure mode.

GAP 7 — Datasets vanish

What's missing: González's Chihuahua dataset is gone. Khandakar et al. explicitly complain that most datasets are simulated or no longer accessible, and that "there was no international authority in charge of the data collection and sharing process."

Our edge: Publish **SETU-IND-1**, an open Indian sensor dataset (accel/gyro/GPS/label/vehicle-class/phone-model) on Figshare/HuggingFace with a DOI and a permissive licence. Costs us nothing. Buys enormous credibility, and is a legitimate research contribution from a hackathon team.

GAP 8 — Incentives and privacy are never designed

What's missing: Jan et al. (2023) lists lack of an incentive model as an open problem. No paper addresses India's DPDP Act.

Our edge: The SDK model *removes the need for incentives entirely* — the rider is already paid to drive. And we design for DPDP compliance from day one (Section 18.3), which is exactly what an enterprise procurement team will ask about first.

GAP 9 — Repairs are invisible; only damage is studied

What's missing: TD-RD includes a "patches" class but nobody builds a **lifecycle** model.

Our edge — defect state machine: `CANDIDATE → CONFIRMED → REPORTED_TO_ULB → UNDER_REPAIR → REPAIRED → RE-OPENED → CLOSED` with timestamps at each transition. That produces a **Mean Time To Repair (MTTR)** metric per ward and per contractor. Remember the Bengaluru data: ₹60,344/pothole in one ward vs ₹20,028 in another. **MTTR + cost-per-pothole per ward is the report that makes a Municipal Commissioner sign a cheque.**

GAP 10 — Severity is binary; it should be actionable

What's missing: Most work outputs "pothole / no pothole". A road engineer cannot act on that.

Our edge — composite severity score aligned to Indian standards. IRC:82-2015 (*Code of Practice for Maintenance of Bituminous Road Surfaces*) already defines maintenance triggers by distressed area percentage (e.g. potholes >0.5% of area). So we output:

```
Severity = w1·(impact magnitude, from IMU peak & jerk)
         + w2·(estimated size/depth, from vision + RAD depth method)
         + w3·(traffic volume, = report density on that segment)
         + w4·(vulnerability, = share of 2-wheeler reports)
         + w5·(persistence, = days unrepaired)
→ mapped to IRC-style bands and to a repair priority (P1/P2/P3)
```

Weighting by **two-wheeler share** is our own idea and it is defensible: two-wheelers are the vehicles that die in pothole crashes, so a pothole on a scooter-heavy route deserves higher priority than an identical one on a truck route.

GAP 11 — Sensor and vision are never fused into one confidence number

Our edge — Bayesian fusion. Treat each evidence type as a likelihood update on a single posterior:

```
P(defect | evidence) ~ P(defect)
   × Π P(sensor_report_i | defect)   [weighted by device trust score]
   × Π P(human_vote_j  | defect)     [weighted by user reputation]
   × P(vision_verdict  | defect)     [strongest single term]
```

One number, 0–1, explainable, and it lets us show a municipality *why* something is confirmed. Explainability is a procurement requirement, not a nice-to-have.

GAP 12 — No anti-gaming / Sybil defence

What's missing: Every crowdsourcing paper assumes honest participants. Once money is attached to reports, that assumption dies.

Our edge — device trust scores. Each device has a reputation, updated by how often its reports survive to CONFIRMED. Plus physics checks: was the phone actually moving (GPS speed > 5 km/h)? Is the trajectory continuous? Does the accelerometer signature match a vehicle at all? Reports from stationary or teleporting devices are dropped.

SECTION 7 OF 25

7 INDIA vs OTHER COUNTRIES

Global comparison

Dimension	India	Japan	USA	Europe (Norway / Czech)	Africa
Dominant vehicle in crashes	Two-wheelers — extremely vulnerable to potholes	Cars	Cars/trucks	Cars	Mixed
Road surface	Mostly bituminous, patch-heavy, monsoon-destroyed	High-quality, well-maintained	Concrete + asphalt, freeze-thaw cracking	High quality, frost damage	Often unpaved/gravel
Damage type that dominates	**Potholes + patches**	Cracks (longitudinal/alligator)	Cracks + freeze-thaw potholes	Cracks, frost heave	Erosion, gravel loss
Data availability	Poor; states report "zero" pothole deaths	Excellent (RDD/Maeda lineage from Univ. of Tokyo)	Good (Street View, state DOT PMS)	Good	Very poor
Public dataset presence	India split in RDD2022; RAD (Bengaluru); mostly small academic sets	Largest and cleanest splits	Large, Street View-derived	Present in RDD2022	MIIA only
Standard framework	IRC:82-2015, IRC:SP:83-2018, PCI studies	Municipal manuals, well digitised	PCI / ASTM D6433, IRI-driven PMS	IRI-driven asset management	Weak
Typical published accuracy	Sensor 91–98% (DTW, city-dependent)	Vision F1 ~74–82% on RDD	Vision F1 ~81.7% (Street View)	Similar to RDD baselines	Low
Repair loop	Weeks to months, reactive, complaint-driven	Days, scheduled, preventive	Weeks, budgeted PMS cycles	Scheduled asset management	Ad hoc

What this comparison teaches us

1. **Do not copy a Japanese model architecture and expect it to work here.** Japan's dominant class is cracks; India's is potholes and patches. RDD2022's own paper separates by country for exactly this reason. **Always fine-tune on the India split and report India numbers separately.**
2. **India's defect *density* is our advantage, not our problem.** Western crowdsourcing struggles because potholes are rare, so you need enormous mileage to find any. In India, a rider hits a defect every few hundred metres. **We reach statistical confidence far faster than a US or EU deployment could.** Say this to judges — it reframes a national embarrassment as a technical moat.
3. **The two-wheeler is a better sensor platform than a car.** A car's suspension and cabin mass *damp* the signal; a scooter transmits it almost directly to the rider's pocket. Papers using 1995 Toyotas are working with a low-pass-filtered version of the truth. Bicycle-mounted sensing was already validated in MDPI Sensors 18(3):914. **India's two-wheeler dominance gives us a higher signal-to-noise ratio than any Western study.** This is a genuine, defensible, unique insight.
4. **The West optimises IRI (comfort/asset life). India must optimise fatalities.** So Western severity models weight ride quality; ours must weight *danger to a two-wheeler*. Different objective function → different product. This justifies building something new rather than importing RoadBounce-style IRI reporting wholesale.
5. **Monsoon seasonality is India-specific.** No RDD2022 country has a 3-month window in which the entire road network degrades. Our system must be **seasonally aware**: expect a report surge in June–September, and treat pre-monsoon vs post-monsoon segment health as a *predictive* signal. Nobody in the literature models this. **Potential for a "predict where potholes will form" model in v2** — trained on pre-monsoon roughness trend + previous-year defect history + drainage proximity.

SECTION 8 OF 25

8 WHERE OUR CURRENT PLAN IS WRONG (HONEST CRITIQUE)

Honest critique

This section exists because judges *will* find these. Better we find them first, fix them, and present the fix as evidence of rigour.

WRONG 1 — "10,000+ reports in a 5-metre radius"

The problem: Two separate errors.

- **5 m is smaller than the GPS error.** Published mean localisation error is **27–32 m at highway speed** (MDPI Sensors 20(2):409). At 5 m you would never form a cluster at all; reports from the same real pothole would scatter across 6+ different 5 m bins.
- **10,000 is wildly too high.** At even generous adoption, a single city segment might see 500 passes/day of which perhaps 3% fire. Waiting for 10,000 hits on one pothole could take months. The pothole will have swallowed a bike by then.

FIX:

Parameter	Original	Corrected	Why
Clustering unit	5 m radius circle	**20 m road segment** (map-matched) + DBSCAN `eps ≈ 15–25 m`	Bigger than GPS error; aligned with 50 m segments validated in Sci. Rep. s41598-025-34396-3
Reports to become CANDIDATE	10,000	**≥ 8 reports from ≥ 5 distinct devices within 7 days**	Distinct-device count is what actually kills noise, not raw volume
Reports to request video	—	**≥ 15 reports OR ≥ 3 human YES votes**	Escalate only when it's worth the bandwidth
Reports to CONFIRM	—	**vision verdict positive on ≥ 2 independent clips**	Two clips from two riders ≈ certainty

Key principle: count DISTINCT DEVICES, not reports. One rider passing the same pothole 40 times on their daily route must count roughly once, not 40 times. Otherwise a single rider with a loose phone mount can manufacture a fake pothole. Use `COUNT(DISTINCT device_id)` everywhere, and cap per-device contribution per cluster (e.g. max 3).

WRONG 2 — "Show a notification to people passing within 5–10 metres"

The problem: Three sub-problems.

- 5–10 m is inside GPS error; the geofence will fire late, early, or never.
- **Asking a rider a question while they are riding is dangerous.** If our app causes one crash, the project is over — legally and morally.
- Riders are mid-delivery, on a clock. Interrupting them is exactly how the host platform (Swiggy) rips out our SDK.

FIX:

- Geofence radius **50–100 m** for detection of "near", not 5–10 m.
- **Never prompt while moving.** Queue the question. Fire it only when `GPS speed < 3 km/h for > 5 s` (stopped at a light or delivery point) or **at end of shift** as a batched "You passed 3 possible potholes today — help us verify?" screen with a small thumbnail map.
- Cap at **≤ 2 prompts per rider per day.** Anything more and engagement collapses.
- Add a **"Not sure"** button. Forcing a binary Yes/No pollutes the data with guesses.

WRONG 3 — "The app records video automatically"

The problem: This is the single biggest risk in the whole project, on four fronts.

1. **Legal/privacy:** silently recording video from a person's phone captures faces, licence plates, shopfronts, homes. Under the **DPDP Act 2023** (in force, with DPDP Rules notified 13 Nov 2025, phased compliance and Schedule-1 penalties up to **₹250 crore** effective 13 May 2027), video containing identifiable people is personal data requiring **specific, informed, purpose-limited consent**. "Automatically, without the user knowing" is not a defensible posture.
2. **Physical:** a phone in a pocket or a bag records darkness. A phone in a bike mount often faces the rider, not the road.
3. **Cost:** 6 s of 720p30 ≈ 6–12 MB. Gig workers pay for their own data.

4. **Platform risk:** no Play Store reviewer or enterprise partner will accept silent background camera access. Android's own permission model increasingly forbids it.

> **FIX — reframe from "automatic" to "opted-in and visible":**

- **Explicit opt-in** at onboarding: a separate toggle, "Help verify road damage with short video clips", default **OFF**, with a plain-language explanation.
- **Persistent visible indicator** while recording (Android shows a camera indicator anyway from Android 12+; we add our own banner).
- **Only from a mount.** Detect stable landscape orientation + steady gravity vector; if the phone isn't mounted, silently skip. This alone removes most useless clips.
- **On-device pre-filtering:** run a tiny classifier on 3 sampled frames. If the scene isn't road (too dark, blurred, indoors), **discard without uploading**. Saves bandwidth and privacy exposure.
- **On-device redaction before upload** (v2, but state the intent now): blur faces and licence plates locally.
- **Delete on verified upload:** delete the local file only after HTTP 200 *and* server-side checksum match. Also delete after 24 h regardless, and on app uninstall.
- **Server-side retention limit:** keep raw video max **7 days**, then keep only the cropped defect bounding-box patch and the derived metadata. Publish this in the privacy policy.
- **Reward the rider.** Even ₹1–2 of data reimbursement per accepted clip changes the entire consent conversation and is a rounding error on our unit economics.

WRONG 4 — "Average the latitude and longitude"

The problem: A plain arithmetic mean is the wrong estimator here.

- It is **not robust**: one report with 80 m GPS error drags the centroid off the road entirely.
- It ignores the `accuracy` value each fix carries.
- If a single road segment has **two** potholes 30 m apart, the mean lands in the smooth gap between them — and we send a repair crew to good tarmac.

FIX:

1. Map-match all reports to the road centreline first.
2. Run **DBSCAN** (`eps ≈ 20 m`, `min_samples ≈ 5`) — density-based clustering (Ester et al.) naturally separates two nearby potholes and labels stragglers as noise instead of averaging them in.
3. Within each cluster, compute an **inverse-variance-weighted centroid**, weight `wi = 1/accuracyi²`.
4. Report the **median** alongside the weighted mean, and a **confidence ellipse**, not a bare point. The dashboard should show a small uncertainty circle — engineers trust a system that admits its own error bars far more than one that shows a false pinpoint.
5. At scale use **Uber's H3** hexagonal index for O(1) neighbour lookup before running DBSCAN inside candidate cells (the GEOSCAN approach). PostGIS handles our hackathon scale fine; H3 is the path to national scale.

WRONG 5 — "One counter, counter += 1"

The problem: A single integer throws away everything that matters: *who* reported, *how hard* the impact was, *how fast* they were going, *what vehicle*, and *how many other vehicles passed without firing*. A counter of 50 from one rider means nothing; 50 from 50 riders means everything.

> **FIX:** Store **events, not counters.** One immutable row per report with `device_id_hash, lat, lon, gps_accuracy, speed, heading, peak_z, jerk, rms, confidence, vehicle_class, timestamp`. Counters become `COUNT(DISTINCT ...)` at query time (or a materialised view). Additionally store a `pass_count` per segment — the denominator. **The ratio `fires / passes` is the real signal; the raw numerator is not.**

WRONG 6 — "Ignore all readings from a false-positive area for a week"

The problem: Over-broad suppression is dangerous. Real potholes cluster *exactly* where traffic slows — at signals, junctions, and bus stops — because braking and standing loads destroy pavement there. Blanket-blocking 50–100 m around a traffic light would blind us to some of the worst real defects in the city.

FIX:

- **Down-weight (×0.2), never hard-block.**
- **Never suppress a report that carries a human YES vote.**
- Suppression is scoped **per reason code**: a "traffic jam" suppression should only suppress *low-magnitude* events, because a jam produces gentle rocking, not a sharp shock. A high-jerk event inside a jam zone is still interesting.
- **Escalating TTL:** first FP → 3 days; repeat FP at the same spot → 7 days → 30 days.
- **Auto-review:** if suppressed reports keep arriving at high magnitude, re-open the candidate for a fresh clip. Fail-safe, not fail-silent.

WRONG 7 — "50K+ images will be enough / we'll train from scratch"

The problem: Two mistakes. (a) *Never train a vision model from scratch* for this — you will burn the whole hackathon and land below a fine-tuned baseline. Every strong result in Section 5.2 is a fine-tune. (b) 50K raw images is meaningless; what matters is **50K images with the right class balance from Indian roads.** RDD2022's entire 6-country corpus is 47,420 images with 55,000+ instances, and potholes (D40) are the *minority, smallest, hardest* class — which is exactly why YOLO-ROC needed a special design to gain +16.8% on D40 alone.

> **FIX:** Fine-tune YOLOv8n/YOLO11n from COCO weights on RDD2022 (India split first), then add Roboflow/Chitholian/MIIA/TD-RD/RAD, then add our own labelled frames from real rider clips. Track mAP on a **held-out India-only test set**. Target ~65–75% mAP50 for potholes and say so honestly.

WRONG 8 — "AI on the app will be trained from scratch"

The problem: For the *sensor* model, from-scratch is actually fine (it's small), but the framing invites the wrong approach — a big deep net. P4 shows a **Random Forest** hits 88.5% precision, and P7 shows an ANN on **four** features hits ~90%.

> **FIX:** Ship a small 1D-CNN (~50–400 KB) *or* a gradient-boosted tree converted to TFLite. Keep a rule-based threshold detector as the *first* gate before the model even runs — it eliminates 99% of windows for free and saves battery. **Do not put a transformer on a delivery rider's phone.**

WRONG 9 — Unstated assumption: the phone is mounted, screen-up, facing forward

Reality: Riders keep phones in pockets, in bags, in chest pouches, in bad mounts. Orientation is arbitrary and changes mid-ride.

FIX — mandatory, do this in week 1:

- Compute **orientation-independent features**: total acceleration magnitude $\sqrt{(x^2+y^2+z^2)}$ is rotation-invariant.
- Use the **gravity sensor** to derive a rotation matrix and project acceleration into the *vehicle* frame (vertical / longitudinal / lateral) regardless of how the phone sits. Khandakar et al. did this alignment manually; we must do it automatically.
- Detect and **reject "pocket mode"** for pothole detection (body movement dominates), or use a separate model for it.

WRONG 10 — No plan for "what happens after we tell the municipality"

The problem: A dashboard full of red dots that nobody acts on is a science project, not a solution. This is the failure mode of *most* civic-tech hackathon winners.

> **FIX:** Close the loop in the product itself — auto-generated work orders, a ward-engineer mobile view, an SLA clock per defect, MTTR dashboards, and the **repair-verification** feature from GAP 5. See Sections 9 and 22.

SECTION 9 OF 25

9 SIMPLER ALTERNATIVES WE CONSIDERED

Cheaper options

Judges love hearing that you evaluated cheaper options and can justify your choice. This also protects us if the AI underperforms — we have documented fallbacks.

#	Simpler approach	How it works	Cost	Verdict
S1	**Pure threshold rule, no ML**	Fire when `|z| > k·σ_baseline` and jerk > threshold	Nearly zero	**OK BUILD THIS FIRST.** It is our week-1 baseline and our permanent pre-filter. Honestly delivers maybe 70% of the value. If our ML model ever fails on demo day, this still works.
S2	**Crowd voting only, no sensors**	Riders tap a "bad road here" button	Zero	**NO** Depends on human effort; sparse, biased, gameable. But keep the button — it is free extra labels.
S3	**Dashcam / camera-only**	Continuous camera + detector	High (battery, data, opt-in)	**NO** As primary. **OK** As our Stage-4 verifier. This is exactly what NHAI is doing on highways — and it needs dedicated hardware, which is why it can't scale to city roads.
S4	**IRI roughness score instead of discrete potholes**	Output a comfort/roughness index per segment (RoadBounce, MIT Carbin)	Low	**OK ADD THIS AS A SECOND OUTPUT.** Easier than pothole detection, standards-compatible (r = 0.862 vs Roughometer), and it's what engineers already procure against. Cheap insurance if pothole precision disappoints.
S5	**Fixed IoT sensors on lampposts**	Vibration/vision sensors on infrastructure	Very high	**NO** Thousands of devices, SIMs, power, theft, maintenance. Dead end for a national system.
S6	**Satellite / drone imagery**	Aerial photogrammetry	High	**NO** Resolution insufficient for a 30 cm pothole under tree cover; no live refresh. Drones need permissions.
S7	**Municipal garbage trucks / buses as the fleet**	Put phones in vehicles the city already owns	Low	**OK EXCELLENT PILOT WEDGE.** ~50 city buses on fixed routes = high repeat coverage, one single owner to sign the contract, no consent complexity, no platform partner needed. **This should be our first paid pilot.** Buses re-drive identical routes daily, which is perfect for the negative-evidence and repair-verification features.
S8	**Just parse existing citizen complaints with an LLM**	NLP over Mhari Sadak / grievance portal text	Very low	**!** Useful *supplementary* input, and a nice extra data layer. Not a substitute — it inherits all the reporting bias.
S9	**Buy commercial data (RoadBounce / RoadMetrics)**	License existing surveys	Medium	**NO** For us. But it proves a market exists and prices it.
S10	**Audio-based detection (microphone)**	Tyre-impact sound classification	Very low	**?** Genuinely under-explored and very cheap. Bigger privacy problem than video though (captures speech). Interesting research side-note; mention as future work, don't build it.
S11	**OBD-II / vehicle CAN bus**	Read real suspension data	Medium	**NO** Two-wheelers in India don't expose this. Non-starter for our fleet.

Our chosen strategy is a deliberate layering of the cheapest options: S1 (threshold) → ML sensor model → S4 (roughness as a bonus output) → S2 (voting for free labels) → S3 (camera, gated) → and S7 (city buses) as the go-to-market wedge.

SECTION 10 OF 25

10 DATASETS — SENSOR + IMAGE

What data to use

10.1 Sensor datasets (accelerometer / gyroscope) — ranked by usefulness to us

Rank	Dataset	Size / content	Access	Why it matters
#1 1	**Khandakar et al. — "Harnessing Smartphone Sensors for Enhanced Road Safety"** (Scientific Data 12:418, 2025)	**10 sensor streams**: accelerometer (cal + uncal), gyroscope (cal + uncal), magnetometer (cal + uncal), gravity, orientation, GPS/Location, total acceleration. Labelled folders for **Bump**, **Pothole**, and driving behaviour (**Aggressive / Standard / Slow**). Route >30 km, Rajshahi, Bangladesh. Avg sampling **89.82 Hz** (60–99 Hz). Collected with Poco X2 + *Sensor Logger* app, dashboard-mounted, coordinate frames aligned.	**Open access on Figshare**, DOI `10.6084/m9.figshare.25460755`. Analysis code on GitHub (`naznine/Harnessing-Smartphone-Sensors...`)	**Start here on day 1.** South-Asian roads, close to Indian conditions. Has calibrated *and* uncalibrated data (so we can test our own calibration pipeline). Already statistically validated for bump-vs-pothole separability ($p = 0.0008$). Includes CC BY-NC-ND licence — fine for a hackathon/research use; check terms before any commercial use.
#2 2	**Our own collected data — "SETU-IND-1"**	Target: **≥ 8 hours / ≥ 150 km** across ≥ 3 phone models, ≥ 2 vehicle types (2-wheeler + car), ≥ 2 mount positions, day + night, with a second person ground-truth-labelling via a big on-screen button + a synchronised dashcam video	We create it	**Non-negotiable.** No public dataset contains Indian two-wheeler data. This is also our research contribution — publish it with a DOI (fixes GAP 7).
#3 3	**Pothole Lab** (Carlos et al.)	Open-access web platform that *synthesises* virtual roads with configurable anomaly types/counts	Free web platform	Generates unlimited labelled synthetic data for augmentation, and lets us unit-test the clustering logic without leaving the room.
4	**CarSim-generated data** (method from Fox et al., IEEE SECON 2015)	Vehicle-dynamics simulation	Commercial sim (carsim.com)	The documented workaround for scarce real data. Only if we get access.
5	**GitHub:** `aswathselvam/Potholes`	Android IMU pothole detection, **50 Hz / 20 ms refresh**, format `timestamp, aX, aY, aZ, gX, gY, gZ`; SVM in MATLAB → exported to C → Java NDK	Public repo + linked paper	A complete working reference implementation of *exactly* our Stage-1. Read their code before writing ours. Also confirms 50 Hz is workable.
6	**GitHub:** `AdityaPune/Pothole-Detection`	Takes **RMS of 10 readings** around the pothole instant for accel + gyro	Public repo + paper	Gives us a concrete, proven feature: windowed RMS. Cheap and effective.
7	**GitHub:** `VishalSingh25/Pothole-Project`	Classifies potholes, **unmarked speed breakers**, and more from phone IMU	Public repo	Directly supports our multi-class taxonomy — especially "unmarked speed breaker", a very Indian category.
8	**González et al. — Chihuahua, Mexico** (12 cars and trucks)	Multi-vehicle benchmark	**! No longer publicly available**	Listed so we don't waste hours hunting it. Cited as a field-wide problem by Khandakar et al.
9	**Zylius dataset** (3-axis accelerometer, normal vs aggressive driving)	Driving-style labels	Via papers	For the driver-behaviour side-product.
10	**Ferreira et al. (2017), PLoS One**	7 categories of aggressive driving events, multi-sensor	Open access	Ready-made multi-class driving behaviour labels for the insurance/fleet revenue stream.
11	**RoadSens-4M** (Nature Sci. Data, 2026)	Multimodal smartphone + camera dataset for roadway analysis	Open access	Newest available multimodal set — worth checking for a combined sensor+vision baseline.

Sensor data collection protocol (do exactly this — it determines whether our model works):

```
SETUP
  □ 3+ phone models (1 budget, 1 mid, 1 flagship) — heterogeneity is the point
  □ 2 vehicles minimum: scooter/bike + car (also try an auto-rickshaw)
  □ 3 mount conditions: rigid handlebar mount, chest pouch, trouser pocket
  □ Sampling: SENSOR_DELAY_GAME or 100 Hz explicit
  □ Log: accel(x,y,z), gyro(x,y,z), gravity(x,y,z), GPS(lat,lon,acc,speed,bearing), timestamp_ns

GROUND TRUTH (two independent channels — this is what most papers get wrong)
  □ Channel A: pillion rider taps a big on-screen button the instant the wheel hits
  □ Channel B: a second phone dashcam records the road, synced by an initial
               3-tap "clap" spike visible in both accel and video
  □ Post-process: align by the clap, then hand-label windows using the video.
    Button taps have ~300-500 ms human reaction lag — video is the true label.

CLASSES TO COLLECT (aim for balance; class 5 is the one everyone forgets)
  □ 0 smooth road        (easy, lots of it — will dominate, so downsample later)
  □ 1 rough road         (broken but continuous surface)
  □ 2 POTHOLE            (the target; collect 200+ instances)
  □ 3 speed bump         (marked + unmarked — collect 100+)
  □ 4 rumble strip / expansion joint / manhole cover / railway crossing
  □ 5 NON-ROAD EVENT     (phone drop, hard brake, hard turn, pothole-free
                          pocket movement, door slam, walking with phone)

SPLIT RULE — CRITICAL
  □ Split by ROUTE and by DEVICE, never randomly by window.
    Random splits leak neighbouring windows across train/test and produce fake
    99-100% accuracy — exactly the red flag we noted in MDPI App.Sci. 14(21):10027.
  □ Hold out one entire phone model and one entire route as the final test set.
```

10.2 Image / video datasets (for the backend vision model)

Rank	Dataset	Size	Classes	Access	Why
#1 1	**RDD2022** (Arya, Maeda, Ghosh, Toshniwal, Sekimoto — IIT Roorkee + Univ. of Tokyo)	**47,420 images, 55,000+ damage instances, 6 countries: Japan, India, Czech Republic, Norway, USA, China**	D00 longitudinal crack, D10 transverse crack, D20 alligator crack, **D40 pothole**	Open, released with CRDDC2022 (IEEE BigData Cup); also mirrored via the Wiley *Geoscience Data Journal* paper	**Primary training set.** Has an India split. Benchmarked, so our numbers are directly comparable to published work (65.7% mAP, F1 74–82%). Explicitly intended for municipalities and road agencies.
#2 2	**RAD — Road Anomaly Detection (Bengaluru)**, Springer 2024	Indian road damage dataset with **pothole depth estimation** via smartphone camera	Indian damage types	Via paper (Springer LNCS)	**Indian + depth.** Depth is what converts "a pothole" into "a P1 severity pothole". Powers our severity score.
#3 3	**TD-RD (Top-Down Road Damage)**, arXiv 2501.14302	**7,088 high-resolution images, 12,882 annotated instances**	cracks, potholes, **patches**	Open	Different viewpoint = better generalisation. The **"patches" class lets us detect repairs**, which powers our audit-trail/MTTR feature.
4	**Roboflow Universe pothole dataset** (`public.roboflow.com/object-detection/pothole`)	~1,200–4,000 images by version	pothole	Free, one-line download, YOLO-ready	**Fastest day-1 baseline.** Get a model training within an hour.
5	**Chitholian pothole dataset**	**665 annotated potholes**, re-split 70/20/10, many formats	pothole	Free (Roboflow / GitHub)	Small, clean, well-documented.
6	**MIIA Pothole Image Dataset** (Machine Intelligence Institute of Africa, 2019)	Classification challenge set	pothole / no pothole	Public	Developing-country road context similar to India; good extra negatives.
7	`michelpf/dataset-pothole`	3,125 train / 843 test, all YOLO-annotated	pothole	GitHub	Ready to concatenate.
8	`jaygala24/pothole-detection`	**1,243 images**, YOLO format, from a pothole *dimension estimation* paper	pothole	GitHub	Paired with dimension-estimation code → severity.
Rank	**Dataset**	**Size**	**Classes**	**Access**	**Why**
9	**SoV Pothole Detection Dataset** (ZED 2 stereo camera)	447 images with **left frame, right frame AND depth map**, potholes + drains	pothole, drain	GitHub (`achireistefan/Pothole-Detection`)	**Stereo depth maps** — the cleanest way to learn depth estimation. Also includes

Rank	Dataset	Size	Classes	Access	Why
					drains, a real Indian FP source.
10	**Pre-trained:** `cvtechniques/road-damage-detection-yolov11` (HuggingFace)	Fine-tuned YOLOv11s, 4 road-damage classes from street-level imagery	4 damage types	HuggingFace	**A ready-made checkpoint.** Download, evaluate, and we have a working vision stage on day 1 as a safety net.
11	**Google Street View** (method from the YOLOv7 CRDDC2022 paper)	Unlimited road imagery	self-label	API (billed)	The paper found Street View collection "efficient" and reached F1 81.7% with it. Our route to unlimited India-specific images. Mind the API cost and Terms of Service.
12	**Our own frames from real rider clips**	Target: 2,000+ labelled frames	our 6-class scheme	We create it	Closes the domain gap: our production camera angle, resolution, motion blur and lighting are unique to us. **This is what makes the deployed model actually work.**

10.3 Negative / confuser images we MUST collect (nobody does this, and it's why systems fail)

The pothole class is easy. The **confusers** are what break a deployed model. Deliberately collect and label:

```
CONFUSERS THAT LOOK LIKE POTHOLES
  □ Dark tar patches / bitumen repair blobs  (looks like a hole, is flat)
  □ Shadows — of trees, poles, buildings, vehicles, the rider's own bike
  □ Wet patches and puddles (a water-filled pothole hides its own depth)
  □ Manhole and utility covers (round, dark, edged)
  □ Storm drains and gratings
  □ Oil stains, paan stains, burn marks
  □ Loose gravel patches, sand piles
  □ Cattle dung (genuinely a problem on Indian roads — dark, round)
  □ Speed breakers (raised, but visually a dark band across the road)
  □ Faded/worn lane markings and zebra crossings

SCENES THAT MUST RETURN "NOT A ROAD DEFECT"
  □ Traffic jam — bumper-to-bumper, road not even visible
  □ Traffic signal / stop line / junction
  □ Toll plaza and speed-breaker-heavy approach
  □ Construction zone with barricades, cones, dug trench
  □ Railway level crossing
  □ Unpaved / mud / gravel road (the whole surface is "damaged" — needs its own class)
  □ Night, headlight glare, rain on lens, dirty lens
  □ Camera obstructed: bag strap, finger, jacket, inside a pocket (pure black)

CONDITION VARIATIONS (augmentation targets)
  □ Bright noon, golden hour, dusk, night with streetlights, night without
  □ Rain, fog, dust
  □ Motion blur at 20 / 40 / 60 km/h
  □ Wet vs dry surface
```

Rough target mix for the vision training set: `~40% real potholes · ~25% confusers · ~20% clean road (true negatives) · ~15% other damage types (cracks, patches, ruts)`

10.4 Data augmentation plan

- Standard: mosaic, random scale (0.5–1.5×), HSV jitter, horizontal flip.
- **Motion blur** (essential — our frames come from a moving two-wheeler).
- **Synthetic rain / low-light / glare** overlays.
- **Copy-paste augmentation**: paste real pothole crops onto clean road images. Cheapest way to fix D40 class imbalance, which is the documented hard class.
- **! Do NOT vertically flip.** Road scenes have a fixed gravity direction; vertical flips teach the model nonsense.

SECTION 11 OF 25

11 SYSTEM ARCHITECTURE

Full diagrams

11.1 The big picture

```
  GPU worker (or CPU for demo):
    1. ffmpeg → sample ~6-10 fps (not all 180 frames; 30 fps is wasteful)
    2. quality gate: too dark / too blurred / not-a-road → REJECT clip
    3. YOLOv8/YOLO11 detector → boxes + classes per frame
    4. ByteTrack → persistent IDs → UNIQUE defect count (not 40 duplicates)
    5. scene classifier → jam / signal / construction / crossing?
           └→ if yes: create SUPPRESSION ZONE (reason, TTL, ×0.2 weight)
    6. size & depth estimate (bbox geometry + RAD-style depth method)
    7. counter2 += 1 for the cluster; store best bbox crop
    8. DELETE raw video after 7 days; keep only crop + metadata
        ▼
  Update defect_clusters: posterior, severity, state → CONFIRMED
                                    │
                                    │  NOTIFY (PostgreSQL LISTEN/NOTIFY)
                                    ▼
                        LAYER 4 — MUNICIPAL WEB PORTAL

  FastAPI WebSocket  ──►  React + deck.gl + Google Maps
  • live red dots (ScatterplotLayer / IconLayer), no page refresh
  • HeatmapLayer for severity density, H3HexagonLayer for ward rollups
  • segment health colouring (green→amber→red) via PathLayer
  • defect drawer: photo crop, severity, first-seen, votes, MTTR clock
  • admin login (municipality), RBAC, work-order export, PDF/CSV reports
```

11.2 Request/response contracts (write these first — they unblock parallel work)

```
// POST /v1/events   (app → server, batched)
{
  "sdk_version": "0.4.1",
  "device": {
    "id_hash": "sha256(install_id + salt)",   // never a real device ID
    "model_class": "android_midrange",         // not exact model → less fingerprinting
    "vehicle_class": "two_wheeler",
    "calibration": { "noise_floor_z": 0.42, "samples": 60000 }
  },
  "events": [
    {
      "ts": 1786512345678,          // epoch ms, device clock
      "lat": 26.84671, "lon": 80.94623,
      "gps_accuracy_m": 6.4,
      "speed_kmph": 34.2, "heading_deg": 271.5,
      "label": "pothole",           // model's 6-class output
      "confidence": 0.87,
      "peak_z": 18.4,               // m/s^2, vehicle-frame vertical
      "jerk_max": 220.5,            // m/s^3  <-- discriminates speed of impact
      "rms_window": 4.9,
      "duration_ms": 180
    }
  ],
  "passes": [ { "segment_id": "osm:way/1234", "count": 12 } ]  // the DENOMINATOR
}

// 200 OK  (server → app; commands ride back on the ACK — no extra polling)
{
  "accepted": 1,
  "commands": [
    {
      "type": "CAPTURE_VIDEO",
      "cmd_id": "c_9f2a",
      "target": { "lat": 26.84702, "lon": 80.94588 },
      "trigger_radius_m": 60,          // server computes from expected speed
      "clip_seconds": 6, "fps": 30, "resolution": "1280x720",
      "expires_at": 1786598745000,
      "upload_url": "https://s3.../presigned?...",
      "max_attempts": 1
    },
    {
      "type": "ASK_USER",
      "cmd_id": "c_1b7d",
      "target": { "lat": 26.84702, "lon": 80.94588 },
      "prompt_when": "stopped",        // NEVER while moving
```

```
      "question_key": "pothole_confirm_v1",
      "landmark_hint": "near Hazratganj crossing"
    }
  ]
}

// POST /v1/votes
{ "cmd_id": "c_1b7d", "answer": "yes", "ts": 1786512999000, "device_id_hash": "..." }
```

Design note worth defending: commands ride back on the **HTTP response to the event upload**, so in the common case we need **no separate polling and no persistent socket** on the phone. FCM push is only used for time-sensitive commands. This is a real battery/data saving and it is the kind of detail that shows engineering maturity.

SECTION 12 OF 25

12 AI MODEL 1 — ON-DEVICE SENSOR MODEL

Edge AI design

12.1 Signal processing chain

```
raw accel (x,y,z) @100 Hz + gyro (x,y,z) + gravity (gx,gy,gz)
        |
        ├─[1] TIME SYNC & RESAMPLE
        |     Android sensor events are NOT evenly spaced. Resample to a fixed
        |     100 Hz grid by linear interpolation. Skipping this quietly corrupts
        |     every frequency-domain feature you compute afterwards.
        |
        ├─[2] ORIENTATION CORRECTION  (fixes WRONG-9)
        |     Build rotation matrix R from the gravity vector (defines "down")
        |     + heading. Project accel into vehicle frame:
        |         a_vertical, a_longitudinal, a_lateral
        |     Also always keep the rotation-invariant magnitude:
        |         a_mag = sqrt(x^2 + y^2 + z^2)
        |
        ├─[3] GRAVITY REMOVAL
        |     a_dynamic = a_total − g       (as in Khandakar et al., eq. 6-8)
        |
        ├─[4] FILTERING
        |     Band-pass 0.5–30 Hz. Below 0.5 Hz = vehicle body roll and drift;
        |     above 30 Hz = engine and tyre hum. Pothole shock energy lives
        |     roughly in the 3–20 Hz band.
        |
        ├─[5] PER-DEVICE NORMALISATION  (fixes GAP 2)
        |     z_norm = (a_vertical − mu_device) / sigma_device
        |     mu/sigma learned during a 10-minute rolling calibration on the
        |     smoothest 20% of recent driving.
        |
        ├─[6] WINDOWING
        |     2.0 s windows, 50% overlap → 200 samples per window per axis
        |
        ├─[7] CHEAP THRESHOLD GATE  ◄── the battery saver
        |     if max|z_norm| < 3.0 AND rms < r0 : DROP, do not run the model
        |     Kills ~99% of windows. Model runs only a few times per minute.
        |
        └─[8] MODEL → 6-class softmax + confidence
```

12.2 Features (use these if you go the tree/SVM route; the CNN learns them itself)

Domain	Features
Time	mean, std, variance, RMS, min, max, **peak-to-peak range**, skewness, kurtosis, zero-crossing rate, signal magnitude area, **max jerk (dz/dt)**, energy, entropy, autocorrelation peak
Frequency	FFT peak magnitude, dominant frequency, spectral centroid, spectral spread, spectral entropy, band energy ratios (0.5–3, 3–10, 10–20, 20–30 Hz)
Wavelet	Daubechies db4 detail-coefficient energy at levels 1–4 (per RoADS / Seraj et al.)
Cross-axis	correlation(vertical, lateral) — **a pothole hit by one wheel produces roll; a speed bump hit by both wheels does not.** This is the single best bump-vs-pothole feature.
Gyro	roll-rate peak, pitch-rate peak, ratio roll:pitch
Context	speed (a 20 cm hole at 20 km/h ≠ at 60 km/h), vehicle class, phone-mount confidence

Why `max jerk` deserves special attention: peak acceleration is *counter-intuitive* across speeds — at high speed the suspension absorbs the impact over a shorter time, which can give a *lower* peak but a much higher rate of change. Jerk (the derivative) is the more speed-stable discriminator. Do not rely on peak magnitude alone.

Why `speed` must be an input, not ignored: the same defect produces wildly different signatures at different speeds. Feed speed in, or normalise features by speed. Most papers skip this and then complain about variance.

12.3 Model choice — build in this order

Stage	Model	Size	Expected	When
Baseline 0	Pure threshold rule (S1)	0 KB	~60–70% precision	**Day 2.** Always keep it as the gate.
Baseline 1	Random Forest / XGBoost on engineered features	~200 KB	~88% precision, ~75% recall (matches MDPI 20(19):5564)	**Day 4.** This is our safe fallback and it is genuinely competitive.
Production	**1D-CNN** over the raw normalised window (2×3 channels)	~300–500 KB → **~120 KB after INT8 quantisation**	90–95% target	**Week 2.** Learns its own features; no hand-crafted FFT needed at inference.
Optional	CNN + small GRU head (sequence context)	~800 KB	+1–3%	Only if time allows.
NO Avoid	Transformer / large LSTM	MBs	—	Too heavy for a background service on a budget phone.

Proposed 1D-CNN (small, fast, quantisable):

```
Input: (200 timesteps, 6 channels)  [a_vert, a_long, a_lat, gyro_x, gyro_y, gyro_z]
  Conv1D(32, k=7, stride=2) → BatchNorm → ReLU
  Conv1D(64, k=5)           → BatchNorm → ReLU → MaxPool(2)
  Conv1D(64, k=3)           → BatchNorm → ReLU
  GlobalAveragePooling1D
  Concat( pooled_features , [speed_norm, vehicle_class_onehot] )   ← context injection
  Dense(32) → Dropout(0.3) → Dense(6, softmax)
```

Train in PyTorch or Keras → export to **LiteRT (formerly TensorFlow Lite)** with INT8 post-training quantisation. LiteRT is the successor to TFLite and is the current Google on-device runtime; it reports ~1.4× faster GPU performance than TFLite and supports NPU acceleration, though for a model this small CPU (XNNPACK) is more than enough.

12.4 Handling class imbalance and evaluation

- Smooth road will be **>95%** of all windows. Downsample it, or use focal loss / class weights.
- **Report per-class precision and recall, plus a confusion matrix.** A single "accuracy" number is meaningless at 95% class imbalance — a model that always says "smooth road" would score 95%.
- **Tune the operating point for PRECISION on-device, RECALL in aggregate.** Reasoning: a false positive costs us server noise and possibly a wasted video request; a false negative costs almost nothing because 50 more riders will pass the same hole within the hour. **So set the on-device threshold high (τ ≈ 0.8) and let the crowd recover the recall.** This is a direct architectural consequence of the crowd-redundancy thesis, and it is a strong point to make to judges.
- Evaluate on the **held-out device and held-out route**, and report both separately. Expect a drop — the DTW paper saw 98% → 91% across Indian cities. If our numbers *don't* drop across devices, we have a leakage bug.

12.5 Battery and data budget (a partner's first question)

Item	Budget	How we hit it
Battery	**< 2%/hour additional**	Threshold gate means the model runs ~5×/min, not 50×/sec. No camera. No screen. Reuse the host app's existing GPS fix — never request our own. Batch network I/O to wake the radio once a minute, not once an event.
Mobile data	**< 1 MB/day** in passive mode	Batched, gzipped JSON. ~200 bytes/event. Even 500 events/day ≈ 100 KB.
Video data (opt-in only)	~6–12 MB per clip, **≤ 2 clips/day**, WiFi-preferred	Hard cap in the SDK; user-visible counter; optional reimbursement.
CPU	< 3% average	INT8 model, 120 KB, single-threaded XNNPACK.
Storage	< 5 MB	Ring buffer in memory; only queued events persisted (Room/SQLite).

SECTION 13 OF 25

13 AI MODEL 2 — BACKEND VISION MODEL

Video AI design

13.1 Pipeline

```
Uploaded clip (5-8 s, 30 fps, 720p, ~180-240 frames, ~6-12 MB)
        |
        ├─[1] INTEGRITY: checksum matches? duration sane? codec readable?
        |       └─ fail → reject, tell app to keep local file for one retry
        |
        ├─[2] FRAME SAMPLING: ffmpeg, extract at 6-10 fps  (~40-60 frames)
        |       Why not all 180? Consecutive frames are near-duplicates.
        |       30 fps buys you nothing and costs 5x the GPU time.
        |       (We still RECORD at 30 fps — it gives sharper individual frames
        |        and more chances that at least a few are unblurred.)
        |
        ├─[3] QUALITY GATE (cheap, runs before the detector)
        |       • mean luminance too low/high → reject (pocket, night, glare)
        |       • Laplacian variance low → reject (motion blur / dirty lens)
        |       • "is this a road?" tiny classifier → reject if not
        |       Rejected clips never reach the detector and never get stored.
        |
        ├─[4] DETECTOR: YOLOv8n/s or YOLO11 fine-tuned on RDD2022(India)+ours
        |       classes: pothole, longitudinal_crack, transverse_crack,
        |                alligator_crack, patch, rut, manhole, speed_bump
        |       output: boxes + class + per-frame confidence
        |
        ├─[5] TRACKER: ByteTrack  ◄── fixes the duplicate-counting bug
        |       Assigns a persistent ID to each physical defect across frames.
        |       A defect seen in 30 frames = ONE defect, not 30.
        |       Also gives us temporal consistency as a confidence booster:
        |       a box that persists across 20 frames is real; one that flickers
        |       in 2 frames is a shadow.
        |
        ├─[6] SCENE CLASSIFIER (parallel branch)
        |       classes: normal_road | traffic_jam | signal_junction |
        |                construction | railway_crossing | toll_plaza |
        |                unpaved_road | obstructed_view
        |       └─► if not normal_road → CREATE SUPPRESSION ZONE
        |           {reason, centre, radius 50-100 m, weight 0.2,
        |            ttl 3d→7d→30d escalating}
        |
        ├─[7] GEOMETRY / SEVERITY
        |       • bbox width as fraction of lane width → physical size estimate
        |       • bbox vertical position → rough distance (perspective)
        |       • depth: RAD-style monocular depth estimation, or stereo-trained
        |         proxy from the SoV depth-map dataset
        |       • combine with IMU peak_z & jerk from the triggering reports
        |
        ├─[8] AGGREGATE VERDICT for this clip
        |       confirmed = (unique pothole tracks ≥ 1)
        |                   AND (best track persisted ≥ 8 sampled frames)
        |                   AND (max confidence ≥ 0.6)
        |                   AND (scene == normal_road)
        |
        ├─[9] UPDATE CLUSTER: counter2 += 1, posterior update,
        |       store best bbox CROP (not the video) + thumbnail
        |
        └─[10] RETENTION: raw video deleted after 7 days (or immediately if
              the clip was rejected at step 3). Crop + metadata retained.
```

13.2 Model choice and honest expectations

Model	Params	Where it runs	Expected mAP50 (pothole)
YOLOv8n (start here)	3.2 M	CPU ok, GPU better	~60–68%
YOLOv8s / YOLO11s (production)	11 M	GPU (T4 free tier / Colab)	~65–75%

Model	Params	Where it runs	Expected mAP50 (pothole)
YOLOv8-PD style (pruned)	**2.3 M, 6.1 GFLOPs**	edge-capable	reported +1.4 pp over baseline while 74% the size
YOLO-ROC style (ultra-light)	**2.0 MB model**	on-device v2	mAP50 **67.6%** on RDD2022_China_Drone; **+16.8% on D40 potholes**
RF-DETR + ByteTrack	larger	GPU	strong, and the Roboflow pipeline gives tracking + severity out of the box
Pre-trained `road-damage-detection-yolov11`	—	GPU	Download as an immediate baseline / safety net

Set expectations honestly with judges. Published state of the art on RDD2022 is roughly **65.7% mAP** and **F1 74–82%**; one recent paper reports just **25.75%** on the hard Japan split. Potholes (D40) are the *smallest and hardest* class — which is precisely why YOLO-ROC needed a bespoke design to gain 16.8% on that one class.

> ***Say this:*** *"Our vision model targets ~70% mAP on potholes, in line with published state of the art. We do not need 99%, because vision is our third layer — it only has to break ties on candidates that sensors and humans have already agreed on. Two independent clips agreeing at 70% each gives us a combined confidence well above 90%."*

That reasoning — that **stage-wise composition beats single-model accuracy** — is the most sophisticated thing we can say in the room.

13.3 Small-object tricks for potholes specifically

- Train at **higher input resolution** (960 or 1280, not 640). Potholes are small in frame.
- **Tiled inference**: split each frame into overlapping tiles, detect, then merge with NMS. Costly but effective.
- Weight the pothole class higher in the loss.
- **Copy-paste augmentation** of pothole crops onto clean road images to fix imbalance.
- Crop to the **lower 60% of the frame** — the road is never in the sky, and this both speeds inference and removes distracting background (shops, faces, signage) which is also a privacy win.

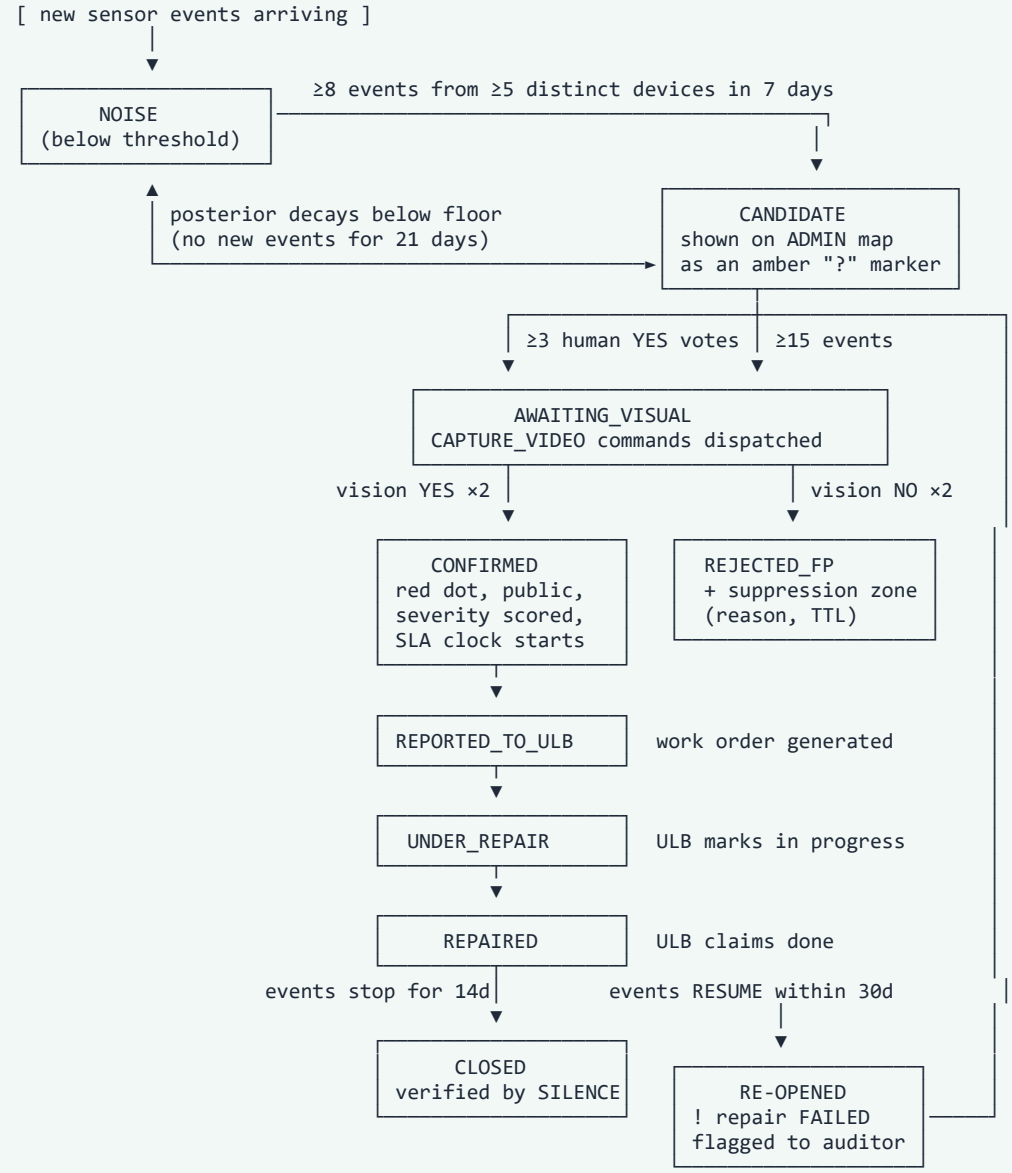
SECTION 14 OF 25

14 CLUSTERING, CONFIRMATION & TRUST LOGIC

The real brain

This section is the actual product. The models are commodity; **this logic is the moat.**

14.1 Defect lifecycle state machine



`CLOSED` **is verified by silence, not by a claim.** That is the whole audit-trail idea in one line, and it is only possible because we track the *denominator* (passes) as well as the numerator (fires). If 3,000 vehicles cross the repaired spot in 14 days and nothing fires, the repair is real. If reports resume, it isn't. **No competitor we found does this.**

14.2 The clustering job (pseudocode)

```
# Runs every 5 minutes via Celery Beat
def cluster_job():
    for cell in active_h3_cells(resolution=10):          # ~65 m edge hexagons
        events = fetch_events(cell, window_days=7)
        if len(events) < MIN_EVENTS: continue
```

```
        events = map_match_to_osm(events)                # kill perpendicular GPS error
        events = drop_bad(events, max_gps_acc=50, min_speed_kmph=5)
        events = apply_device_trust_weights(events)
        events = apply_suppression_weights(events)       # x0.2, never below x0.2

        # DBSCAN in metres, using a projected CRS (not raw degrees!)
        labels = DBSCAN(eps=20, min_samples=5, metric='euclidean') \
                    .fit_predict(project_to_metres(events))

        for cid in set(labels) - {-1}:                   # -1 == noise, discard
            grp = events[labels == cid]

            distinct_devices = grp.device_hash.nunique()
            capped = cap_per_device(grp, max_per_device=3)
            if distinct_devices < 5: continue

            # inverse-variance weighted centroid
            w   = 1.0 / (capped.gps_accuracy_m ** 2)
            lat = (capped.lat * w).sum() / w.sum()
            lon = (capped.lon * w).sum() / w.sum()
            ellipse = confidence_ellipse(capped, conf=0.95)

            passes    = pass_count(cell, window_days=7)
            fire_rate = len(capped) / max(passes, 1)     # THE real signal

            posterior = bayes_update(
                prior       = 0.02,
                sensor_evidence = capped,
                human_votes = fetch_votes(lat, lon, radius=40),
                vision      = fetch_vision_verdicts(lat, lon, radius=40),
            )
            severity = severity_score(capped, vision_meta, fire_rate, two_wheeler_share(capped))
            upsert_cluster(lat, lon, ellipse, posterior, severity,
                           distinct_devices, fire_rate)
            advance_state_machine(cluster_id)
```

Two implementation traps:

1. **Never run DBSCAN on raw latitude/longitude with `eps` in degrees.** 1 degree of longitude is ~111 km at the equator and shrinks with latitude. Project to a metric CRS (UTM zone, or use `haversine` metric with `eps` in radians). Getting this wrong silently produces garbage clusters — and it's a classic bug.
2. **Partition the events table by day** and index `(h3_cell, ts)` + a GiST index on geometry. Without this, the 5-minute job will start timing out once you have a few million rows.

14.3 Bayesian confidence fusion

```
log_odds = log(prior/(1-prior))                    # prior ~0.02 (2% of segments defective)
         + Σ_i  trust_i · llr_sensor(confidence_i, peak_z_i, jerk_i)
         + Σ_j  reputation_j · llr_vote(answer_j)   # yes:+1.2  no:−1.5  notsure:0
         + Σ_k  llr_vision(verdict_k, n_frames_k)   # strong term: ±2.5
         − suppression_penalty(zone_weight)

posterior = sigmoid(log_odds)
```

Posterior	State shown	Action
< 0.30	not shown	discard / decay
0.30–0.60	**CANDIDATE** (amber ?)	admin-only; may request video
0.60–0.85	**LIKELY** (orange)	admin-only; request video
> 0.85	**CONFIRMED** (red)	public map, work order, SLA clock

Every posterior comes with a **"why" breakdown** in the UI: *"47 sensor reports from 31 vehicles · 6 rider confirmations · verified in 2 video clips."* Explainability is a procurement requirement.

14.4 Device trust / reputation (anti-gaming)

```
trust(device) starts at 0.5, range [0.1, 1.0]

+0.02  each report that ends up inside a CONFIRMED cluster
−0.05  each report inside a REJECTED_FP cluster
−0.30  physics violation: reports while GPS speed < 5 km/h
−0.30  impossible trajectory (teleport / >150 km/h in city)
−0.50  event rate absurdly high (>200/hour = loose mount or spoofing)
×0.5   if the device is the ONLY reporter for a cluster (no corroboration)

Additional checks:
  □ Is the accelerometer signature consistent with vehicle motion at all?
  □ Is a mock-location provider enabled? (Android exposes this) → drop
  □ Does the device attest via Play Integrity API? (v2)
```

14.5 Severity score (concrete formula)

```
S =  0.30 × norm(peak_z_p95)          # how hard vehicles are hit (IMU, 95th pct)
   + 0.20 × norm(estimated_area_m2)   # how big it is (vision)
   + 0.15 × norm(estimated_depth_cm)  # how deep (vision/RAD depth)
   + 0.15 × norm(passes_per_day)      # how many people are exposed
   + 0.10 × two_wheeler_share         # how VULNERABLE those people are  ← our idea
   + 0.10 × norm(days_unrepaired)     # how long it's been ignored

Bands:   S ≥ 0.75 → P1 CRITICAL  (repair within 48 h)
         0.50-0.75 → P2 HIGH     (repair within 7 days)
         0.25-0.50 → P3 MEDIUM   (next maintenance cycle)
         < 0.25    → P4 MONITOR

Cross-reference to IRC:82-2015 — which triggers maintenance on distressed-area
percentage (potholes >0.5% of surface area) — by aggregating S over each road
segment to produce a segment-level condition rating engineers already recognise.
```

Also emit a **segment-level roughness/IRI-proxy** per 50 m (validated against Roughometer at r = 0.862 in the published literature), so the output plugs into existing pavement-management procurement language.

SECTION 15 OF 25

15 BACKEND, DATABASE & PIPELINES

Server design

15.1 Database schema (PostgreSQL 16 + PostGIS 3)

```
-- ═══════════════ IDENTITY & ACCESS ═══════════════
CREATE TABLE organisations (            -- a municipality / ULB / platform partner
  id            BIGSERIAL PRIMARY KEY,
  name          TEXT NOT NULL,
  org_type      TEXT NOT NULL CHECK (org_type IN ('municipality','platform','admin')),
  boundary      GEOMETRY(MultiPolygon, 4326),   -- their jurisdiction; RLS uses this
  created_at    TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE users (                    -- portal logins (municipal staff)
  id            BIGSERIAL PRIMARY KEY,
  org_id        BIGINT REFERENCES organisations(id),
  email         CITEXT UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,          -- argon2id
  role          TEXT NOT NULL CHECK (role IN ('super_admin','city_admin','ward_engineer','viewer','auditor')),
  ward_ids      BIGINT[],               -- scope for ward_engineer
  totp_secret   TEXT,                   -- 2FA; mandatory for city_admin+
  last_login_at TIMESTAMPTZ,
  is_active     BOOLEAN DEFAULT true
);

CREATE TABLE devices (                  -- contributing phones (pseudonymous)
  id                BIGSERIAL PRIMARY KEY,
  device_hash       TEXT UNIQUE NOT NULL,        -- sha256(install_id + server_salt)
  model_class       TEXT,                        -- 'android_budget' | 'android_mid' | ...
  vehicle_class     TEXT,                        -- 'two_wheeler' | 'car' | 'auto' | 'bus'
  platform_org_id   BIGINT REFERENCES organisations(id),   -- which SDK host
  trust_score       REAL DEFAULT 0.5 CHECK (trust_score BETWEEN 0 AND 1),
  noise_floor_z     REAL,                        -- per-device calibration
  calibration_n     INTEGER DEFAULT 0,
  video_opt_in      BOOLEAN DEFAULT false,       -- explicit, revocable consent
  consent_version   TEXT,                        -- DPDP audit trail
  consent_at        TIMESTAMPTZ,
  first_seen_at     TIMESTAMPTZ DEFAULT now(),
  last_seen_at      TIMESTAMPTZ
);

-- ═══════════════ RAW EVIDENCE (immutable, high volume) ═══════════════
CREATE TABLE sensor_events (
  id              BIGSERIAL,
  device_id       BIGINT NOT NULL REFERENCES devices(id),
  ts              TIMESTAMPTZ NOT NULL,
  geom            GEOMETRY(Point, 4326) NOT NULL,
  gps_accuracy_m  REAL NOT NULL,
  speed_kmph      REAL,
  heading_deg     REAL,
  label           TEXT NOT NULL,     -- pothole|rough_road|speed_bump|joint|non_road
  confidence      REAL NOT NULL,
  peak_z          REAL, jerk_max REAL, rms_window REAL, duration_ms INTEGER,
  segment_id      TEXT,              -- OSM way id after map-matching
  h3_r10          TEXT,              -- H3 index, resolution 10 (~65 m edge)
  cluster_id      BIGINT,            -- set by the clustering job
  weight          REAL DEFAULT 1.0,  -- trust x suppression
  sdk_version     TEXT,
  PRIMARY KEY (id, ts)
) PARTITION BY RANGE (ts);            -- one partition per day; drop old ones cheaply

CREATE INDEX ON sensor_events USING GIST (geom);
CREATE INDEX ON sensor_events (h3_r10, ts DESC);
CREATE INDEX ON sensor_events (segment_id, ts DESC);
CREATE INDEX ON sensor_events (cluster_id) WHERE cluster_id IS NOT NULL;

-- THE DENOMINATOR — without this, fire counts are meaningless
CREATE TABLE segment_passes (
  segment_id   TEXT NOT NULL,
```

```
  day          DATE NOT NULL,
  pass_count   INTEGER DEFAULT 0,
  device_count INTEGER DEFAULT 0,
  two_wheeler_passes INTEGER DEFAULT 0,
  PRIMARY KEY (segment_id, day)
);

-- ═══════════ DERIVED TRUTH ═══════════
CREATE TABLE defect_clusters (
  id               BIGSERIAL PRIMARY KEY,
  geom             GEOMETRY(Point, 4326) NOT NULL,   -- inv-variance weighted centroid
  uncertainty_m    REAL,                             -- radius of 95% conf ellipse
  segment_id       TEXT,
  ward_id          BIGINT,
  org_id           BIGINT REFERENCES organisations(id),
  state            TEXT NOT NULL DEFAULT 'candidate'
                   CHECK (state IN ('candidate','likely','awaiting_visual','confirmed',
                                    'rejected_fp','reported_to_ulb','under_repair',
                                    'repaired','reopened','closed')),
  defect_type      TEXT,             -- pothole|crack|rut|patch|rough_segment
  posterior        REAL NOT NULL DEFAULT 0.0,
  severity_score   REAL,
  priority         TEXT,             -- P1|P2|P3|P4
  est_area_m2      REAL, est_depth_cm REAL,
  counter_sensor   INTEGER DEFAULT 0,   -- total sensor reports
  distinct_devices INTEGER DEFAULT 0,   -- the number that actually matters
  counter_votes_yes INTEGER DEFAULT 0,
  counter_votes_no INTEGER DEFAULT 0,
  counter2_vision  INTEGER DEFAULT 0,   -- vision confirmations
  fire_rate        REAL,                -- fires / passes
  two_wheeler_share REAL,
  first_seen_at    TIMESTAMPTZ,
  confirmed_at     TIMESTAMPTZ,
  reported_at      TIMESTAMPTZ,
  repaired_at      TIMESTAMPTZ,
  closed_at        TIMESTAMPTZ,
  sla_due_at       TIMESTAMPTZ,
  best_crop_url    TEXT,
  updated_at       TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX ON defect_clusters USING GIST (geom);
CREATE INDEX ON defect_clusters (state, priority, severity_score DESC);

CREATE TABLE cluster_state_log (      -- full audit trail; never update, only insert
  id BIGSERIAL PRIMARY KEY,
  cluster_id BIGINT REFERENCES defect_clusters(id),
  from_state TEXT, to_state TEXT,
  reason     TEXT,
  actor      TEXT,                    -- 'system' | user email
  at         TIMESTAMPTZ DEFAULT now()
);

-- ═══════════ COMMANDS, VOTES, VIDEO ═══════════
CREATE TABLE commands (
  id           BIGSERIAL PRIMARY KEY,
  cmd_uid      TEXT UNIQUE NOT NULL,
  cluster_id   BIGINT REFERENCES defect_clusters(id),
  device_id    BIGINT REFERENCES devices(id),
  cmd_type     TEXT CHECK (cmd_type IN ('ASK_USER','CAPTURE_VIDEO')),
  payload      JSONB NOT NULL,
  status       TEXT DEFAULT 'pending'
               CHECK (status IN ('pending','delivered','executed','expired','failed','skipped')),
  issued_at    TIMESTAMPTZ DEFAULT now(),
  expires_at   TIMESTAMPTZ,
  completed_at TIMESTAMPTZ
);

CREATE TABLE user_votes (
  id BIGSERIAL PRIMARY KEY,
  cluster_id BIGINT REFERENCES defect_clusters(id),
  device_id  BIGINT REFERENCES devices(id),
  answer     TEXT CHECK (answer IN ('yes','no','not_sure')),
  ts         TIMESTAMPTZ DEFAULT now(),
  UNIQUE (cluster_id, device_id)        -- one vote per device per defect
);
```

```
CREATE TABLE video_clips (
  id BIGSERIAL PRIMARY KEY,
  cluster_id     BIGINT REFERENCES defect_clusters(id),
  device_id      BIGINT REFERENCES devices(id),
  s3_key         TEXT, checksum_sha256 TEXT,
  duration_s     REAL, fps INTEGER, resolution TEXT, size_bytes BIGINT,
  quality_status TEXT,   -- accepted|rejected_dark|rejected_blur|rejected_not_road
  vision_status  TEXT DEFAULT 'queued',   -- queued|processing|done|failed
  vision_verdict TEXT,   -- pothole_confirmed|no_defect|scene_excluded
  scene_class    TEXT,
  unique_tracks  INTEGER,
  max_confidence REAL,
  best_crop_url  TEXT,
  uploaded_at    TIMESTAMPTZ DEFAULT now(),
  processed_at   TIMESTAMPTZ,
  purge_after    TIMESTAMPTZ            -- uploaded_at + 7 days; enforced by cron
);

-- ═══════════ SUPPRESSION (the false-positive memory) ═══════════
CREATE TABLE suppression_zones (
  id BIGSERIAL PRIMARY KEY,
  geom        GEOMETRY(Polygon, 4326) NOT NULL,
  reason      TEXT NOT NULL,     -- traffic_jam|signal|construction|crossing|toll|unpaved
  weight      REAL DEFAULT 0.2,  -- multiplier, NEVER 0 (down-weight, don't block)
  source      TEXT,              -- 'vision' | 'manual_admin'
  min_magnitude_exempt REAL,     -- high-jerk events above this ignore suppression
  created_at  TIMESTAMPTZ DEFAULT now(),
  expires_at  TIMESTAMPTZ NOT NULL,
  strike_count INTEGER DEFAULT 1  -- escalating TTL: 3d -> 7d -> 30d
);
CREATE INDEX ON suppression_zones USING GIST (geom);

-- ═══════════ WORK ORDERS (closing the loop) ═══════════
CREATE TABLE work_orders (
  id BIGSERIAL PRIMARY KEY,
  cluster_id  BIGINT REFERENCES defect_clusters(id),
  org_id      BIGINT REFERENCES organisations(id),
  ward_id     BIGINT,
  assigned_to TEXT, contractor TEXT,
  priority    TEXT, status TEXT,
  est_cost    NUMERIC, actual_cost NUMERIC,
  created_at  TIMESTAMPTZ DEFAULT now(),
  due_at      TIMESTAMPTZ, completed_at TIMESTAMPTZ,
  verified_by_silence BOOLEAN DEFAULT false   -- our audit feature
);

-- ═══════════ MATERIALISED VIEWS for the dashboard ═══════════
CREATE MATERIALIZED VIEW mv_ward_stats AS
SELECT ward_id,
       COUNT(*) FILTER (WHERE state = 'confirmed')                   AS open_defects,
       COUNT(*) FILTER (WHERE priority = 'P1' AND state='confirmed')  AS critical,
       AVG(EXTRACT(EPOCH FROM (repaired_at - confirmed_at))/86400.0)  AS mttr_days,
       COUNT(*) FILTER (WHERE state = 'reopened')                     AS failed_repairs,
       AVG(severity_score)                                            AS avg_severity
FROM defect_clusters GROUP BY ward_id;
-- REFRESH MATERIALIZED VIEW CONCURRENTLY mv_ward_stats;  -- every 5 min
```

Why these choices:

- **PostGIS, not plain Postgres:** we need `ST_DWithin`, `ST_ClusterDBSCAN`, GiST indexes and geometry types. Doing geo queries by hand in Python is a guaranteed performance disaster.
- **Partition `sensor_events` by day:** this is the only high-volume table. Partitioning makes retention (`DROP PARTITION`) instant instead of a multi-hour `DELETE`.
- **Immutable events + separate derived table:** we can always re-run clustering with better parameters. If you mutate events in place, you can never re-derive.
- `cluster_state_log` **is append-only:** this *is* the audit trail we sell.

15.2 Pipelines

```
PIPELINE A — INGESTION (hot path, must be fast)
  App → POST /v1/events (gzip, batched)
      → FastAPI: pydantic validate → JWT/device-token auth → rate limit (Redis)
      → write to Redis Stream "raw_events"  → return 202 + commands  [< 50 ms]
  Celery worker (consumer group):
      → map-match to OSM segment (local OSRM container)
      → compute h3_r10
      → apply device trust + suppression weight
      → COPY-batch insert into sensor_events  (batch of 500, not row-by-row)
      → increment segment_passes

PIPELINE B — CLUSTERING (every 5 min, Celery Beat)
      → per active H3 cell: fetch 7-day events
      → DBSCAN in projected metres → weighted centroid → posterior → severity
      → advance state machine → write cluster_state_log
      → issue commands (ASK_USER / CAPTURE_VIDEO) into the commands table
      → NOTIFY 'cluster_update'  → WebSocket fan-out to dashboards

PIPELINE C — VIDEO (async, GPU)
      → S3 ObjectCreated → Redis queue "video_jobs"
      → worker: checksum → ffmpeg sample → quality gate → YOLO → ByteTrack
                → scene classifier → severity → update cluster + counter2
      → nightly cron: purge video where purge_after < now()

PIPELINE D — TRAINING (offline, weekly)
      → export labelled windows / frames  → Label Studio for human labelling
      → train (Colab/local GPU) → evaluate on held-out device+route+city
      → MLflow: log params, metrics, artefacts
      → if metrics improve: quantise → LiteRT → publish new model version
      → SDK downloads the new model over-the-air (versioned, with rollback)

PIPELINE E — REPORTING (daily)
      → refresh materialised views
      → generate per-ward PDF/CSV: new defects, MTTR, failed repairs, cost/pothole
      → email to city_admin; webhook to ULB systems
```

15.3 API surface

Method	Endpoint	Auth	Purpose
POST	`/v1/devices/register`	app key	Register device, get token + salt-hashed id
POST	`/v1/events`	device token	Batched sensor events + pass counts → returns commands
POST	`/v1/votes`	device token	Human Yes/No/Not-sure
POST	`/v1/clips/presign`	device token	Get pre-signed S3 upload URL
POST	`/v1/clips/{id}/complete`	device token	Signal upload done + checksum
GET	`/v1/model/latest`	device token	OTA model version + download URL
POST	`/v1/consent`	device token	Record/revoke video consent (DPDP)
—	—	—	—
POST	`/admin/auth/login`	—	Municipal login (email + password + TOTP)
GET	`/admin/defects`	JWT	Filter by state/ward/priority/date/bbox
GET	`/admin/defects/{id}`	JWT	Detail + evidence + full state log
PATCH	`/admin/defects/{id}/state`	JWT (city_admin+)	Mark under_repair / repaired
GET	`/admin/tiles/{z}/{x}/{y}.mvt`	JWT	**Vector tiles** — the scalable way to serve many points
GET	`/admin/stats/wards`	JWT	Ward rollups, MTTR, failed repairs
GET	`/admin/export?format=csv|pdf|geojson`	JWT	Reports
WS	`/ws/live`	JWT	Live updates (new confirmed defects, state changes)
—	—	—	—
GET	`/public/defects.geojson`	none	**Confirmed** defects only, coarse, cached — for citizens/press

Note on the public endpoint: exposing confirmed defects publicly is a strategic decision, not a technical one. Public visibility creates political pressure, which is what actually gets potholes fixed. But it must be **confirmed-only and coarsened**, never raw reports (which would leak rider movement patterns).

15.4 Scaling path (be able to answer "what if 1 million riders?")

Stage	Riders	Events/day	Stack
Hackathon demo	5–20 (us)	~5 K	1 VM: Docker Compose — Postgres + Redis + FastAPI + React. Free tier.
Pilot (1 city, buses)	50–500	~500 K	Managed Postgres (Supabase/RDS), Redis, 2 API containers, 1 GPU box for video
City-wide	10 K	~10 M	Read replicas, TimescaleDB or partitioned Postgres, Kafka instead of Redis Streams, S3 + CloudFront, autoscaled workers
National	1 M+	~1 B	Kafka → Flink/Spark streaming, ClickHouse for analytics, H3-sharded clustering, Postgres only for the derived truth layer, tile server on CDN

Key insight for judges: the raw event volume looks scary but events are ~200 bytes and **99% of the intelligence happens in aggregate**, which is exactly the workload columnar stores and stream processors are built for. The architecture does not need to be rewritten to scale — only the components swapped, one layer at a time.

SECTION 16 OF 25

16 THE MUNICIPAL WEB PORTAL

Website design

16.1 Screens

```
0. LOGIN  (municipality only)
   email + password (argon2id) + TOTP 2FA · rate-limited · audit-logged
   ! There is NO public signup. Accounts are provisioned by super_admin.

1. LIVE MAP  (the hero screen — this is what wins the demo)
  ┌──────────────────────────────────────────────┬──────────────────────┐
  │ Google Maps basemap + deck.gl overlays       │ LIVE FEED            │
  │                                              │ ● 14:32 New P1,      │
  │  ● red     = CONFIRMED pothole               │   Ward 7             │
  │  ● orange  = LIKELY                          │ ● 14:29 Repair       │
  │  ? amber   = CANDIDATE (admin only)          │   verified, Ward 3   │
  │  × grey    = REJECTED false positive         │ ! 14:20 RE-OPENED    │
  │  ━ path    = segment health green→amber→red  │   W12 (repair failed)│
  │  ○ hex     = ward-level severity rollup      │                      │
  │  ▒ heatmap = report density                  │ FILTERS              │
  │  ○ ring    = uncertainty radius (honest!)    │ □ P1 □ P2 □ P3       │
  │                                              │ ward ▼  date ▼       │
  │ [Animated pulse on newly confirmed defects]  │ □ show candidates    │
  └──────────────────────────────────────────────┴──────────────────────┘
 All updates arrive over WebSocket. No refresh. Ever.

2. DEFECT DETAIL (slide-over drawer)
   • best video-frame crop with the bounding box drawn
   • severity P1, score 0.82, est. 0.6 m² × 11 cm deep
   • EVIDENCE: "47 reports from 31 vehicles · 6 rider confirmations ·
     verified in 2 clips" ← explainability, not a black box
   • timeline: first seen → confirmed → reported → SLA clock (red if overdue)
   • exposure: ~1,400 vehicles/day, 68% two-wheelers
   • actions: assign to ward engineer · mark under repair · export work order

3. WARD SCORECARD  ← the screen that sells the product
   Ward | Open | P1 | MTTR (days) | Failed repairs | ₹/pothole | Health score
   Sortable, with sparkline trends. Compare wards side by side.
   (Remember the real Bengaluru numbers: ₹60,344 vs ₹20,028 per pothole in
    two wards. THIS table is where that becomes visible and undeniable.)

4. ANALYTICS
   • defects created vs repaired per week (are we winning?)
   • monsoon seasonality curve
   • repeat-offender segments (same spot failing again and again)
   • coverage map: which roads have enough data to trust, which don't
     ! Showing coverage honestly builds MORE trust than pretending
       full coverage. Grey out low-confidence areas.

5. WORK ORDERS · 6. AUDIT LOG · 7. SETTINGS (thresholds, wards, users, API keys)
```

16.2 deck.gl layer plan

Layer	Use	Notes
`ScatterplotLayer`	defect dots	radius by severity, colour by state, `pickable` for the drawer
`IconLayer`	P1 criticals	a distinct warning icon so criticals never get lost among dots
`HeatmapLayer`	report density	shows *where data exists*, useful for coverage honesty
`H3HexagonLayer`	ward/zone rollups	elevation = defect count. This is the "wow" 3D shot for the demo.

Layer	Use	Notes
`PathLayer` / `TripsLayer`	segment health / animated coverage	animated trips replay looks spectacular in a pitch
`PolygonLayer`	suppression zones, ward boundaries	admin-only visibility
`MVTLayer`	vector tiles at scale	**switch to this the moment you exceed ~50 K points**; sending 200 K GeoJSON features to a browser will freeze it

Google Maps + deck.gl integration: use `GoogleMapsOverlay` from `@deck.gl/google-maps` with the Maps JavaScript API (`vector` rendering mode) — this is the officially supported path and it gives correct 3D camera sync.

> **CAUTION**
> **Cost warning:** Google Maps JavaScript API is billed per map load. For a hackathon it's inside the free tier, but budget for it, keep a **MapLibre GL + OpenStreetMap fallback** behind a config flag, and never commit an unrestricted API key. Restrict the key by HTTP referrer on day one.

16.3 Live updates without refresh

```
Clustering job updates a cluster
        ▼
PostgreSQL:  NOTIFY cluster_update, '{"id":123,"state":"confirmed",...}'
        ▼
FastAPI background task LISTENs on that channel
        ▼
Broadcasts over WebSocket to all connected dashboards subscribed to that ward
        ▼
React: Zustand store patches just that one feature → deck.gl re-renders only the diff
```

- Fall back to **SSE** if WebSockets are blocked by a government proxy (this happens more often than you'd think).
- Fall back to **polling every 15 s** if SSE also fails. Always have this third option, especially for a live demo on venue WiFi.
- **Throttle**: batch updates into one message per 2 seconds. Otherwise a burst of 200 new defects will jank the UI.

16.4 Interactions and animations that matter (and ones that don't)

Worth building: newly-confirmed defect pulses once and the map gently flies to it; smooth `flyTo` on clicking a list item; hover tooltip; time-slider scrubbing through the last 30 days; before/after crop comparison for repairs; smooth colour transition when a defect changes state.

Not worth building: a 3D city model, particle effects, a custom cursor, an intro splash animation. These cost hours and win nothing. Judges reward *information density and clarity*, not visual noise.

16.5 Accessibility & practical requirements (genuinely matters for a govt buyer)

- Colour is never the only signal — pair colour with shape/icon (red-green colour blindness affects ~8% of men).
- Full keyboard navigation; visible focus rings; `aria-label` on every icon button.
- The map must have a **table view equivalent** — a screen reader cannot read a canvas map. This is also a legal requirement for government procurement in many jurisdictions.
- Contrast ratio ≥ 4.5:1 for text.
- Works on a 1366×768 laptop — that is what a municipal office actually has, not a 4K monitor.
- Works in Chrome on Windows 10. Test there, not just on your MacBook.

SECTION 17 OF 25

17 COMPLETE TECH STACK (BASIC → ADVANCED)

Basic → advanced

Legend: **MUST COMPULSORY** (project fails without it) · **REC STRONGLY RECOMMENDED** · **OPT OPTIONAL / NICE-TO-HAVE** · **v2 v2 / FUTURE**

17.1 The absolute basics (everyone on the team must know these)

Tech	Priority	Why it's here	Feasibility
HTML5	**MUST**	Semantic structure of every page. `<canvas>` is what deck.gl draws into.	Trivial
CSS3 (flexbox, grid, custom properties)	**MUST**	Dashboard layout. Grid for the ward table, flexbox for the map/panel split.	Easy
JavaScript ES6+	**MUST**	Everything in the browser. async/await, destructuring, modules, array methods.	Easy
Git + GitHub	**MUST**	6 people cannot work without branches and PRs.	Easy, but **enforce a branching convention on day 1** or you will lose a day to merge hell
JSON	**MUST**	Every API payload.	Trivial
REST concepts (verbs, status codes, idempotency)	**MUST**	Our whole API. Understand *why* POST /events returns 202 not 200.	Easy
SQL (joins, indexes, aggregates, GROUP BY)	**MUST**	Non-negotiable. If nobody can write a JOIN, the project stalls.	Medium — **assign one person to own this**
Linux CLI + SSH	**MUST**	Deployment, logs, debugging the server at 2 a.m.	Easy
HTTP/HTTPS, DNS, TLS basics	**REC**	Why the app can't reach the server is usually one of these.	Easy

17.2 Mobile app

Tech	Priority	Why	Feasibility for us
Kotlin + Android SDK (native)	**MUST Recommended path**	We need `SensorManager` at 100 Hz, a reliable `ForegroundService`, `CameraX`, `Doze`-mode survival, and precise wake-lock control. Native gives full access with no plugin gaps.	Medium. **Highest-risk component.** Assign 2 people.
— *alternative:* **Flutter +** `sensors_plus` / `camera`	**REC**	Faster UI development, cross-platform.	**!** Risk: plugin sensor sampling rates are less reliable, and long-running background services in Flutter on Android are genuinely painful. **Choose native if anyone on the team knows Kotlin.**
— **NO** *avoid:* React Native for this	—	Bridge overhead on a 100 Hz sensor stream is a real problem.	Don't.
`SensorManager` / `SensorEventListener`	**MUST**	The core data source. Use `SENSOR_DELAY_GAME` (~50 Hz) or `samplingPeriodUs = 10000` (100 Hz).	Easy
`SensorDirectChannel`	**OPT**	Lower-overhead high-rate sensor path on supported devices.	Optional micro-optimisation
`FusedLocationProviderClient` (Play Services)	**MUST**	Better and more battery-efficient than raw GPS. Gives `accuracy`, `speed`, `bearing`.	Easy
`ForegroundService` **+ persistent notification**	**MUST**	Android will kill a background service. A foreground service with a notification is the *only* reliable way to sample sensors continuously. Android 14+ requires a declared `foregroundServiceType`.	Medium — **test on Xiaomi/Realme/Oppo, which have aggressive custom battery killers.** This will bite you.
`WorkManager`	**MUST**	Guaranteed, battery-aware, retrying background upload. Survives reboots.	Easy

Tech	Priority	Why	Feasibility for us
Room (SQLite)	**MUST**	Queue events locally so nothing is lost with no network. **India has patchy connectivity — offline-first is mandatory, not optional.**	Easy
LiteRT / TensorFlow Lite (`org.tensorflow:tensorflow-lite`)	**MUST**	Runs the on-device model. LiteRT is the current successor to TFLite.	Medium
NNAPI / XNNPACK delegate	**OPT**	Hardware acceleration. Unnecessary for a 120 KB model.	Optional
CameraX	**MUST** (for Mode 3)	Video recording, much saner API than Camera2. `VideoCapture` use case, `Quality.HD`.	Medium
Firebase Cloud Messaging (FCM)	**REC**	Push for time-sensitive commands. Mostly avoidable since commands ride on the event-upload response.	Easy
Geofencing API	**REC**	Battery-efficient OS-level geofences. Note the **~100 geofence limit per app** and its latency — for precise, speed-adaptive triggering you may need your own distance check on location updates.	Medium
OkHttp / Retrofit + gzip	**MUST**	Networking. Gzip the batch — it compresses JSON ~5–8×.	Easy
Play Integrity API	**v2**	Anti-spoofing attestation.	v2

17.3 Backend

Tech	Priority	Why	Feasibility
Python 3.11+	**MUST**	Same language as our ML work. One language for backend + ML = one team, no context switching.	Easy
FastAPI	**MUST**	Async (needed for high-concurrency ingestion), automatic OpenAPI docs (huge for parallel app/frontend work), native Pydantic validation, built-in WebSocket support.	Easy — **best choice available**
Pydantic v2	**MUST**	Schema validation at the edge. Rejects malformed events before they poison the DB.	Easy
Uvicorn + Gunicorn	**MUST**	ASGI server + process manager.	Easy
PostgreSQL 16	**MUST**	Relational, transactional, mature, free.	Easy
PostGIS 3	**MUST**	`ST_DWithin`, `ST_ClusterDBSCAN`, GiST indexes, geometry types. **Trying to do geospatial without PostGIS is the single most common fatal mistake in projects like this.**	Medium — learn `ST_DWithin` and SRID 4326 vs projected CRS
SQLAlchemy 2.0 + Alembic	**REC**	ORM + migrations. Migrations matter when 6 people are changing the schema.	Medium
Redis 7	**MUST**	Three jobs: Celery broker, rate limiting, hot cache. Redis **Streams** for durable ingestion buffering.	Easy
Celery + Celery Beat	**MUST**	Async workers + the 5-minute clustering schedule.	Medium
`h3-py`	**REC**	Hexagonal spatial indexing for O(1) neighbour lookup. Not needed at demo scale; essential at national scale.	Easy
`scikit-learn` (DBSCAN)	**MUST**	Clustering. Also our Random Forest baseline.	Easy
OSRM (Docker) or **Valhalla**	**REC**	Map-matching to road segments. Free, self-hosted, uses OSM extracts. **This is what kills GPS perpendicular error** (GAP 3).	Medium — worth the effort
MinIO (dev) / **S3** (prod)	**MUST**	Video object storage with pre-signed URLs, so videos never pass through our API server.	Easy
`ffmpeg` / `PyAV`	**MUST**	Frame extraction from clips.	Easy
JWT (`python-jose`) + `argon2-cffi`	**MUST**	Auth + password hashing. **Argon2id, never MD5/SHA1, never plaintext.**	Easy

Tech	Priority	Why	Feasibility
`slowapi` rate limiting	MUST	An open ingestion endpoint *will* be abused.	Easy
Nginx / Caddy	REC	Reverse proxy, TLS termination. Caddy does Let's Encrypt automatically.	Easy
Docker + Docker Compose	MUST	"Works on my machine" is fatal with 6 people. Compose gives everyone an identical stack in one command.	Medium — **highest ROI investment in the whole project**
Kafka / Flink	v2	Replaces Redis Streams at national scale.	v2
TimescaleDB / ClickHouse	v2	Time-series and analytics at scale.	v2
Kubernetes	v2	NO **Do NOT use for a hackathon.** It will consume two days and win zero points.	Actively harmful now

17.4 Frontend

Tech	Priority	Why	Feasibility
React 18 + TypeScript	MUST	Component model + type safety across a 6-person team. TS catches the "I renamed that field" class of bug at compile time.	Medium
Vite	MUST	Fast dev server and build.	Easy
deck.gl (`@deck.gl/react`, `/layers`, `/aggregation-layers`, `/google-maps`)	MUST	GPU-accelerated rendering of 100K+ points. Plain Google Maps markers die at ~1,000.	Medium-Hard — **budget real learning time; assign one owner**
Google Maps JavaScript API (vector mode)	MUST (per spec)	Basemap. Indian road/landmark data is better than OSM in many cities.	Easy — **! billed per load; restrict the key; keep a MapLibre fallback**
MapLibre GL JS	REC	Free OSM fallback. Insurance against a billing surprise or a quota block on demo day.	Easy
TanStack Query	REC	Server-state caching, refetching, optimistic updates. Removes a lot of hand-written state code.	Medium
Zustand	REC	Simple global store for map filters and the live feed. Much less ceremony than Redux.	Easy
Tailwind CSS	REC	Fast, consistent styling without writing CSS files.	Easy
shadcn/ui + Radix	REC	Accessible, unstyled primitives (dialog, drawer, select, table). Accessibility for free.	Easy
Recharts or **visx**	REC	Charts for the analytics screen.	Easy
`framer-motion`	OPT	Tasteful transitions and the "new defect" pulse. Don't overdo it.	Easy
`react-window`	OPT	Virtualised list for thousands of defect rows.	Easy
Next.js	OPT	Only if you want SSR/SEO for a public marketing page. Adds complexity to the dashboard.	Optional

17.5 ML / AI

Tech	Priority	Why	Feasibility
NumPy + Pandas	MUST	All data wrangling.	Easy
SciPy (`signal`)	MUST	Butterworth band-pass filter, `find_peaks`, `resample`, FFT/`welch`.	Easy
scikit-learn	MUST	Random Forest / XGBoost baseline, `StandardScaler`, metrics, DBSCAN.	Easy
PyTorch	MUST	Train the 1D-CNN. Also what Ultralytics uses underneath.	Medium

Tech	Priority	Why	Feasibility
Ultralytics YOLO (v8 / v11)	**MUST**	Vision detector. `model.train(data='rdd.yaml', imgsz=960)` — genuinely 3 lines to a working model.	Easy — **! check the AGPL-3.0 licence implications for a commercial product; Ultralytics also sells a commercial licence.** Flag this now, not later.
ByteTrack	**MUST**	Multi-object tracking → unique defect counting. Fixes the duplicate bug.	Medium
OpenCV	**MUST**	Frame handling, blur/luminance quality gates, drawing boxes.	Easy
Albumentations	**REC**	Motion blur, rain, brightness augmentation.	Easy
`tf.lite` **/ LiteRT converter**	**MUST**	Export + INT8 quantise the on-device model.	Medium
ONNX / ONNX Runtime	**OPT**	Framework-agnostic export path; sometimes faster server inference.	Optional
Label Studio or **CVAT**	**REC**	Labelling our own sensor windows and video frames. Self-hosted, free.	Easy
Roboflow	**REC**	Fastest path to a merged, augmented, YOLO-formatted dataset. Free tier is enough.	Easy
MLflow or **Weights & Biases**	**REC**	Experiment tracking. Without it, by day 10 nobody remembers which of 40 runs was the good one. **Learned the hard way by every team ever.**	Easy
PyWavelets	**OPT**	Wavelet features (the RoADS approach) if the CNN underperforms.	Easy
Colab / Kaggle free GPU	**MUST**	Free T4/P100. Enough to fine-tune YOLOv8n/s.	Easy — **!** sessions time out; checkpoint to Drive every epoch

17.6 DevOps, observability, security

Tech	Priority	Why	Feasibility
Docker Compose	**MUST**	One command, identical stack, 6 developers.	Medium
GitHub Actions	**REC**	Run tests + lint on every PR. Catches breakage before it reaches main.	Easy
`pytest` + `httpx`	**REC**	API tests. **At minimum test the clustering logic** — it is the part most likely to be silently wrong.	Easy
`ruff` + `black` + `mypy`	**REC**	Lint/format/typecheck Python. Ends all style arguments.	Easy
ESLint + Prettier	**REC**	Same for the frontend.	Easy
Sentry	**REC**	Crash reporting for app + backend. You cannot debug a rider's phone remotely without it.	Easy
structlog / JSON logging	**REC**	Structured logs you can actually grep and correlate by request id.	Easy
Prometheus + Grafana	**OPT**	Metrics dashboards. Nice, but Docker logs suffice for a demo.	Optional
Railway / Render / Fly.io	**REC**	Easiest deployment for a hackathon. Free tiers, git-push deploy.	Easy
AWS EC2/RDS/S3 or **Oracle Cloud free tier**	**OPT**	For a longer pilot. Oracle's always-free ARM instances are genuinely generous.	Medium
Cloudflare	**OPT**	Free CDN, DDoS protection, TLS.	Easy
`.env` + `python-dotenv`**, secrets never in git**	**MUST**	**A leaked Google Maps key or DB password is a real, immediate cost.** Add `.env` to `.gitignore` in the first commit.	Easy — **but the most commonly violated rule in hackathons**

17.7 Feasibility summary of the stack

Component	Difficulty	Risk	Time (6-person team, AI-assisted)
Backend API + DB schema	Medium	Low	3–4 days
Clustering + state machine	**Medium-Hard**	**Medium** — the logic is subtle	3–4 days
Android sensor collection	Medium	**HIGH** — OEM battery killers, permissions, 100 Hz reliability	4–5 days
On-device model + TFLite	Medium	Medium — depends on our own data	3–4 days
Video capture + upload	Medium	**HIGH** — consent, CameraX, geofence timing	3 days
YOLO fine-tune + ByteTrack	Easy-Medium	Low — very well-trodden path	2–3 days
React + deck.gl dashboard	**Medium-Hard**	Medium — deck.gl learning curve	4–5 days
WebSocket live updates	Medium	Low	1–2 days
Auth + RBAC	Easy	Low	1–2 days
Docker + deploy	Medium	Medium	2 days
Data collection drives	Easy	**HIGH** — needs physical time on roads, weather-dependent	3–4 days (parallel)

SECTION 18 OF 25

18 THINGS EVERYONE OVERLOOKS (DO NOT SKIP THESE)

Hidden traps

18.1 Android will fight you (the #1 practical risk)

- **OEM battery killers.** Xiaomi (MIUI), Oppo, Vivo, Realme and OnePlus aggressively kill background services regardless of what Android's docs say. Our foreground service *will* be killed on some phones. Mitigations: request battery-optimisation exemption, guide the user to "Autostart" settings, and — critically — **test on a real Xiaomi device, not just an emulator or a Pixel.**
- **Android 14+ requires** `foregroundServiceType` to be declared and justified. `dataSync` or `location` as appropriate. Get this wrong and the app crashes on launch on new devices.
- **Background location** needs `ACCESS_BACKGROUND_LOCATION` and a Play Store justification form. As an SDK we inherit the host's permission — say this to judges, it's a genuine advantage.
- **Sensor sampling is best-effort.** Requesting 100 Hz gives you ~80–110 Hz with jitter. **You must resample onto a fixed grid** or every FFT feature is silently corrupted.
- **Doze mode and App Standby** will throttle network. `WorkManager` handles this correctly; hand-rolled threads do not.
- **Clock skew.** Device clocks are wrong by minutes. Send both device time and let the server record receipt time; use server time for ordering.

18.2 GPS reality

- **Cold start** takes 10–60 seconds. The first few fixes after starting are bad — discard fixes with `accuracy > 50 m`.
- **Urban canyons**: multipath from glass and metal can shift position by tens of metres; documented pothole-localisation error is **27–32 m at highway speeds**. Design for it, don't wish it away.
- **Tunnels, flyovers, underpasses**: complete signal loss. Detect gaps and don't interpolate across them.
- `accuracy` **is a 68% confidence radius**, not a guarantee. Use it as a weight, not a filter alone.
- Android 11+ on some hardware supports **3D-mapping-aided GNSS corrections**, which materially improves urban accuracy. Use `Location.getAccuracy()` and prefer high-accuracy fixes when available.
- **Sensor and GPS timestamps use different clocks** (`elapsedRealtimeNanos` vs epoch). Align them explicitly or your event locations will be offset by seconds — which at 40 km/h is tens of metres.

18.3 Legal, privacy, and consent (this can kill the project, not just delay it)

Under the **Digital Personal Data Protection Act, 2023** (DPDP Rules notified 13 Nov 2025; phased compliance, Schedule-1 penalties up to **₹250 crore**, full enforcement from 13 May 2027), we are a **Data Fiduciary**. Requirements we must design for now:

Requirement	What we do
Notice & specific consent	Plain-language notice at onboarding, in English + Hindi + regional language. Video is a *separate* toggle, default OFF.
Purpose limitation	Data used only for road condition assessment. Written into the privacy policy and enforced in code (no secondary use without new consent).
Data minimisation	We do **not** store continuous location traces. Only discrete event points. **This is the most important privacy decision in the project** — a continuous trace of a delivery rider is a surveillance dataset; a scatter of pothole hits is not.
No PII	`device_hash = sha256(install_id + server_salt)`. No phone number, no name, no IMEI, no advertising ID. Store `model_class`, not exact model (reduces fingerprinting).
Right to erasure & access	An endpoint that deletes everything tied to a `device_hash`. Must actually work.
Consent withdrawal	One toggle, effective immediately, and it must stop collection — not just hide it.
Retention limits	Raw video ≤ 7 days. Raw events ≤ 90 days, then aggregate only. Enforce with a cron job, and log the deletions.
Breach notification	Documented incident process.

Requirement	What we do
Security safeguards	TLS everywhere, encryption at rest, least-privilege DB roles, audit logs, argon2id passwords.
Consent records	Store `consent_version` + `consent_at` per device (see schema). Records retained per the rules.

Additional legal points to have an answer for:

- **Video may capture faces and licence plates.** Blur before storage (v2), crop to the lower road region (v1), retain only the defect crop.
- **Municipal liability.** Once we tell a municipality about a pothole and it isn't fixed, we have created a documented record of negligence. That is powerful — and it is exactly why some officials may resist. **Frame it as "helping you prove you acted", not "proving you failed."** Position the tool as protecting the honest engineer who can now show a prioritised, evidence-backed queue.
- **Our own liability.** Add a clear disclaimer: this is a decision-support system, not a safety-critical warning system. Never present it as something a rider should rely on to avoid a pothole in real time.
- **Google Maps Terms of Service** restrict certain uses of the basemap and prohibit some kinds of data extraction. Read the ToS before building a product on it, and keep the MapLibre/OSM fallback viable.
- **Ultralytics YOLO is AGPL-3.0.** For a commercial product this is a real constraint — either open-source our stack, or buy the Ultralytics commercial licence, or use an Apache/MIT-licensed detector. **Decide this before writing the business plan into a contract.**
- **Dataset licences.** Khandakar et al. is CC BY-NC-**ND** (non-commercial, no-derivatives) — fine for research and a hackathon, but check carefully before commercial training. RDD2022 has its own terms. Track the licence of every dataset in a table in the repo.

18.4 The demo itself (where hackathon projects actually die)

- **Venue WiFi will fail.** Have (a) a mobile hotspot, (b) a fully local Docker Compose deployment, and (c) a pre-recorded video of the working flow. All three.
- **Never live-train anything on stage.** Show pre-computed results.
- **Seed the database.** A map with 4 dots is unimpressive. Pre-load a few thousand realistic synthetic points across a real city so the map looks alive — and **say clearly which data is real and which is seeded.** Getting caught faking is fatal; disclosing it is fine and shows integrity.
- **Have a "simulate a rider" button** that replays a recorded sensor trace through the real pipeline. This lets you demonstrate the full end-to-end flow — event → cluster → command → confirm → map update — in 30 seconds without leaving the room. **Build this. It is the single highest-value demo asset.**
- **Rehearse the 3-minute version and the 8-minute version.** You will not know which you get.
- **Bring a phone with the app already installed and permissions already granted.** Never grant permissions live.
- Charge everything. Bring cables. Bring an HDMI adapter.

18.5 Team and process traps for a 6-person AI-assisted build

- **Define the API contract on day 1** (Section 11.2). It is the only thing that lets app, backend, and frontend work in parallel. Without it, three people block on each other for a week.
- **Use FastAPI's auto-generated OpenAPI docs** as the shared source of truth, and generate TypeScript types from it.
- **One owner per component.** Shared ownership means nobody debugs it.
- `main` **is always deployable.** Feature branches + PR review, even if review is fast.
- **AI writes the code; humans must be able to defend it.** Set a hard rule: **no code is merged unless one human can explain, line by line, what it does and why that approach was chosen.** Judges will ask "why FastAPI and not Django?" and "where exactly is Redis used?" — a team that cannot answer loses instantly, regardless of how good the code is. This directly addresses the brief that 3 members focus on understanding the code and all 6 on understanding the workflow, stack and rationale.
- **Write the "why" down as you go.** Keep a `DECISIONS.md` with one paragraph per significant choice. It becomes your Q&A prep for free.
- **Start data collection in week 1, not week 3.** It is weather-dependent and physically time-boxed. Every other task can be compressed; driving on roads cannot.

18.6 Technical details that silently break things

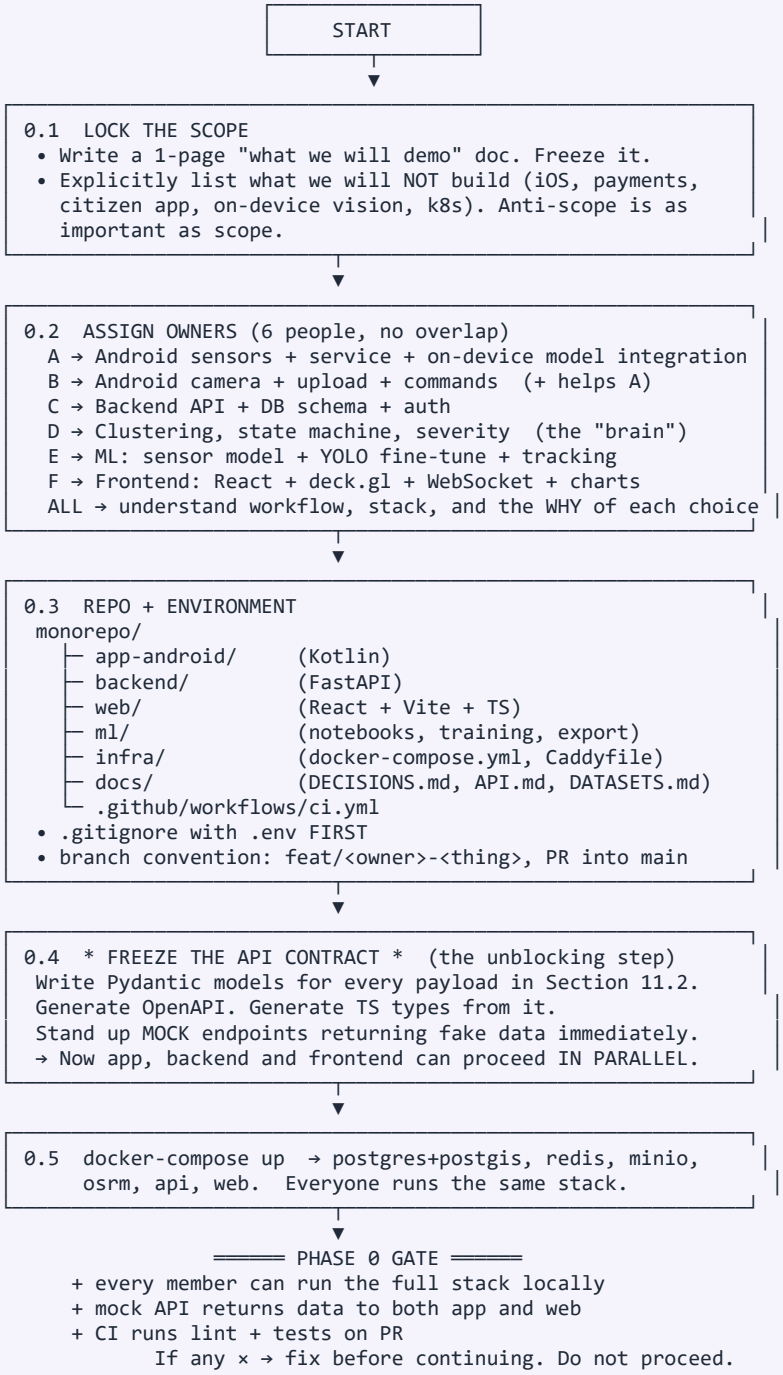
- **DBSCAN on raw lat/lon degrees.** Covered above. This bug is invisible — it produces plausible-looking, wrong clusters.
- **Timezones.** Store everything in `TIMESTAMPTZ` / UTC. Display in IST. Mixing these produces bugs that only appear at 00:00 IST.
- **Float precision for coordinates.** Use `double precision` / `NUMERIC(9,6)`. A `float4` latitude is accurate to about 1 metre at best — which is the same order as the thing we're trying to measure.
- **Row-by-row inserts.** Use batched `COPY` or `execute_many`. Row-by-row will cap you at a few hundred writes/second.
- **Missing DB indexes.** Add the GiST index before you have data, not after the query starts taking 40 seconds.
- **Sending 200,000 GeoJSON features to a browser.** It will freeze. Switch to vector tiles (`MVTLayer`) or server-side aggregation past ~50 K points.
- **N+1 queries** in the defect list endpoint. Use joins or `selectinload`.
- **CORS.** You will lose an hour to this. Configure it once, properly, with an explicit origin allowlist — never `*` with credentials.
- **No idempotency on event upload.** If the app retries after a timeout, you'll double-count. Give each event a client-side UUID and make the insert `ON CONFLICT DO NOTHING`.
- **No model versioning.** Log which model version produced every event so you can compare old and new detections fairly.

SECTION 19 OF 25

19 MASTER DEVELOPMENT FLOWCHART

Start to end

19.1 PHASE 0 — FOUNDATION (before any feature code)



19.2 PHASE 1 — DATA (runs in parallel with everything; start immediately)

```
   □ inspect: 10 CSVs/folder        □ Roboflow pothole (instant baseline)
   □ replot their Fig 9/10 to       □ Chitholian 665
     confirm we read it right       □ TD-RD (patches class)
   □ note sampling 60-99 Hz         □ MIIA, SoV(stereo depth), RAD
                                    □ pre-trained yolov11 road-damage ckpt
                              ▼
1.2 BUILD THE LOGGER APP (before the real app!)  — 1 day
  A bare Android app that ONLY logs raw sensors + GPS to a CSV, with
  6 big label buttons and a 3-tap sync marker.
  ! This must exist before any drive. Do not "collect data later".
                              ▼
1.3 DATA COLLECTION DRIVES (3-4 days, PARALLEL to dev)
  For each run:  2 people (rider + labeller), 2nd phone as dashcam
  Vary: 3 phone models × 3 mounts × 2+ vehicles × day/night
  Targets: 200+ potholes, 100+ speed bumps, 50+ non-road events,
           plus lots of smooth/rough baseline
  ! SAFETY: never label while riding. Pillion or stationary only.
                              ▼
1.4 LABEL & CURATE
  □ align accel & video by the clap spike
  □ hand-label windows FROM VIDEO (button taps lag 300-500 ms)
  □ extract video frames → Label Studio/Roboflow → bounding boxes
  □ collect the confuser images list (Section 10.3)
  □ SPLIT BY ROUTE AND DEVICE — hold out 1 phone + 1 route entirely
                              ▼
1.5 PUBLISH "SETU-IND-1" to Figshare/HuggingFace with a DOI + licence
     (free credibility; closes GAP 7; a real research contribution)
                              ▼
                    ═══════ PHASE 1 GATE ═══════
        + ≥150 km / ≥8 h labelled sensor data across ≥3 devices
        + ≥2,000 labelled image frames incl. confusers
        + held-out device + route + city splits defined and frozen
```

19.3 PHASE 2 — THE THREE PARALLEL TRACKS

```
──────────── TRACK A: MOBILE (owners A, B) ────────────

A1 Foreground service + sensor listener @100 Hz          ─┐
A2 Resample to fixed grid; ring buffer 2 s / 50% overlap  │
A3 Gravity-based orientation correction → vehicle frame   │ core sensing
A4 Band-pass 0.5-30 Hz; per-device calibration (10 min)   │
A5 Threshold gate (the battery saver)                    ─┘
        ▼
A6 TFLite/LiteRT interpreter loads model, runs on survivors
A7 Event queue in Room (SQLite) — OFFLINE FIRST
A8 WorkManager batched gzip upload every 60 s / 50 events
A9 Parse `commands` from the upload response
        ▼
A10 ASK_USER: queue → show ONLY when speed<3 km/h for 5 s → Yes/No/Notsure
A11 CAPTURE_VIDEO: consent check → mount check → speed-adaptive geofence
      → CameraX 6 s @30 fps 720p → on-device frame sanity → pre-signed
      upload → verify checksum → DELETE local file
A12 Battery/data telemetry screen (proves our <2%/h claim to partners)
A13 Test on Xiaomi/Realme/Oppo. Fix the OEM battery-killer issues.
A14 Refactor the sensing core into an SDK module (:setu-sdk AAR)
    + a 30-line demo host app → proves the SDK story to judges


──────────── TRACK B: BACKEND + BRAIN (owners C, D) ────────────

B1  Alembic migrations for the full schema (Section 15.1)
B2  POST /v1/events: validate → auth → rate-limit → Redis Stream → 202
B3  Celery consumer: map-match (OSRM) → h3 → weights → batch COPY insert
B4  segment_passes increment (THE DENOMINATOR — don't forget it)
```

```
B5  Sanity filters: speed>5, gps_acc<50, trajectory continuity, mock-loc
        ▼
B6  * CLUSTERING JOB (Celery Beat, 5 min)
      project→metres → DBSCAN(eps 20, min 5) → distinct-device count(≥5)
      → cap 3/device → inverse-variance centroid + conf ellipse
      → fire_rate = fires/passes → suppression weights
B7  Bayesian posterior fusion (sensor + votes + vision)
B8  Severity score + P1..P4 banding
B9  State machine + append-only cluster_state_log
B10 Command issuer: ASK_USER / CAPTURE_VIDEO, with expiry + dedupe
B11 Video pipeline: presign → S3 event → queue → worker (Track C model)
      → scene classifier → SUPPRESSION ZONE creation → counter2
B12 Device trust score updater
B13 Admin API + JWT + RBAC + TOTP; audit logging on every mutation
B14 Vector tile endpoint (MVT) + materialised views + WebSocket fan-out
B15 Retention crons: purge video >7 d, aggregate events >90 d
B16 * REPLAY HARNESS: feed a recorded trace through the real pipeline
      → the demo button, and also our integration test
```

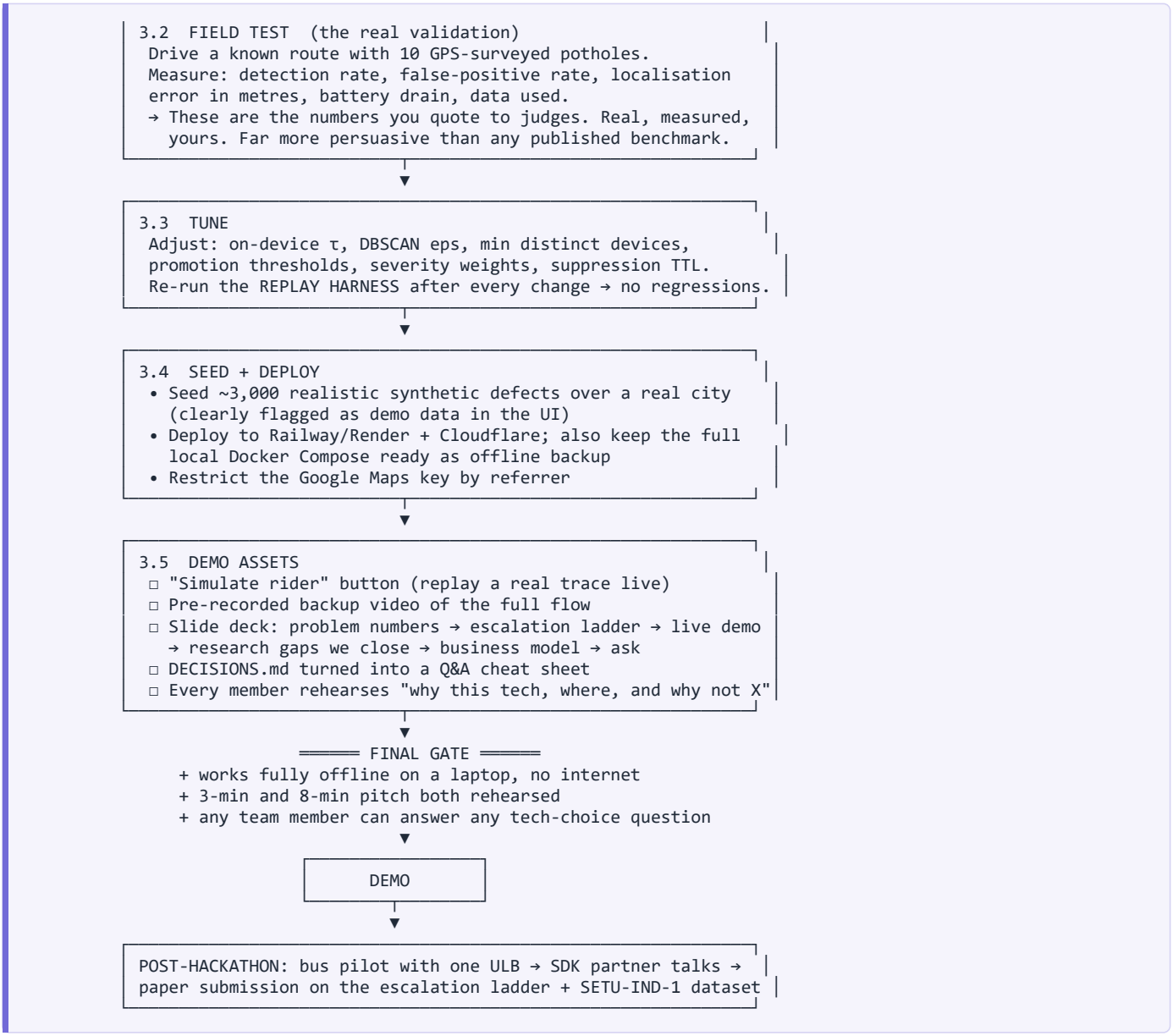
```
TRACK C: ML (owner E)

SENSOR MODEL                              VISION MODEL
C1 EDA on Khandakar set; reproduce        C6 YOLOv8n on Roboflow pothole
   their bump-vs-pothole stats               set → working baseline in 1 h
C2 Feature pipeline (time/freq/wavelet    C7 Fine-tune YOLOv8s on RDD2022
   /cross-axis/context)                      India split, imgsz=960
C3 Random Forest baseline → confirm       C8 Add TD-RD, Chitholian, MIIA,
   ~88% precision                            RAD + our own frames
C4 1D-CNN (Section 12.3) → beat it        C9 Copy-paste aug for D40 balance
C5 INT8 quantise → LiteRT → verify        C10 ByteTrack → unique counting
   on-device accuracy ≈ desktop           C11 Scene classifier (jam/signal/
   (quantisation CAN cost 1-3%)               construction/crossing/unpaved)
                                          C12 Quality gate (dark/blur/road)
                                          C13 Depth/size estimation
C14 MLflow tracking for BOTH. Report per-class P/R + confusion matrix.
C15 Evaluate on held-out DEVICE and held-out ROUTE separately.
    ! If accuracy does NOT drop, suspect leakage.
```

```
TRACK D: FRONTEND (owner F)
D1 Vite + React + TS + Tailwind + shadcn/ui scaffold
D2 Login + JWT + protected routes + role-based menu
D3 Google Maps (vector mode) + deck.gl GoogleMapsOverlay
   + MapLibre fallback behind a flag
D4 ScatterplotLayer from mock API → then real API
D5 Icon/Heatmap/H3Hexagon/Path/Polygon layers + legend
D6 Defect detail drawer: crop, evidence breakdown, timeline, SLA clock
D7 WebSocket live feed + 2 s batching + SSE and polling fallbacks
D8 Ward scorecard table (MTTR, failed repairs, ₹/pothole) + sparklines
D9 Analytics charts + coverage-honesty layer (grey out low-data areas)
D10 Work orders, audit log, settings, CSV/PDF/GeoJSON export
D11 Accessibility pass: keyboard, aria, contrast, table view of the map
D12 Test at 1366×768 in Chrome on Windows
```

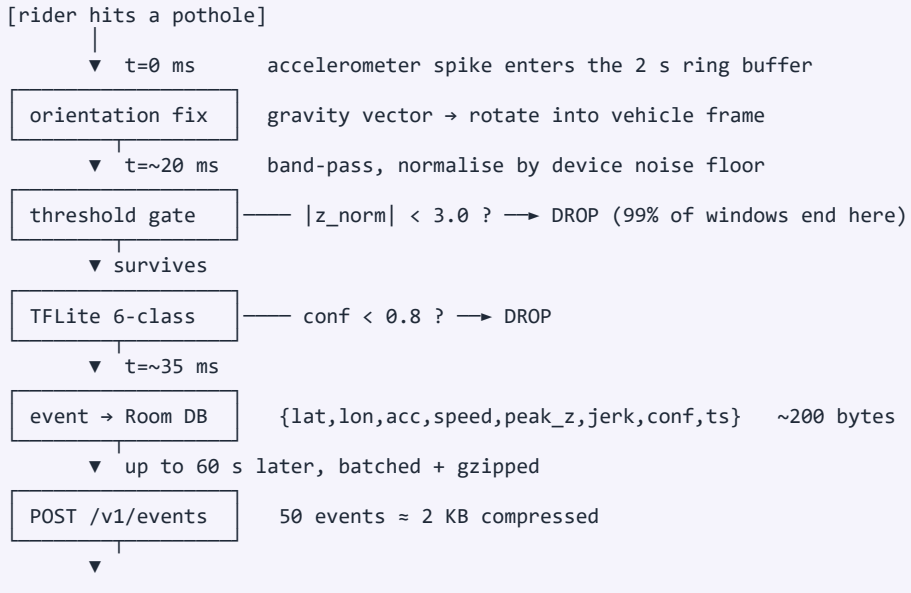
```
═══════ PHASE 2 GATE ═══════
+ real event from a real phone appears as a dot on the real map
+ clustering promotes candidate→confirmed on seeded data
+ a CAPTURE_VIDEO command round-trips: issued → recorded → uploaded → YOLO → confirmed
+ sensor model ≥85% precision on held-out DEVICE
+ vision model ≥60% mAP50 on held-out India test set
```

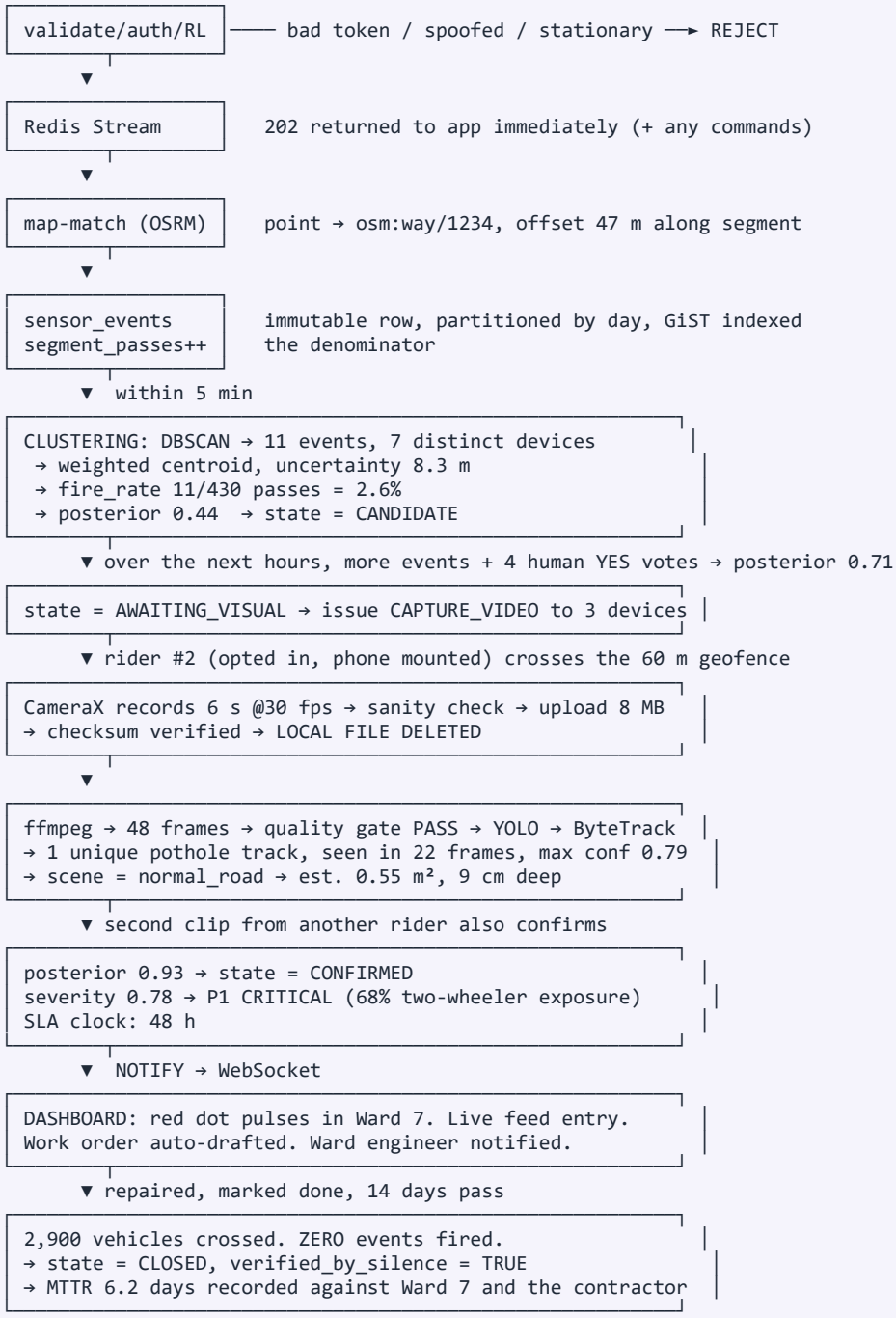
19.4 PHASE 3 — INTEGRATION, HARDENING, DEMO

```
3.1  END-TO-END INTEGRATION
 Swap every mock for the real endpoint. One flow, no gaps:
 phone → event → cluster → command → vote → video → YOLO →
 confirmed → map → work order → repair → verified by silence
                          ▼
```



19.5 Runtime data flow (one event's complete life)




validate/auth/RL
bad token / spoofed / stationary → REJECT
Redis Stream
202 returned to app immediately (+ any commands)
map-match (OSRM)
point → osm:way/1234, offset 47 m along segment
sensor_events
segment_passes++
immutable row, partitioned by day, GiST indexed
the denominator
within 5 min
CLUSTERING: DBSCAN → 11 events, 7 distinct devices
→ weighted centroid, uncertainty 8.3 m
→ fire_rate 11/430 passes = 2.6%
→ posterior 0.44 → state = CANDIDATE
over the next hours, more events + 4 human YES votes → posterior 0.71
state = AWAITING_VISUAL → issue CAPTURE_VIDEO to 3 devices
rider #2 (opted in, phone mounted) crosses the 60 m geofence
CameraX records 6 s @30 fps → sanity check → upload 8 MB
→ checksum verified → LOCAL FILE DELETED
ffmpeg → 48 frames → quality gate PASS → YOLO → ByteTrack
→ 1 unique pothole track, seen in 22 frames, max conf 0.79
→ scene = normal_road → est. 0.55 m², 9 cm deep
second clip from another rider also confirms
posterior 0.93 → state = CONFIRMED
severity 0.78 → P1 CRITICAL (68% two-wheeler exposure)
SLA clock: 48 h
NOTIFY → WebSocket
DASHBOARD: red dot pulses in Ward 7. Live feed entry.
Work order auto-drafted. Ward engineer notified.
repaired, marked done, 14 days pass
2,900 vehicles crossed. ZERO events fired.
→ state = CLOSED, verified_by_silence = TRUE
→ MTTR 6.2 days recorded against Ward 7 and the contractor

SECTION 20 OF 25

20 WEEK-BY-WEEK ROADMAP

Timeline

Assumes a ~6-week runway. Compress or expand proportionally.

Week	Everyone	A+B (Mobile)	C+D (Backend)	E (ML)	F (Frontend)
1	Scope freeze, owners, repo, **API contract frozen**, docker-compose up	Logger app; sensor service; 100 Hz verified on 3 phones	Schema + migrations; `/v1/events`; Redis stream; mock endpoints live	Download all datasets; EDA; reproduce Khandakar stats; **YOLOv8n baseline in 1 h**	Scaffold; login; Google Maps + deck.gl "hello dots" from mock API
2	**Data drives (all hands, 2 days)**	Orientation correction, calibration, threshold gate, Room queue	Map-matching (OSRM); batch insert; segment_passes; sanity filters	Label sensor data; feature pipeline; **RF baseline ~88% precision**	Real API wiring; all deck.gl layers; legend; filters
3	Mid-point review: cut anything not on track	TFLite integration; batched upload; command parsing	**Clustering job + DBSCAN + weighted centroid + state machine**	1D-CNN training; **YOLOv8s on RDD2022 India**; ByteTrack	Defect drawer; WebSocket live feed; ward scorecard
4	—	ASK_USER (stopped-only); CameraX capture; geofence; upload+delete	Video pipeline; scene classifier hookup; **suppression zones**; trust scores	Quantise to LiteRT; verify on-device; scene classifier; quality gate; depth	Analytics; coverage honesty; export; accessibility pass
5	**End-to-end integration + field test on a surveyed route**	OEM battery-killer fixes; SDK refactor (AAR + demo host)	Admin API, RBAC, TOTP, MVT tiles, materialised views, **replay harness**	Final training; per-device/route/city eval; MLflow writeup	Polish, animations, 1366×768 + Chrome/Windows test
6	**Tune thresholds → seed DB → deploy → rehearse ×3**	Battery/data telemetry screen	Retention crons; load test; offline Docker bundle	Model card; publish SETU-IND-1 dataset	Backup video; slide deck; demo script

Hard rule: if a component is not working by end of week 4, cut it and fall back. Fallback ladder, in order of what to drop first:

1. Drop video capture → demo Mode 3 with a pre-uploaded clip (the vision AI still runs live).
2. Drop the 1D-CNN → ship the Random Forest (88% precision is honestly fine).
3. Drop ASK_USER prompts → show the flow with seeded votes.
4. Drop the SDK refactor → present it as architecture, with the module boundary visible in code.
5. **Never drop:** sensor collection, clustering, the live map. Those three *are* the project.

SECTION 21 OF 25

21 FEASIBILITY ANALYSIS

Can 6 people do it

21.1 Overall verdict

Aspect	Verdict	Confidence
Technical feasibility	OK **HIGH.** Every single component has published precedent (Section 5) and mature open-source tooling. We are integrating known parts in a novel arrangement, not inventing new science.	90%
Hackathon deliverability (6 weeks, 6 people, AI-assisted)	OK **HIGH for the core**, ! **MEDIUM for the full vision.** The sensor→cluster→map spine is very achievable. Video capture on real devices is the risky part.	80% core / 60% full
Model accuracy	OK **HIGH for the system**, ! **MEDIUM for individual models.** Published ceilings are ~88–98% (sensor) and ~65–82% F1 (vision). Our escalation ladder means we don't need any single model to be excellent.	85%
Real-world deployment	! **MEDIUM.** Needs a platform partner (Swiggy/Ola) or a fleet owner (a ULB with buses). Technically ready; commercially requires a signature.	60%
Business viability	OK **MEDIUM-HIGH.** RoadMetrics (50,000+ km, adopted by Chennai) and RoadBounce prove municipalities buy this. Our cost structure is far lower.	70%
Scientific novelty (publishable)	OK **HIGH.** The escalation ladder, per-device calibration in a crowdsourced setting, verified-by-silence repair auditing, and an open Indian two-wheeler sensor dataset are each genuinely novel.	85%

21.2 Why AI-assisted coding makes this feasible for 6 people

Where AI gives the biggest multiplier (do lean on it heavily):

- Boilerplate: Pydantic models, SQLAlchemy models, Alembic migrations, React components, Retrofit interfaces. Days → hours.
- Well-trodden integrations: CameraX recording, WorkManager upload, FastAPI WebSocket, deck.gl layer setup. These have thousands of public examples.
- Data plumbing: pandas transforms, feature extraction functions, augmentation pipelines.
- Training scripts: Ultralytics is 3 lines; PyTorch training loops are template code.
- SQL: complex PostGIS queries, window functions, materialised views.
- Debugging: pasting a stack trace is dramatically faster than reading a forum.
- **Documentation and this very report.**

Where AI will NOT save you (budget human time here):

- ! **Physically driving on roads to collect data.** Irreducible. 3–4 days of real time.
- ! **Labelling.** Even with assist tools, a human must look at the video.
- ! **OEM-specific Android bugs.** "Works on Pixel, killed on Xiaomi" requires holding a Xiaomi.
- ! **Tuning thresholds.** Judgement, informed by field results.
- ! **Deciding what NOT to build.** AI will happily build you 40 features you don't need. Scope discipline is entirely human.
- ! **Understanding the code well enough to defend it.** This is the one thing that cannot be delegated, and it's exactly what the brief asks of the team.

21.3 The learning plan for the team (matches the brief)

All 6 must be able to answer, for every technology in the stack: what is it, where in our system is it used, why it, and why not the obvious alternative. Rehearse this. Sample answers:

Question	Answer to memorise
Why **FastAPI** not Django/Flask?	Async is needed because ingestion is I/O-bound with high concurrency. Auto-generated OpenAPI let three tracks develop in parallel from day 1. Pydantic validation at the edge. Native WebSockets for the live map. Django's ORM+admin are great but we don't need a CMS, and its sync model is wrong for this workload.
Why **PostgreSQL + PostGIS** not MongoDB?	Our core operation is a *geospatial* query (`ST_DWithin`, `ST_ClusterDBSCAN`) over *relational* data with strict integrity (a defect must reference real events). PostGIS is the most mature geospatial engine that exists, and it's free. MongoDB's geo support is far weaker and we'd lose transactions.
Why **Redis**?	Three distinct jobs: Celery message broker, rate limiting (atomic INCR with TTL), and Redis Streams as a durable buffer so an ingestion burst never blocks the API or drops data.
Why **Celery** not run it in the request?	Clustering takes seconds to minutes over millions of rows. Doing it inside an HTTP request would time out and block the app. Celery Beat also gives us the 5-minute schedule for free.
Why **deck.gl** not Google Maps markers?	Google Maps DOM markers become unusable past roughly a thousand. deck.gl renders on the GPU via WebGL and handles 100K+ points at 60 fps, plus it gives us heatmaps and H3 hexagon aggregation out of the box.
Why **on-device** inference not send raw sensors to the server?	100 Hz × 6 axes × 24 h ≈ tens of MB per rider per day. Unaffordable for a gig worker and for us. On-device inference sends only ~200 bytes per *event* — a reduction of several orders of magnitude. It's also better for privacy: raw motion never leaves the phone.
Why **LiteRT/TFLite** not ONNX or PyTorch Mobile?	Best-supported Android runtime, INT8 quantisation to ~120 KB, XNNPACK/NNAPI delegates, and it's the runtime Google itself ships in Chrome and Pixel. LiteRT is the current successor to TFLite.
Why **YOLO** not Faster R-CNN / SSD?	Published comparison on our exact dataset: the YOLO-family approach reached 65.7% mAP on RDD2022, beating both Faster R-CNN and SSD. Single-stage is also far faster, which matters when we process video.
Why **DBSCAN** not k-means?	k-means requires you to know k in advance and forces every point into a cluster. We don't know how many potholes exist, and we specifically *need* outliers labelled as noise rather than absorbed. DBSCAN is density-based and does exactly that.
Why **H3** hexagons not a square grid?	Hexagons have uniform neighbour distance (6 neighbours, all equidistant); squares have 4 near + 4 diagonal neighbours at different distances, which distorts density calculations. H3 is also hierarchical and battle-tested at Uber's scale.
Why **Kotlin native** not Flutter/React Native?	We need reliable 100 Hz sensor sampling and a long-lived foreground service that survives OEM battery managers. Cross-platform plugin layers are unreliable at both. Also, our real product is an *Android SDK*, so native is the correct artefact anyway.
Why **an SDK** not an app?	Distribution: 690,000 Swiggy riders on day one vs 0 downloads. Permissions: we inherit the host's background location. Retention: nobody uninstalls what they can't see. Business: sell to 10 companies, not 100 million citizens.
Why **WebSocket** not polling?	The dashboard must update the instant a defect is confirmed, and polling every second with hundreds of connected clients wastes both server and client resources. We use Postgres `LISTEN/NOTIFY` → WebSocket fan-out, with SSE and polling as documented fallbacks.
Why **Docker Compose** not just install things?	Six developers, one identical stack, one command. It also means our demo runs fully offline on a laptop if venue WiFi dies.

Question	Answer to memorise
Why not **Kubernetes**?	Deliberate choice. It solves problems we do not have at our scale and would cost two days of the runway. Correct engineering is choosing the *simplest* thing that meets the requirement.

21.4 Cost to build (hackathon)

Item	Cost
Cloud hosting (Railway/Render/Fly free tiers)	₹0
Google Maps API (within free tier)	₹0 (! restrict the key)
GPU training (Colab/Kaggle free)	₹0
Datasets (all open)	₹0
Phones	₹0 (our own)
Fuel for data collection drives	~₹1,500
Phone mounts (×3)	~₹1,500
Domain name (optional)	~₹800
Total	**≈ ₹4,000**

Cost to run a real 1-city pilot: roughly ₹15,000–40,000/month (a managed Postgres instance, one small GPU box or spot instances for video, object storage, and a modest Maps API bill).

22

SECTION 22 OF 25

BUSINESS PLAN — HOW THIS MAKES MONEY

How we earn

22.1 Positioning

> ***We are not a pothole app. We are the measurement layer for India's road network*** *— the instrument that tells you, continuously and independently, what condition every road is in and whether the money spent on it worked.*

Analogy for the pitch: *"Nielsen for road quality."* Nielsen doesn't make TV shows; it sells the ratings that everybody in the industry has to buy. We don't fix roads; we sell the ground truth that everybody in the road economy needs.

22.2 Why this is a business and not just a project

The market already exists and is already spending. The problem is that it's spending *blind*:

- BMC: ₹90–203 crore/year on pothole repair tenders alone.
- Indore: ₹50 crore/year pothole budget, ~₹14 lakh/day.
- Pune: ₹1.10 crore/year to run *each* road-repair vehicle, ~₹16 crore/year total.
- Bengaluru: **₹60,344 vs ₹20,028 per pothole in two wards of the same city.**
- India-wide: road crashes cost **3–5% of GDP** annually.
- RoadMetrics has mapped **50,000+ km** and been adopted by Chennai. RoadBounce has been operating since 2016. **Municipalities demonstrably sign these cheques.**

Our advantage: their cost per km requires someone to *drive with the app on*. Ours is effectively **zero marginal cost per km**, because the driving is already happening and paid for by someone else.

22.3 Revenue streams (in the order we should pursue them)

Stream 1 — Municipal SaaS (primary, launch first)

- Annual subscription per city, tiered by road-network length and population.
- Indicative: ₹8–15 lakh/year for a Tier-2 city (≤1,000 km network); ₹40 lakh–1.5 crore/year for a metro.
- Justification is easy: if a city spends ₹50 crore/year on repairs, a ₹40 lakh tool that improves allocation by even 5% pays for itself **6×**.
- Procurement routes: Smart Cities Mission budgets, AMRUT, state urban development departments, GeM (Government e-Marketplace) listing, World Bank/ADB-funded urban projects.
- **!** Reality check: **government sales cycles are 6–18 months** and payment can be slow. Plan cash flow accordingly and don't build a business that needs a municipal cheque in month 3.

Stream 2 — Contractor accountability / audit module (highest margin) This is the killer feature and it prices differently because it saves money directly, not indirectly.

- MTTR per ward and per contractor; failed-repair (`RE-OPENED`) detection; verified-by-silence closure; before/after evidence.
- Sell to: municipal audit departments, CAG-adjacent auditors, state vigilance, and — importantly — **honest contractors**, who currently cannot prove their work was good while a rival's was not.
- ₹5–20 lakh/year as an add-on. Very high margin because it's the same data, re-presented.

Stream 3 — Platform / fleet licensing (the SDK deal)

- Swiggy, Zomato, Ola, Uber, Rapido, Blinkit, Zepto, Delhivery, Porter, Amazon, Flipkart, and B2B fleet operators.
- Two possible shapes:
  - **They pay us**: rider-safety routing, reduced vehicle damage and insurance claims, better ETA prediction (rough roads slow riders down), and an ESG/CSR story. Sell it as a rider-welfare feature — that is politically valuable to them right now given gig-worker labour pressure.
  - **We pay them / revenue-share**: they get a share of municipal revenue derived from data their riders generated.

- Most likely: **free SDK + data-sharing agreement** initially, converting to revenue share once municipal revenue is proven. Getting the data flowing matters more than getting paid by them in year 1.

Stream 4 — Insurance & telematics data

- Motor insurers want road-risk maps for pricing and for claim validation ("was this suspension claim plausible on that route?").
- The *same* sensor stream also scores driving behaviour (Ferreira et al.; Cambridge Mobile Telematics' DriveWell is a proven commercial model) — so we can offer a driver-risk product with **no additional data collection**.
- Context: the connected-car / automotive data-monetisation market is measured in **billions of USD and growing at 12–26% CAGR** depending on segment. Road-condition data is a recognised category inside it (McKinsey explicitly cites cities using sensory data to identify potholes).

Stream 5 — Navigation & mapping partners

- Google Maps, Ola Maps, MapmyIndia/Mappls, HERE, TomTom.
- Pothole and roughness layers improve routing quality, ETA accuracy, and two-wheeler-specific routing (a genuinely underserved need in India).
- Licensed data feed / API.

Stream 6 — ADAS, EV and OEM

- Two-wheeler and car OEMs want road-roughness data for suspension tuning, EV range modelling (rough roads cut range), and ADAS/autonomy. NHAI's own AI-dashcam programme signals where this is going.
- Long-cycle, high-value.

Stream 7 — Research, media and open data

- Publish an annual **"India Road Health Report"** with city rankings. Costs almost nothing, generates enormous press, and creates the political pressure that drives municipal procurement.
- **This is our marketing engine, not a revenue line.** A newspaper headline saying "City X has the worst roads in the state, per SETU data" gets us a meeting with City X's commissioner faster than any sales call.
- Also: licensed research access, and government open-data mandates.

22.4 Go-to-market sequence

```
STEP 1  (months 0-3)   PROVE IT — one small fleet, no partner needed
  Target: ONE Municipal Corporation's own vehicles (buses, garbage trucks,
  water tankers) — alternative S7 from Section 9.
  Why: single decision-maker, no consent complexity, fixed repeat routes
  (perfect for negative evidence + repair verification), and it is a
  paid pilot rather than a free trial.
  Deliverable: "we found N defects you didn't know about, and M of your
  recorded repairs failed." That second number is what sells everything else.
        |
        ▼
STEP 2  (months 3-9)   REFERENCE CUSTOMER
  Convert the pilot into a paid annual contract. Get a public case study
  and a quotable official. Enter the GeM catalogue. Apply to Smart Cities
  and state innovation funds.
        |
        ▼
STEP 3  (months 6-15)  SDK PARTNER
  Approach ONE delivery platform with the municipal contract already in hand.
  Lead with rider welfare and ESG, not with data. Start with one city.
  → Coverage jumps 100x overnight. This is the inflection point.
        |
        ▼
STEP 4  (months 12-24) MULTI-CITY SAAS
  10-20 cities. Standardise onboarding. Build the annual India Road Health
  Report for press and pressure.
        |
        ▼
STEP 5  (months 24-48) DATA BUSINESS
  With national coverage, the insurance, mapping, and OEM streams become
  viable — these all require scale before they have any value.
```

22.5 Unit economics (illustrative, at city scale)

Item	Value
Cost per km of road scanned	**≈ ₹0** (the driving is already paid for)
Marginal cost per sensor event	~₹0.000002 (200 bytes storage + compute)
Marginal cost per verified video clip	~₹0.50–2 (bandwidth + GPU inference + storage)
Clips needed per confirmed defect	~2
Marginal cost per CONFIRMED defect	**≈ ₹1–5**
What a municipality currently pays to *find* a defect	manual inspection, effectively ₹100s–1,000s per defect in staff time
What a municipality pays to *fix* one	**₹17,693 (BMC) to ₹60,344 (Bengaluru ward)**

The pitch line: *"It costs us about ₹3 to confirm a pothole. It costs a city ₹20,000 to ₹60,000 to fix one. If our data improves their prioritisation by even 2%, we have paid for ourselves a hundred times over."*

Fixed costs are the real cost base — engineering salaries, cloud, sales — which means this is a **classic high-gross-margin data SaaS**: brutal to get the first customer, extremely profitable at 20 customers.

22.6 Moats (why we don't get instantly copied)

1. **The data network effect.** Every km driven improves the model, which improves detection, which attracts more partners. A new entrant starts from zero and cannot catch up on coverage.
2. **Platform integration lock-in.** Once our SDK ships inside a major driver app, replacing it requires an app release cycle and a re-negotiation. Very sticky.
3. **The historical record.** Our value compounds: two years of defect history, MTTR per ward, contractor performance. **A competitor can copy the algorithm in a month but cannot copy two years of history.** This is the strongest moat.
4. **Municipal switching costs.** Once work orders, audits and SLA reporting run on our system, switching means retraining staff and losing history.
5. **The open dataset + published research.** Establishes us as the credible authority, which matters enormously in government procurement.
6. **Being government-aligned, not adversarial.** Positioning as "we help you prove you're doing your job" rather than "we're exposing you" determines whether officials become customers or obstacles.

22.7 Honest risks to the business

Risk	Reality	Mitigation
No platform partner signs	Genuinely likely at first. They have no obligation to help us.	Municipal-fleet wedge (buses) needs no partner at all. Prove value first, then approach platforms from strength.
Government sales cycles are brutal	6–18 months, slow payments, tender requirements we may not qualify for	Start with Smart Cities innovation budgets and pilot funds, which are faster. Partner with an existing empanelled vendor if needed.
Municipality doesn't want to be measured	Real political resistance	Sell to auditors and to reform-minded commissioners. Frame as protective, not accusatory. Public data creates pressure from outside.
Google/Uber/a large incumbent builds it	They already have the sensor data and the distribution	Speed, India-specific focus, municipal relationships, and the audit layer they will never build. Also: acquisition is a perfectly good outcome.
Data quality is challenged in a dispute	"Your data says there's a pothole; we say there isn't"	This is exactly why the **video verification layer and uncertainty ellipse exist**. Evidence, with error bars, not assertions.
Privacy backlash	One bad story about "delivery apps secretly recording video" would be devastating	Opt-in by default-OFF, visible indicator, on-device filtering, 7-day retention, published policy, DPDP compliance from day 1. **Do not cut corners here — it is an existential risk, not a compliance chore.**

Risk	Reality	Mitigation
YOLO's AGPL licence	Could force us to open-source, or to pay	Decide early: buy the commercial licence, or use an Apache/MIT detector. Budget for it.

SECTION 23 OF 25

23 RISK REGISTER

What can go wrong

#	Risk	Likelihood	Impact	Mitigation	Owner
R1	Foreground service killed by OEM battery manager (Xiaomi/Oppo/Vivo/Realme)	**HIGH**	HIGH	Test on a real Xiaomi in week 1. Request battery-optimisation exemption, guide user through Autostart, auto-restart via `WorkManager` watchdog, and detect+report gaps in coverage.	A
R2	Not enough labelled sensor data to train a good model	**HIGH**	HIGH	Start drives in week 1. Use the Khandakar dataset as the base and ours for fine-tuning. Keep the Random Forest baseline, which needs far less data. Pothole Lab for synthetic augmentation.	E
R3	Video capture doesn't work reliably (mount, timing, consent, CameraX)	**HIGH**	MEDIUM	Speed-adaptive geofence, mount detection, on-device frame sanity check. **Fallback: pre-uploaded clips so the vision AI still demos live.**	B
R4	GPS error makes clusters land off-road or merge two defects	**HIGH**	HIGH	Map-matching + 20 m DBSCAN + inverse-variance centroid + show the uncertainty ellipse honestly.	D
R5	Too many false positives (speed bumps, jams, phone drops)	**HIGH**	HIGH	6-class model with explicit negative classes, high on-device threshold ($\tau \approx 0.8$), distinct-device requirement, suppression zones, vision as final arbiter.	E, D
R6	Vision model accuracy below expectations	MEDIUM	MEDIUM	Published SOTA is only ~65–75% mAP — plan for it. Require 2 independent clips. Higher input resolution, copy-paste augmentation, tiled inference. Pre-trained YOLOv11 road-damage checkpoint as a safety net.	E
R7	deck.gl learning curve eats the frontend timeline	MEDIUM	MEDIUM	One dedicated owner from day 1. Start from official examples. `ScatterplotLayer` alone is enough for a great demo.	F
R8	Integration hell in week 5 (mocks don't match reality)	MEDIUM	**HIGH**	The frozen API contract in Phase 0 is the entire mitigation. Integrate a thin end-to-end slice in week 2, not week 5.	C
R9	Demo fails on venue WiFi	MEDIUM	**CRITICAL**	Full offline Docker Compose bundle + mobile hotspot + pre-recorded backup video. All three, tested.	ALL
R10	Team member unavailable (illness, exams)	MEDIUM	MEDIUM	No single point of failure: pair each critical component with a documented second reader. `DECISIONS.md` keeps context transferable.	ALL
R11	Scope creep (iOS, citizen app, payments, gamification)	**HIGH**	HIGH	The written anti-scope list from Phase 0. One person empowered to say no.	ALL
R12	Google Maps API bill or quota block	LOW	HIGH	Restrict key by referrer, set a billing alert, keep the MapLibre+OSM fallback behind a config flag and test it.	F
R13	Privacy/consent objection from a judge or partner	MEDIUM	HIGH	Have the DPDP compliance table (18.3) on a slide. Opt-in default-OFF. Show the data-minimisation decision (no continuous traces) proactively.	ALL
R14	Ultralytics AGPL licence conflicts with the commercial story	MEDIUM	MEDIUM	Flag it openly. Plan: commercial licence, or swap to an Apache/MIT detector, or open-source our stack. Know the answer before you're asked.	E
R15	Quantisation drops on-device accuracy below the desktop model	MEDIUM	MEDIUM	Measure both. Use quantisation-aware training if post-training INT8 costs more than ~2%. Keep a float16 fallback.	E
R16	Clustering job times out as data grows	MEDIUM	MEDIUM	Partition by day, GiST + H3 indexes, process only *active* H3 cells, incremental clustering rather than full recompute.	D
R17	Data leakage produces fake high accuracy, then embarrassment on stage	**MEDIUM**	HIGH	Split by route AND device, never randomly. Treat a suspiciously high score as a bug to investigate, not a success. (See the 100%-accuracy paper in 5.1.)	E
R18	The demo map looks empty and unimpressive	MEDIUM	MEDIUM	Seed ~3,000 realistic synthetic defects, clearly labelled as demo data, plus the live "simulate rider" button for real end-to-end proof.	F, D
R19	Judges ask a tech-choice question nobody can answer	MEDIUM	**HIGH**	Section 21.3 table. Every member rehearses it. This is graded, effectively.	ALL
R20	Monsoon / bad weather blocks data collection drives	MEDIUM	MEDIUM	Front-load drives into week 1–2. Have a backup indoor plan (public datasets + Pothole Lab synthetic).	ALL

SECTION 24 OF 25

24 JUDGE Q&A PREPARATION

Demo defence

The hardest questions, with the answers. Rehearse these out loud.

Q: "This already exists — RoadBounce, RoadMetrics, NHAI's dashcam programme. What's new?"

> *Three things. First, they all need dedicated collection — someone drives specifically to survey, or a special vehicle, or a mounted dashcam. Ours is completely passive and rides inside apps that are already running, so our marginal cost per kilometre is zero. Second, NHAI's programmes are **highways**; most pothole deaths happen on city roads, which nobody covers at high frequency. Third, and most importantly, nobody closes the loop: we verify whether a repair actually worked, by watching for the absence of new reports over thousands of subsequent vehicle passes. That audit trail is the product municipalities will actually pay for.*

Q: "Accelerometer pothole detection has been in papers since 2010. Why hasn't it worked?"

> *Because published systems solve detection and stop there. The documented failure modes are all systemic, not model-related: false positives from speed bumps, GPS localisation error of 27 to 32 metres, and vehicle and phone heterogeneity. Our design attacks each one directly — a six-class model that treats speed bumps as their own class instead of noise, map-matching plus 20-metre density clustering instead of naive point averaging, and per-device self-calibration. And we accept that a single sensor reading is nearly worthless; we only act on agreement across many independent vehicles.*

Q: "What's your accuracy?"

> *Two different numbers, and the distinction matters. The on-device sensor model targets around 88 to 95 percent precision on a held-out device — in line with published work, which reports 88.5 percent precision for Random Forest and 91 to 98 percent for a DTW approach across three Indian cities. The vision model targets roughly 70 percent mAP on potholes, which is where published state of the art on RDD2022 sits. But **system-level confidence is much higher than either model**, because we require agreement from at least five distinct devices, plus human votes, plus two independent video confirmations. Stage-wise composition, not single-model excellence, is the design.*

Q: "Doesn't this destroy the battery?"

> *No, and the architecture is specifically built to avoid it. We never run the neural network on the raw stream. A cheap statistical threshold check eliminates roughly 99 percent of windows first, so the model runs about five times a minute rather than fifty times a second. We use no camera in normal operation, no screen, and we reuse the host app's existing GPS fix rather than requesting our own. Network I/O is batched to once a minute. Target is under 2 percent additional battery per hour and under 1 MB of data per day, and we ship a telemetry screen so a partner can verify it themselves.*

Q: "You're secretly recording video from people's phones. That's a privacy nightmare."

> *We agree, which is why we don't do that. Video is a separate opt-in toggle that defaults to OFF, with a visible recording indicator, and it only triggers when the phone is detected as being in a mount. Clips are five to eight seconds, no audio, and we discard them on-device before upload if the frames aren't actually road. Raw video is deleted after seven days; we keep only the cropped defect region. And the more important decision is what we don't collect: we never store continuous location traces, only discrete event points. A continuous trace of a delivery rider is a surveillance dataset. A scatter of pothole hits is not. We designed to the DPDP Act 2023 from the start, including consent versioning and a working erasure endpoint.*

Q: "What if someone fakes reports to get their street repaired?"

Several layers. We count distinct devices, not reports, and we cap any single device's contribution to a cluster at three. Each device carries a trust score that rises when its reports survive to confirmation and falls when they land in rejected clusters. We run physics checks: was the device actually moving above 5 km/h, is the trajectory continuous, is the acceleration signature consistent with vehicle motion, is a mock-location provider enabled. And the final gate is visual — video evidence from a different rider is very hard to fake.

Q: "Why not just use a camera all the time? Cameras are more accurate."

They are more accurate per observation, and far more expensive per observation. Continuous camera use costs battery, mobile data that a gig worker pays for, and a privacy exposure that no platform partner or app store will accept. Our insight is that you don't need accuracy everywhere — you need cheap coverage everywhere and expensive accuracy in the roughly 0.1 percent of places that cheap coverage has already flagged. That's the cost-gated escalation ladder, and it's the core of our design.

Q: "What happens if the municipality just ignores your dashboard?"

That's the failure mode of most civic-tech projects, and it's why the product doesn't stop at a map. We generate work orders, run an SLA clock per defect, and publish ward-level MTTR and failed-repair counts. Failed repairs are detected automatically, without anyone inspecting anything. And we publish confirmed defects publicly, which creates outside pressure. We also sell to auditors, not just to the department being audited — different buyer, different incentive.

Q: "This is a hackathon app. How is it a real product?"

The app is a demo shell; the product is an SDK. The sensing core is already a separate Android module with a documented interface, and we can show a thirty-line host app that consumes it. That's the real artefact — something Swiggy or Ola integrates in an afternoon and that reaches 690,000 riders without a single new download.

Q: "10,000 reports in a 5-metre radius — is that realistic?"

No, and we changed it. That was in our first design and it was wrong on both counts. Five metres is smaller than the GPS error itself — published measurements put localisation error at 27 to 32 metres — so the cluster would never even form. And 10,000 would take months, by which point the pothole has already caused harm. We now use 20-metre map-matched segments and require at least eight reports from at least five distinct devices within seven days. We're happy to walk through why each number changed.

Q: "What's the single biggest technical risk?"

Android's OEM battery managers. Xiaomi, Oppo, Vivo and Realme kill background services regardless of what the platform documentation says, and those brands are a large share of the Indian market. We're testing on real devices from week one rather than on emulators, because this is the kind of problem you cannot discover any other way.

Q: "Why should a delivery company help you?"

Rider welfare is under intense scrutiny right now, and this is a genuine safety feature they can point to — with zero rider effort, no extra downloads, and negligible battery cost. Practically, better road data also improves ETA accuracy on rough routes and reduces vehicle damage and claim costs. And we'd share back a rider-safety routing layer built from the data their own riders generated.

Q: "What if you're wrong about a pothole and a city wastes money?"

Then the uncertainty was mis-stated, which is why we never show a bare pinpoint. Every confirmed defect carries a confidence ellipse, a posterior probability, and an explicit evidence breakdown — how many reports, from how many distinct vehicles, how many human confirmations, how many video verifications. A city engineer can see exactly why we believe something and decide whether that's enough. We're a decision-support system, and we're careful never to present it as more than that.

25 SECTION 25 OF 25

ALL SOURCES

References

25.1 Government & institutional data (India)

1. Ministry of Road Transport and Highways (MoRTH), reply in Lok Sabha on pothole-related crashes, deaths and injuries, 2020–2024 — 9,438 deaths; 1,555 (2020) → 2,385 (2024); accidents 3,713 → 5,432; >19,000 injuries; UP 5,127 deaths. Reported by:
   - Times of India — https://timesofindia.indiatimes.com/india/potholes-killed-9438-from-2020-to-2024-govt/articleshow/128279774.cms
   - Economic Times Auto — https://auto.economictimes.indiatimes.com/news/industry/potholes-killed-9438-from-2020-to-2024-govt/128289161
   - Indian Express — https://indianexpress.com/article/business/pothole-related-road-fatalities-increase-by-53-per-cent-in-5-years-10529465/
   - Economic Times — https://m.economictimes.indiatimes.com/news/india/pothole-deaths-rise-53-in-5-years-up-accounts-for-over-half-of-9400-fatalities/articleshow/128311120.cms
   - OpIndia (year-wise breakdown) — https://www.opindia.com/news-updates/9438-people-died-due-to-pothole-related-road-accidents-in-the-last-five-years/
   - Times Now (accident counts, complaint channels) — https://www.timesnownews.com/auto/potholes-turn-indias-roads-deadlier-accidents-up-53-in-five-years-article-153609742
2. MoRTH — *Road Accidents in India* statistics portal — https://morth.gov.in/ ; accident/fatality tables 2020–2024 — https://morth.gov.in/backend/documents/uploaded/1781177676_V1gUW8tJWT.pdf
3. MoRTH Public Grievances portal — https://morth.nic.in/en/public-grievances
4. World Bank — *India Needs Additional $100-Plus Billion for Safer Roads* (road crashes cost 3–5% of GDP) — https://www.worldbank.org/en/news/press-release/2020/02/20/india-needs-additional-100-plus-billion-for-safer-roads
5. World Bank — *Making India's Roads Safer* (~150,000 killed, ~450,000 injured annually) — https://www.worldbank.org/en/news/video/2022/11/28/making-india-s-roads-safer
6. World Bank — *Traffic Crash Injuries and Disabilities: The Burden on Indian Society* — https://www.worldbank.org/en/country/india/publication/traffic-crash-injuries-and-disabilities-the-burden-on-indian-society ; full report PDF — http://documents1.worldbank.org/curated/en/761181612392067411/pdf/Main-Report.pdf
7. World Bank — halving road deaths 2014–2038 could add ~14% to GDP per capita — https://www.worldbank.org/en/news/speech/2019/10/06/national-road-safety-strategy-india-accidents-death-behavior-change-safe-roads
8. World Bank blog — *How do the poor cope with road crashes in India?* — https://blogs.worldbank.org/en/endpovertyinsouthasia/how-do-poor-cope-road-crashes-india
9. NHAI Network Survey Vehicles across 23 states, ~20,933 km — ET Infra — https://infra.economictimes.indiatimes.com/news/roads-highways/nhai-deploys-network-survey-vehicles-for-road-condition-monitoring-in-23-states/124751754 ; Indian Express explainer — https://indianexpress.com/article/explained/nhai-network-survey-vehicles-10324365/
10. NHAI AI-based dashcam monitoring across ~40,000 km, 30+ defect types — Businessworld — https://www.businessworld.in/article/nhai-plans-ai-based-dashcam-monitoring-across-40-000-km-of-national-highways-598653 ; ET Auto — https://auto.economictimes.indiatimes.com/news/industry/nhai-plans-ai-based-dashcam-monitoring-across-40000-km-of-national-highways/129719720
11. NITI Aayog Frontier Tech Hub — *RoadMetrics Is Using AI to Map and Maintain India's Roads Smarter* (50,000+ km mapped; adopted by Chennai) — https://frontiertech.niti.gov.in/story/ai-driven-road-management-enhancing-indias-infrastructure-with-roadmetrics/
12. NITI Aayog Frontier Tech Hub — AI-driven road safety app (Rakshak) — https://frontiertech.niti.gov.in/story/ai-driven-road-safety-app-a-promising-solution-for-reducing-accident-fatalities-in-india/
13. IndiaAI — *AI-powered road safety initiative launched in Nagpur* (iRASTE; collision avoidance up to 60% reduction claim) — https://indiaai.gov.in/news/ai-powered-road-safety-initiative-launched-in-nagpur
14. IIIT-Hyderabad / INAI — Project iRASTE overview (target ~50% decline in Nagpur road crashes) — https://rndshowcase.iiit.ac.in/tto2025/TTO_website_data/PDF_24/195.pdf

15. IRC:82-2015 — *Code of Practice for Maintenance of Bituminous Road Surfaces*, Indian Roads Congress — http://law.resource.org.s3.amazonaws.com/pub/in/bis/irc/irc.gov.in.082.2015.pdf
16. IRC:SP:83-2018 — maintenance of concrete pavements — https://law.resource.org/pub/in/bis/irc/irc.gov.in.sp.083.2018.pdf
17. Haryana *Mhari Sadak* app + monthly pothole review by DCs — https://timesofindia.indiatimes.com/city/gurgaon/no-end-to-bumpy-rides-govt-orders-monthly-review-of-potholes-by-dcs/articleshow/126327240.cms

25.2 Municipal cost data

18. BMC RTI reply — ₹17,693 to fill one pothole — https://mumbaimirror.indiatimes.com/mumbai/civic/bmc-spends-rs-17693-to-fill-each-pothole-reveals-rti-query/articleshow/70746761.html
19. Bengaluru per-pothole cost variance (₹60,344 vs ₹20,028 across wards) — Bangalore Mirror — https://bangaloremirror.indiatimes.com/bangalore/civic/from-rs-1-lakh-to-rs-1231-bengalurus-pothole-cost-puzzle/articleshow/130620415.cms
20. Mumbai pothole repair tenders ₹203 cr → ₹156 cr → ₹90 cr — TOI — https://timesofindia.indiatimes.com/city/mumbai/pothole-repair-tenders-for-mumbai-roads-drop-55-over-25-bmc/articleshow/130538076.cms ; Indian Express — https://indianexpress.com/article/mumbai/bmc-pothole-tender-criticism-road-concretisation-mumbai-10657011/
21. Nagpur NMC — ₹10 crore / 19,142 potholes over 3 years — https://timesofindia.indiatimes.com/city/nagpur/10cr-in-3yrs-your-money-goes-down-the-pothole/articleshow/105109113.cms
22. Indore IMC — ₹50 crore annual pothole budget — https://timesofindia.indiatimes.com/city/indore/roads-full-of-potholes-make-commuting-difficult-across-city/articleshow/133192369.cms
23. Pune — ₹16 crore/year on road repair vehicles, ₹1.10 crore/year each — https://m.dailyhunt.in/news/india/english/pune+times+mirror-epaper-puntimmr/16+crore+spent+every+year+on+road+repair+vehicles+but+punes+potholes+refuse+to+disappear-newsid-n723024635
24. Bangalore Mirror — *Deadly dent up ahead* — http://bangaloremirror.indiatimes.com/bangalore/cover-story/deadly-dent-up-ahead/articleshow/129427166.cms

25.3 Sensor-based detection research

25. **Khandakar, A., Michelson, D.G., Naznine, M. et al.** (2025) *Harnessing Smartphone Sensors for Enhanced Road Safety: A Comprehensive Dataset and Review*, **Scientific Data 12, 418**. https://doi.org/10.1038/s41597-024-04193-0 — Dataset: https://doi.org/10.6084/m9.figshare.25460755 — Code: https://github.com/naznine/Harnessing-Smartphone-Sensors-for-Enhanced-Road-Safety-A-Comprehensive-Dataset-and-Review
26. **Sattar, S., Li, S. & Chapman, M.** (2018) *Road Surface Monitoring Using Smartphone Sensors: A Review*, **Sensors 18, 3845**. https://doi.org/10.3390/s18113845
27. **Jan, M., Khattak, K.S., Khan, Z.H., Gulliver, T.A. & Altamimi, A.B.** (2023) *Crowdsensing for Road Pavement Condition Monitoring: Trends, Limitations, and Opportunities*, **IEEE Access 11, 133143–133159**. https://doi.org/10.1109/ACCESS.2023.3332667
28. *An Automated Machine-Learning Approach for Road Pothole Detection Using Smartphone Sensor Data* (2020), **Sensors 20(19), 5564** — Random Forest precision 88.5%, recall 75%. https://www.mdpi.com/1424-8220/20/19/5564
29. *Smartphone-Sensor Based Dynamic Time Warping Framework for Enhanced Pothole Detection* (2025), **Journal of The Institution of Engineers (India)** — Delhi 98.04%, Srinagar 97.02%, Rajasthan 91.02%. https://link.springer.com/article/10.1007/s40030-025-00916-7
30. *Efficient pothole detection using smartphone sensors* — NN accuracy 94.78%. https://www.researchgate.net/publication/343284808_Efficient_pothole_detection_using_smartphone_sensors
31. *Smartphone-Based Pothole Detection Utilizing Artificial Neural Networks* (2019), **ASCE Journal of Infrastructure Systems** — ~90% accuracy. https://ascelibrary.org/doi/10.1061/(ASCE)IS.1943-555X.0000489
32. *Road pothole detection from smartphone sensor data using improved LSTM* (2023), **Multimedia Tools and Applications**. https://link.springer.com/article/10.1007/s11042-023-16177-0
33. **Seraj, F., van der Zwaag, B.J., Dilo, A., Luarasi, T. & Havinga, P.** (2016) *RoADS: A Road Pavement Monitoring System for Anomaly Detection Using Smart Phones*. https://doi.org/10.1007/978-3-319-29009-6_7
34. **Celaya-Padilla, J. et al.** (2018) *Speed Bump Detection Using Accelerometric Features: A Genetic Algorithm Approach*, **Sensors 18(2), 443**. https://doi.org/10.3390/s18020443
35. **Fox, A., Kumar, B.V.K.V., Chen, J. & Bai, F.** (2015) *Crowdsourcing undersampled vehicular sensor data for pothole detection*, **IEEE SECON**. https://doi.org/10.1109/SAHCN.2015.7338353
36. **Fox, A. et al.** — multi-lane pothole detection using road inclination and bank angle, **IEEE TMC**. https://doi.org/10.1109/TMC.2017.2690995

37. *Accuracy Enhancement of Anomaly Localization with Participatory Sensing Vehicles* (2020), **Sensors 20(2), 409** — error spread >32 m, mean localisation error >27 m at highway speeds. https://mdpi.com/1424-8220/20/2/409/htm
38. **Martinelli, A. et al.** (2022) *Road Surface Anomaly Assessment Using Low-Cost Accelerometers: A Machine Learning Approach*, **Sensors 22, 3788**. https://doi.org/10.3390/s22103788
39. *Evaluation of data representation techniques for vibration based road surface condition classification* (2024), **Scientific Reports** — 4 classes incl. speedbumps. https://www.nature.com/articles/s41598-024-61757-1
40. *Accelerometer-Based Pavement Classification for Vehicle Dynamics Analysis Using Neural Networks* (2024), **Applied Sciences 14(21), 10027** — NN 100% (overfitting caution), logistic 97.14%. https://www.mdpi.com/2076-3417/14/21/10027
41. **Carlos, M.R. et al.** — Pothole Lab open-access web platform; also *How Smartphone Accelerometers Reveal Aggressive Driving Behavior?—The Key is the Representation*, **IEEE TITS 21, 3377–3387** (2020). https://doi.org/10.1109/TITS.2019.2926639
42. **González, L.C., Moreno, R., Escalante, H.J., Martinez, F. & Carlos, M.R.** — Chihuahua, Mexico dataset (**!** no longer publicly available)
43. *Research on the evaluation and analysis of road surface roughness based on smartphone sensors and SVM* (2026), **Scientific Reports** — 50-m segments, 80–100% classification. https://www.nature.com/articles/s41598-025-34396-3
44. *Influence of surface distresses on smartphone-based pavement roughness evaluation*, **International Journal of Pavement Engineering** — IRI correlation r = 0.862; distresses raise average IRI by 61.8%. https://www.tandfonline.com/doi/full/10.1080/10298436.2020.1714045
45. *Assessing and Mapping of Road Surface Roughness based on GPS and Accelerometer Sensors on Bicycle-Mounted Smartphones* (2018), **Sensors 18(3), 914**. https://www.mdpi.com/1424-8220/18/3/914
46. *Multi-Modal Assessment of Road Roughness Using Smartphone Applications, Acceleration, and Passenger Ratings*, arXiv 2606.03427. https://arxiv.org/html/2606.03427
47. *Measurement of Street Pavement Roughness in Urban Areas Using Smartphone* (2021), **International Journal of Pavement Research and Technology**. https://link.springer.com/article/10.1007/s42947-021-00069-3
48. **Ferreira, J. et al.** (2017) *Driver behavior profiling: An investigation with different smartphone sensors and machine learning*, **PLoS ONE 12(4): e0174959**. https://doi.org/10.1371/journal.pone.0174959
49. *A Crowdsourcing Based Multi-Sensors Fusion Approach* (2023), **Sustainability 15(8), 6610**. https://www.mdpi.com/2071-1050/15/8/6610
50. *Pothole Detection and Analysis System (PoDAS) for Real Time Data Using Sensor Networks*, arXiv 2508.16626. https://arxiv.org/html/2508.16626v1
51. Speed-bump false-positive analysis, **CEUR-WS Vol-2227** (KDD workshop 2018). https://ceur-ws.org/Vol-2227/KDD-UMCit2018-Paper4.pdf
52. *The Citizen Road Watcher – Identifying Roadway Surface Disruptions Based on Accelerometer Patterns* — differentiating potholes, speed bumps, metal humps, rough roads. https://www.researchgate.net/publication/275372091
53. *Accelerometer based road defects identification system*. https://www.researchgate.net/publication/289226869
54. MIT Concrete Sustainability Hub — **Carbin app** research brief (250,000+ miles since 2019). https://cshub.mit.edu/files/2025/06/0708_Carbin_research_brief.pdf
55. *Roughness-induced vehicle energy dissipation from crowdsourced smartphone measurements through random vibration theory*. https://fada.birzeit.edu/bitstream/20.500.11889/6736/1/roughness-induced-vehicle-energy-dissipation-from-crowdsourced-smartphone-measurements-through-random-vibration-theory.pdf
56. *RoadSens-4M: A Multimodal Smartphone & Camera Dataset for Holistic Roadway Analysis* (2026), **Scientific Data**.
57. **Yamansavascilar, B. & Guvensan, M.A.** (2016) *Activity Recognition on Smartphones: Efficient Sampling Rates and Window Sizes*, IEEE PerCom Workshops. https://doi.org/10.1109/PERCOMW.2016.7457154

25.4 Vision-based detection research & datasets

58. **Arya, D., Maeda, H., Ghosh, S.K., Toshniwal, D. & Sekimoto, Y.** (2022) *RDD2022: A multi-national image dataset for automatic Road Damage Detection*, **arXiv:2209.08538** — 47,420 images, 6 countries, 55,000+ instances, 4 damage classes. https://arxiv.org/abs/2209.08538
59. **Arya, D. et al.** — *RDD2022: A multi-national image dataset for automatic road damage detection*, **Geoscience Data Journal 11(4), 846–862**. https://doi.org/10.1002/gdj3.260
60. *Road Damages Detection and Classification with YOLOv7* (CRDDC2022, IEEE BigData) — F1 81.7% (US Street View), 74.1% overall. https://www.researchgate.net/publication/364987880

61. *Road damage detection and classification using deep neural networks* (2024), **Discover Applied Sciences** — 65.7% mAP on RDD2022. https://link.springer.com/article/10.1007/s42452-024-06129-0
62. *YOLOv8-PD: an improved road damage detection algorithm based on YOLOv8n model* (2024), **Scientific Reports** — 2.3M params, 6.1 GFLOPs. https://www.nature.com/articles/s41598-024-62933-z
63. *A High-Precision and Ultra-Lightweight Model for Real-Time Road Damage Detection* (YOLO-ROC), **arXiv:2507.23225** — mAP50 67.6%, D40 +16.8%, 2.0 MB. https://arxiv.org/abs/2507.23225
64. *A Road Damage Detection Method for Effective Pavement Maintenance* (YOLO-RD) (2025), **Sensors 25(5), 1442** — 25.75% on Japan split, +4.93% small objects. https://www.mdpi.com/1424-8220/25/5/1442
65. *A Top-Down Benchmark with Real-Time Framework for Road Damage Detection* (TD-RD), **arXiv:2501.14302** — 7,088 images, 12,882 instances. https://arxiv.org/html/2501.14302v1
66. *Development and Evaluation of a Comprehensive Dataset for Pothole Depth Estimation of Indian Roads Using Smartphone Camera Approach* (2024) — **RAD dataset, Bengaluru**. https://link.springer.com/10.1007/978-981-97-2004-0_34
67. *Sidewalk pothole report improvement through citizen's smartphone* (2025) — SfM-based depth/perimeter with RTK validation. https://link.springer.com/article/10.1007/s12518-025-00686-8
68. *Pothole detection and dimension estimation system using deep learning (YOLO) and image processing* — dataset of 1,243 annotated images. https://github.com/jaygala24/pothole-detection
69. Roboflow — *Automate Pothole Detection with RF-DETR & ByteTrack* (persistent IDs, unique counting, severity). https://blog.roboflow.com/pothole-detection/
70. Roboflow Universe — public pothole object-detection dataset. https://public.roboflow.com/object-detection/pothole/1
71. HuggingFace — `cvtechniques/road-damage-detection-yolov11` pre-trained checkpoint. https://huggingface.co/cvtechniques/road-damage-detection-yolov11
72. `michelpf/dataset-pothole` — 3,125 train / 843 test, YOLO format. https://github.com/michelpf/dataset-pothole
73. `achireistefan/Pothole-Detection` — SoV stereo dataset (447 images with depth maps), Chitholian dataset (665), MIIA Pothole Image Dataset. https://github.com/achireistefan/Pothole-Detection
74. *Smart Pothole Detection and Mapping System using Deep Learning for NMC Application* (2026), **IJRASET** — YOLOv8 + GIS dashboard for Nagpur Municipal Corporation. https://www.ijraset.com/research-paper/smart-pothole-detection-and-mapping-system
75. **Singh, G. et al.** (2023) *ROAD: The Road Event Awareness Dataset for Autonomous Driving*, **IEEE TPAMI 45, 1036–1054**. https://doi.org/10.1109/TPAMI.2022.3150906

25.5 Reference implementations (code)

76. `aswathselvam/Potholes` — realtime Android IMU pothole detection, 50 Hz, SVM in C++ via Java NDK. https://github.com/aswathselvam/Potholes
77. `AdityaPune/Pothole-Detection` — RMS-of-10-readings feature approach. https://github.com/AdityaPune/Pothole-Detection
78. `VishalSingh25/Pothole-Project` — potholes + unmarked speed breakers from phone sensors. https://github.com/VishalSingh25/Pothole-Project
79. Sensor Logger app (used by the Khandakar dataset) — https://play.google.com/store/apps/details?id=com.kelvin.sensorapp ; cross-platform notes — https://github.com/tszheichoi/awesome-sensor-logger/blob/main/CROSSPLATFORM.md
80. `aamend/geoscan` — DBSCAN + Uber H3 geospatial clustering at scale. https://github.com/aamend/geoscan

25.6 Algorithms, platforms, infrastructure

81. **Ester, M., Kriegel, H.-P., Sander, J. & Xu, X.** — DBSCAN; *Density-Based Clustering in Spatial Databases*. https://dl.acm.org/doi/10.1023/A:1009745219419
82. scikit-learn DBSCAN documentation. https://scikit-learn.org/stable/modules/generated/sklearn.cluster.DBSCAN.html
83. Uber Engineering — *H3: Uber's Hexagonal Hierarchical Spatial Index*. https://www.uber.com/en-SE/blog/h3/
84. Esri — Density-based Clustering (DBSCAN vs HDBSCAN) reference. https://pro.arcgis.com/en/pro-app/latest/tool-reference/spatial-statistics/densitybasedclustering.htm
85. Google — **LiteRT: The Universal Framework for On-Device AI** (1.4× faster GPU than TFLite; NPU acceleration). https://developers.googleblog.com/litert-the-universal-framework-for-on-device-ai
86. Google — LiteRT developer documentation. https://developers.google.com/edge/litert
87. Google — LiteRT performance best practices. https://ai.google.dev/edge/litert/conversion/tensorflow/build/best_practices

88. Google — On-Device Training with LiteRT. https://developers.google.com/edge/litert/conversion/tensorflow/build/ondevice_training
89. `google-ai-edge/LiteRT` repository. https://github.com/google-ai-edge/litert
90. Android Developers — Motion sensors. https://developer.android.com/develop/sensors-and-location/sensors/sensors_motion
91. Android Developers — Position sensors. https://developer.android.com/develop/sensors-and-location/sensors/sensors_position
92. Android Developers — `Location` reference. https://developer.android.com/reference/android/location/Location
93. Android Developers Blog — *Improving urban GPS accuracy for your app* (3D-mapping-aided corrections). https://android-developers.googleblog.com/2020/12/improving-urban-gps-accuracy-for-app.html
94. *Semantic VPS for Smartphone Localization in Challenging Urban Environments* (2021), **Sensors 21(18), 6137** — 2.0 m among high-rises, 15.7 m in alleyways. https://mdpi.com/1424-8220/21/18/6137
95. *An Application-Oriented Method Based on Cooperative Map Matching for Improving Vehicular Positioning Accuracy* (2022), **Electronics 11(19), 3258**. https://www.mdpi.com/2079-9292/11/19/3258
96. *Sidewalk matching: a smartphone-based GNSS positioning technique for pedestrians in urban canyons* (2025), **Satellite Navigation**. https://link.springer.com/article/10.1186/s43020-025-00159-8

25.7 Business, market & legal

97. **Digital Personal Data Protection Act, 2023** (Act 22 of 2023) + DPDP Rules notified 13 Nov 2025; penalties up to ₹250 crore, full enforcement 13 May 2027 — https://www.recordinglaw.com/world-laws/world-data-privacy-laws/india-data-privacy-laws
98. DPDP Rules practical guidance — Osano. https://www.osano.com/articles/dpdpa-rules
99. India DPDPA compliance guide (requirements, rights, consent, governance) — OneTrust.
100. DPDP Act comprehensive guide & penalties — BW Legal World. https://www.bwlegalworld.com/article/digital-personal-data-protection-act-2023-comprehensive-guide-compliance-and-penalties-490398
101. DPDP Act consent management & 7-year record retention — miniOrange. https://www.miniorange.com/compliances/dpdp-act
102. Reed Smith — *India in focus: Data protection and AI in India* (extraterritorial applicability). https://www.reedsmith.com/our-insights/blogs/viewpoints/102mshe/india-in-focus-data-protection-and-ai-in-india/
103. **RoadBounce** — company profile (Pune, founded 2016, vibration-based roughness). https://inc42.com/company/roadbounce/ ; https://pitchbook.com/profiles/company/266138-56 ; https://www.cbinsights.com/company/roadbounce/
104. *Testing and evaluation of RoadBounce — mobile phone app based technology for road roughness measurement*. https://www.citedrive.com/en/discovery/testing-and-evaluation-of-roadbounce---mobile-phone-app-based-technology-for-road-roughness-measurement/
105. McKinsey — *Unlocking the full life-cycle value from connected car data* (cities using sensory data to identify potholes). https://www.mckinsey.com/industries/automotive-and-assembly/our-insights/unlocking-the-full-life-cycle-value-from-connected-car-data
106. Fortune Business Insights — Connected Car Data Monetization Service Market (USD 24.6 bn in 2025 → USD 95.1 bn by 2034, 16% CAGR). https://www.fortunebusinessinsights.com/connected-car-data-monetization-service-market-115323
107. Mordor Intelligence — Automotive Data Monetization Market (25.86% CAGR 2026–2031). https://www.mordorintelligence.com/industry-reports/automotive-data-monetization-market
108. Global Market Insights — In-Vehicle Data Monetization Platforms Market (USD 3.4 bn 2025 → USD 11 bn 2035). https://www.gminsights.com/industry-analysis/in-vehicle-data-monetization-platforms-market
109. Cambridge Mobile Telematics — DriveWell (proven smartphone-telematics commercial model). https://www.cmtelematics.com/Safe-Driving-Technology/How-It-Works/
110. India gig workforce: 1.2 crore in FY25, up 55% from 77 lakh in FY21 (Economic Survey) — https://economictimes.indiatimes.com/tech/technology/ride-hailing-gig-worker-unions-call-for-strike-on-february-7/articleshow/127915415.cms
111. Swiggy ~690,000 delivery partners; quick-commerce rider onboarding growth — https://economictimes.com/tech/startups/quick-commerce-ups-gig-rider-onboarding-monthly-base-accelerates-70-80/articleshow/125571540.cms
112. Zomato ~532,000 monthly active delivery partners — https://www.livemint.com/companies/news/gig-workers-trade-unions-food-delivery-quick-commerce-swiggy-zomato-uber-blinkit-labour-codes-11767609650151.html

113. NITI Aayog projection: 2.35 crore gig workers by 2029–30; ~12.7 million currently — https://www.thehindu.com/news/national/swiggy-zomato-magicpin-see-order-surge-on-new-year-eve-negligible-impact-of-gig-workers-strike/article70460313.ece

114. Food delivery sector employment 1.37 million in FY24 (Prosus report) — https://economictimes.indiatimes.com/tech/technology/food-delivery-sector-employment-surges-27-insights-from-prosus-report/articleshow/126055462.cms

115. Outlook Business — *What the numbers say about India's one crore gig workers* — https://www.outlookbusiness.com/news/pay-education-demographics-what-the-numbers-say-about-indias-1-crore-gig-workers

116. Pavement Condition Index methodology references — *Pavement Surface Distress Evaluation Using PCI* (https://www.researchgate.net/publication/325949126) ; Journal of Research in Applied Sciences (https://journalra.org/index.php/jra/article/download/1292/1159)

117. Videonetics — Nagpur Smart City traffic management case study. https://www.videonetics.com/media/pdf/case_studies/nagpur-case-study.pdf

APPENDIX

APPENDIX A — ONE-PAGE CHEAT SHEET (print this)

```
PROBLEM     9,438 pothole deaths in India 2020-24 (+53%). 5,432 accidents in 2024.
            UP alone: 5,127. Several states report ZERO -> the data itself is broken.
            Road crashes cost 3-5% of GDP. BMC pays Rs 17,693 per pothole;
            two Bengaluru wards paid Rs 60,344 vs Rs 20,028 for the same job.

SOLUTION    A 200 KB SDK inside Swiggy/Ola/Rapido driver apps turns 12 million
            gig-worker phones into a passive, daily road inspection network.

THE IDEA    COST-GATED ESCALATION LADDER
            sensors (free, noisy) -> crowd clustering (free, kills FPs)
            -> human votes (1 tap) -> video + YOLO (expensive, 0.1% of places)
            Precision rises at each stage; cost per confirmed defect falls.

NOVEL       1. The 3-layer escalation ladder itself
            2. Per-device + per-vehicle-class self-calibration
            3. "Verified by silence" - repairs confirmed by ABSENCE of new reports
            4. Two-wheeler-weighted severity (weight by who actually dies)
            5. Negative evidence: certify good roads, not just bad ones
            6. Open Indian two-wheeler sensor dataset (SETU-IND-1)

KEY FIXES   5 m radius   -> 20 m map-matched segments (GPS error is 27-32 m!)
TO OUR      10,000 hits  -> 8 reports from 5 DISTINCT devices
FIRST PLAN  auto video   -> opt-in, mount-only, 7-day retention, DPDP-compliant
            notify while riding -> ONLY when stopped (safety-critical)
            mean lat/lon -> DBSCAN + inverse-variance centroid + error ellipse
            hard blacklist -> down-weight x0.2 with escalating TTL

STACK       Kotlin + LiteRT | FastAPI + Postgres/PostGIS + Redis + Celery
            YOLOv8 + ByteTrack | React + deck.gl + Google Maps | Docker

NUMBERS     sensor model  ~88-95% precision (lit: 88.5% RF, 91-98% DTW)
            vision model  ~70% mAP50       (lit SOTA: 65.7% mAP, F1 74-82%)
            system        >90% via multi-stage agreement
            battery       <2%/hour     data: <1 MB/day
            cost          ~Rs 3 to CONFIRM a pothole vs Rs 20,000+ to FIX one

BUSINESS    1 Municipal SaaS (Rs 8L-1.5cr/yr)   2 Contractor audit module
            3 SDK/platform licensing            4 Insurance & telematics
            5 Mapping partners                  6 OEM/ADAS
            GTM: city-owned buses -> reference customer -> SDK partner -> scale
            MOAT: 2 years of repair history cannot be copied in a month
```

End of report.